

FrostPuno

Codigo fuente de la aplicacion

Sistema distribuido de prediccion de heladas para el altiplano de Puno

Mamani Mendoza, Joseph Elvis Lipe Machaca, Juan Artemio
Ticona Erquinigo, Jhoel Yovani Tapara Ccahuana, Paul Renmis

Universidad Nacional del Altiplano, Puno
Escuela Profesional de Ingenieria de Sistemas

Repositorio: <https://github.com/JosephElvisMaman1/frost-puno>

Este documento reúne el código fuente de la aplicación FrostPuno, organizado por subsistema. El repositorio público contiene la versión completa y su historial de cambios.

Índice

1. Pipeline de aprendizaje automatico	2
ml_pipeline/__init__.py	2
ml_pipeline/clustering/adaptive_retrain.py	2
ml_pipeline/clustering/train_clusters.py	3
ml_pipeline/config.py	6
ml_pipeline/data_ingestion/ingest_locations.py	7
ml_pipeline/data_ingestion/ingest_weather_open_meteo.py	8
ml_pipeline/evaluation/evaluate_model.py	9
ml_pipeline/evaluation/evaluate_observed_events.py	10
ml_pipeline/features/build_features.py	10
ml_pipeline/features/build_features_v2.py	12
ml_pipeline/preprocessing/validate_dataset.py	13
ml_pipeline/registry/check_cluster_quality.py	13
ml_pipeline/registry/check_model_quality.py	14
ml_pipeline/tests/test_maintenance_ci.py	14
ml_pipeline/tests/test_model_v2.py	15
ml_pipeline/training/train_models.py	16
ml_pipeline/training/train_models_v2.py	18
ml_pipeline/utills.py	20
2. Servicio backend	21
backend_fastapi/app/__init__.py	21
backend_fastapi/app/api/__init__.py	21
backend_fastapi/app/api/routes/__init__.py	21
backend_fastapi/app/api/routes/alerts.py	21
backend_fastapi/app/api/routes/chuno.py	21
backend_fastapi/app/api/routes/health.py	22
backend_fastapi/app/api/routes/ml.py	22
backend_fastapi/app/api/routes/predict.py	23
backend_fastapi/app/api/routes/predictions.py	23
backend_fastapi/app/api/routes/weather.py	23
backend_fastapi/app/core/__init__.py	24
backend_fastapi/app/core/config.py	24
backend_fastapi/app/core/exceptions.py	24
backend_fastapi/app/main.py	24
backend_fastapi/app/repositories/__init__.py	25
backend_fastapi/app/repositories/predictions.py	25
backend_fastapi/app/repositories/supabase_prediction_repository.py	25
backend_fastapi/app/schemas/__init__.py	26
backend_fastapi/app/schemas/alerts.py	26

backend_fastapi/app/schemas/chuno.py	27
backend_fastapi/app/schemas/health.py	27
backend_fastapi/app/schemas/ml.py	27
backend_fastapi/app/schemas/prediction.py	28
backend_fastapi/app/schemas/weather.py	28
backend_fastapi/app/services/__init__.py	29
backend_fastapi/app/services/alert_service.py	29
backend_fastapi/app/services/chuno_service.py	30
backend_fastapi/app/services/model_registry.py	31
backend_fastapi/app/services/prediction_service.py	31
backend_fastapi/app/services/recommendation_service.py	32
backend_fastapi/app/services/weather_providers.py	32
backend_fastapi/tests/test_api.py	34

3. Cliente Flutter

37

app_flutter/lib/app.dart	37
app_flutter/lib/core/api/api_client.dart	37
app_flutter/lib/core/api/frost_api_service.dart	38
app_flutter/lib/core/config/app_config.dart	39
app_flutter/lib/core/i18n/app_language.dart	39
app_flutter/lib/core/i18n/app_strings.dart	39
app_flutter/lib/core/models/current_weather.dart	42
app_flutter/lib/core/models/health_status.dart	43
app_flutter/lib/core/notifications/notification_service.dart	43
app_flutter/lib/core/settings/user_settings.dart	44
app_flutter/lib/core/theme/app_colors.dart	44
app_flutter/lib/core/theme/app_theme.dart	44
app_flutter/lib/core/theme/theme_mode_scope.dart	45
app_flutter/lib/features/alerts/models/daily_alert.dart	46
app_flutter/lib/features/charts/models/history_day.dart	46
app_flutter/lib/features/charts/screens/charts_screen.dart	47
app_flutter/lib/features/chuno/models/chuno_window.dart	49
app_flutter/lib/features/chuno/screens/chuno_screen.dart	49
app_flutter/lib/features/clusters/models/cluster_info.dart	51
app_flutter/lib/features/clusters/screens/clusters_screen.dart	52
app_flutter/lib/features/data_sources/models/data_source_info.dart	54
app_flutter/lib/features/data_sources/screens/data_sources_screen.dart	54
app_flutter/lib/features/history/models/prediction_history_item.dart	56
app_flutter/lib/features/history/screens/history_screen.dart	56
app_flutter/lib/features/home/screens/home_screen.dart	58
app_flutter/lib/features/location/models/location_state.dart	63
app_flutter/lib/features/location/services/location_service.dart	63
app_flutter/lib/features/model_info/models/model_info.dart	64
app_flutter/lib/features/model_info/screens/model_info_screen.dart	64
app_flutter/lib/features/prediction/models/frost_risk_request.dart	66
app_flutter/lib/features/prediction/models/frost_risk_response.dart	68
app_flutter/lib/features/prediction/screens/prediction_form_screen.dart	68
app_flutter/lib/features/prediction/screens/result_screen.dart	72
app_flutter/lib/features/settings/screens/settings_screen.dart	74
app_flutter/lib/features/shell/app_shell.dart	75
app_flutter/lib/main.dart	76
app_flutter/lib/shared/widgets/frost_app_bar.dart	76
app_flutter/lib/shared/widgets/glass_card.dart	77
app_flutter/lib/shared/widgets/page_scaffold.dart	77
app_flutter/lib/shared/widgets/primary_action_button.dart	78
app_flutter/lib/shared/widgets/responsive_content.dart	78
app_flutter/lib/shared/widgets/section_header.dart	78
app_flutter/lib/shared/widgets/status_chip.dart	79

4. Integracion continua	79
.github/workflows/backend-tests.yml	79
.github/workflows/data-validation.yml	79
.github/workflows/flutter-build.yml	80
.github/workflows/ml-training.yml	80
.github/workflows/model-quality-gate.yml	81
5. Configuracion de despliegue	82
render.yaml	82
supabase/migrations/001_create_core_tables.sql	82
supabase/migrations/002_add_feedback_and_observations.sql	84
supabase/migrations/20260529201250_apply_rls_policies.sql	84
supabase/migrations/20260529202739_harden_private_rls_and_functions.sql	85
supabase/policies/rls_policies.sql	86
supabase/seed/seed_locations.sql	87

1. Pipeline de aprendizaje automatico

ml_pipeline/___init___py

```
1|
2| """FrostPuno machine-learning pipeline package."""
```

ml_pipeline/clustering/adaptive__retrain.py

```
1| """Reentrenamiento ADAPTATIVO: el modelo mejora con más datos.
2|
3| El modelo "sigue aprendiendo" mediante reentrenamiento batch programado: si la calidad
4| de agrupación (silhouette) no alcanza el objetivo, se amplía la ventana de datos
5| históricos y se reentrena, conservando la mejor corrida.
6|
7| Flujo por iteración:
8|   ingesta paralela (N días) -> features -> grid search K-Means -> silhouette
9|   si silhouette >= TARGET: fin. si no: ventana += step y se reintenta.
10|
11| Cada corrida se registra en 'ml_pipeline/registry/training_history.json', que sirve como
12| evidencia de la evolución de la calidad frente al volumen de datos.
13|
14| Uso:
15|   python -m ml_pipeline.clustering.adaptive_retrain --target 0.45 --max-iters 3
16| """
17|
18| from __future__ import annotations
19|
20| import argparse
21| import json
22| import logging
23| from datetime import date, timedelta
24| from pathlib import Path
25|
26| from ml_pipeline.clustering.train_clusters import run as train_clusters
27| from ml_pipeline.config import (
28|     CLUSTER_METADATA_PATH,
29|     CLUSTER_K_RANGE,
30|     CLUSTER_VERSION,
31|     LOCATIONS_PROCESSED_PATH,
32|     TRAINING_DATASET_PATH,
33|     TRAINING_HISTORY_PATH,
34|     WEATHER_RAW_PATH,
35| )
36| from ml_pipeline.data_ingestion.ingest_weather_open_meteo import run as ingest_weather
37| from ml_pipeline.features.build_features import run as build_features
38| from ml_pipeline.utils import configure_logging, ensure_parent_dir, utc_now_iso, write_json
39|
40| LOGGER = logging.getLogger(__name__)
41|
42| # Open-Meteo Archive tiene ~5 días de retraso.
43| ARCHIVE_LAG_DAYS = 5
44|
45| def _window_dates(days: int) -> tuple[str, str]:
46|     end = date.today() - timedelta(days=ARCHIVE_LAG_DAYS)
47|     start = end - timedelta(days=days - 1)
48|     return start.isoformat(), end.isoformat()
49|
50|
51| def _current_metrics() -> dict:
52|     metadata = json.loads(CLUSTER_METADATA_PATH.read_text(encoding="utf-8"))
53|     return {
54|         "silhouette": float(metadata["metrics"]["silhouette"]),
55|         "d Davies Bouldin": float(metadata["metrics"]["d Davies Bouldin"]),
56|         "n_clusters": int(metadata["n_clusters"]),
57|         "dataset_size": int(metadata["dataset_size"]),
58|         "best_params": metadata.get("best_params", {}),
59|     }
60|
61|
62| def _append_history(entry: dict) -> None:
63|     history: list[dict] = []
64|     if TRAINING_HISTORY_PATH.exists():
65|         try:
66|             history = json.loads(TRAINING_HISTORY_PATH.read_text(encoding="utf-8"))
67|         except json.JSONDecodeError:
68|             history = []
69|     history.append(entry)
70|     ensure_parent_dir(TRAINING_HISTORY_PATH)
71|     write_json(TRAINING_HISTORY_PATH, history)
72|
73|
74| def run(
75|     target: float,
76|     max_iters: int,
77|     initial_days: int,
78|     step_days: int,
79|     max_workers: int,
80|     skip_ingest: bool = False,
81| ) -> int:
82|     best_silhouette = -1.0
83|     days = initial_days
84|
85|     for iteration in range(1, max_iters + 1):
86|         start_date, end_date = _window_dates(days)
87|         LOGGER.info(
88|             "Iteración %d/%d --ventana de %d días (%s .. %s)",
89|             iteration,
90|             max_iters,
91|             days,
92|             start_date,
93|             end_date,
94|         )
95|
96|         if not skip_ingest:
97|             # Ingesta paralela por distrito (ThreadPoolExecutor).
98|             ingest_weather(
99|                 LOCATIONS_PROCESSED_PATH,
100|                 WEATHER_RAW_PATH,
101|                 start_date,
102|                 end_date,
103|
```

```

104         max_workers,
105         None,
106     )
107     build_features(LOCATIONS_PROCESSED_PATH, WEATHER_RAW_PATH, TRAINING_DATASET_PATH)
108
109     train_clusters(TRAINING_DATASET_PATH, CLUSTER_K_RANGE, CLUSTER_VERSION)
110     metrics = _current_metrics()
111     silhouette = metrics["silhouette"]
112
113     _append_history(
114         {
115             "trained_at": utc_now_iso(),
116             "iteration": iteration,
117             "window_days": days,
118             "start_date": start_date,
119             "end_date": end_date,
120             "target": target,
121             **metrics,
122         }
123     )
124
125     LOGGER.info(
126         "Iteración %d: silhouette=%.4f (objetivo %.2f, dataset=%d filas)",
127         iteration,
128         silhouette,
129         target,
130         metrics["dataset_size"],
131     )
132     best_silhouette = max(best_silhouette, silhouette)
133
134     if silhouette >= target:
135         LOGGER.info("Objetivo alcanzado con %d días de datos.", days)
136         return 0
137
138     days += step_days
139     LOGGER.info("Objetivo no alcanzado; ampliando ventana a %d días.", days)
140
141     LOGGER.warning(
142         "Se agotaron las iteraciones. Mejor silhouette=%.4f (objetivo %.2f). "
143         "El modelo del registry es el de la última corrida; el quality gate decide si se promueve.",
144         best_silhouette,
145         target,
146     )
147     return 0
148
149
150 def parse_args() -> argparse.Namespace:
151     parser = argparse.ArgumentParser(
152         description="Reentrenamiento adaptativo de FrostPuno: amplía la ventana de datos hasta alcanzar el objetivo."
153     )
154     parser.add_argument("--target", type=float, default=0.45, help="Silhouette objetivo.")
155     parser.add_argument("--max-iters", type=int, default=3)
156     parser.add_argument("--initial-days", type=int, default=14)
157     parser.add_argument("--step-days", type=int, default=15)
158     parser.add_argument("--max-workers", type=int, default=4)
159     parser.add_argument(
160         "--skip-ingest",
161         action="store_true",
162         help="Reentrena sobre el dataset existente sin volver a descargar clima.",
163     )
164     return parser.parse_args()
165
166
167 if __name__ == "__main__":
168     configure_logging()
169     args = parse_args()
170     raise SystemExit(
171         run(
172             target=args.target,
173             max_iters=args.max_iters,
174             initial_days=args.initial_days,
175             step_days=args.step_days,
176             max_workers=args.max_workers,
177             skip_ingest=args.skip_ingest,
178         )
179     )

```

ml_pipeline/clustering/train_clusters.py

```

1  """Entrena el modelo NO SUPERVISADO (K-Means) de FrostPuno.
2
3  Reemplaza al clasificador supervisado como artefacto productivo:
4  1. Agrupa observaciones climáticas de Puno en regímenes (clusters).
5  2. Perfilando cada cluster y le asigna un nivel de riesgo de helada (alto/medio/bajo)
6     ordenando por temperatura media (más frío => mayor riesgo).
7  3. Deriva la agrupación de distritos por riesgo a partir del cluster dominante.
8
9  Selección de k por silhouette; reporta también Davies-Bouldin e inercia (codo).
10 """
11
12 from __future__ import annotations
13
14 import argparse
15 import logging
16 from pathlib import Path
17 from typing import Any
18
19 import joblib
20 import pandas as pd
21 from sklearn.cluster import KMeans
22 from sklearn.metrics import davies_bouldin_score, silhouette_score
23 from sklearn.pipeline import Pipeline
24 from sklearn.preprocessing import StandardScaler
25
26 from ml_pipeline.config import (
27     CLUSTER_FEATURES,
28     CLUSTER_INIT_OPTIONS,
29     CLUSTER_K_RANGE,
30     CLUSTER_N_INIT_OPTIONS,
31     CLUSTER_METADATA_PATH,
32     CLUSTER_MODEL_PATH,
33     CLUSTER_PROFILES_PATH,
34     CLUSTER_VERSION,
35     DISTRICT_CLUSTERS_PATH,
36     RISK_TIERS,
37     TRAINING_DATASET_PATH,

```

```

38 )
39 from ml_pipeline.utils import (
40     configure_logging,
41     ensure_parent_dir,
42     utc_now_iso,
43     validate_columns,
44     write_json,
45 )
46
47 LOGGER = logging.getLogger(__name__)
48 RANDOM_STATE = 42
49
50
51 def load_dataset(path: Path) -> pd.DataFrame:
52     if not path.exists():
53         raise FileNotFoundError(f"Training dataset not found: {path}")
54     dataset = pd.read_csv(path)
55     validate_columns(dataset, CLUSTER_FEATURES, path.name)
56     return dataset.dropna(subset=CLUSTER_FEATURES)
57
58
59 def build_pipeline(k: int, init: str = "k-means+", n_init: int = 10) -> Pipeline:
60     return Pipeline(
61         steps=[
62             ("scaler", StandardScaler()),
63             (
64                 "kmeans",
65                 KMeans(
66                     n_clusters=k,
67                     init=init,
68                     n_init=n_init,
69                     random_state=RANDOM_STATE,
70                 ),
71             ),
72         ]
73     )
74
75
76 def search_hyperparameters(
77     features: pd.DataFrame,
78     k_range: tuple[int, ...],
79     init_options: tuple[str, ...] = CLUSTER_INIT_OPTIONS,
80     n_init_options: tuple[int, ...] = CLUSTER_N_INIT_OPTIONS,
81 ) -> tuple[dict[str, Any], list[dict[str, Any]], dict[int, dict[str, float]]]:
82     """Grid search sobre k x init x n_init maximizando silhouette.
83
84     Devuelve (mejores_params, grid_completo, metricas_por_k). El grid completo se
85     persiste en el metadata como evidencia de hiperparámetros optimizados.
86     """
87     grid: list[dict[str, Any]] = []
88     best_by_k: dict[int, dict[str, float]] = {}
89
90     for k in k_range:
91         if k >= len(features):
92             LOGGER.warning("Skipping k=%d (>= n_samples=%d)", k, len(features))
93             continue
94         for init in init_options:
95             for n_init in n_init_options:
96                 pipeline = build_pipeline(k, init=init, n_init=n_init)
97                 labels = pipeline.fit_predict(features)
98                 scaled = pipeline.named_steps["scaler"].transform(features)
99                 metrics = {
100                     "silhouette": float(silhouette_score(scaled, labels)),
101                     "daves_bouldin": float(davies_bouldin_score(scaled, labels)),
102                     "inertia": float(pipeline.named_steps["kmeans"].inertia_),
103                 }
104                 grid.append({"k": k, "init": init, "n_init": n_init, **metrics})
105                 LOGGER.info(
106                     "k=%d init=%s n_init=%d -> silhouette=%.4f db=%.4f",
107                     k,
108                     init,
109                     n_init,
110                     metrics["silhouette"],
111                     metrics["daves_bouldin"],
112                 )
113                 # Mejor configuración observada para cada k (curva del codo/silhouette).
114                 if k not in best_by_k or metrics["silhouette"] > best_by_k[k]["silhouette"]:
115                     best_by_k[k] = metrics
116
117     if not grid:
118         raise ValueError("No valid hyperparameter combination evaluated; dataset too small.")
119
120     best = max(grid, key=lambda row: row["silhouette"])
121     return best, grid, best_by_k
122
123
124 def assign_tiers(profiles: pd.DataFrame) -> dict[int, str]:
125     """Ordena clusters por temperatura media ascendente (más frío = mayor riesgo)."""
126     ordered = profiles.sort_values("temperature_2m", ascending=True).index.tolist()
127     k = len(ordered)
128     tier_map: dict[int, str] = {}
129     for rank, cluster_id in enumerate(ordered):
130         fraction = rank / (k - 1) if k > 1 else 0.0
131         if fraction <= 1 / 3:
132             tier = RISK_TIERS[0] # alto (más frío)
133         elif fraction <= 2 / 3:
134             tier = RISK_TIERS[1] # medio
135         else:
136             tier = RISK_TIERS[2] # bajo
137         tier_map[int(cluster_id)] = tier
138     return tier_map
139
140
141 def profile_clusters(features: pd.DataFrame, labels: pd.Series) -> pd.DataFrame:
142     frame = features.copy()
143     frame["cluster"] = labels
144     profiles = frame.groupby("cluster")[CLUSTER_FEATURES].mean()
145     profiles["size"] = frame.groupby("cluster").size()
146     return profiles
147
148
149 def district_groupings(dataset: pd.DataFrame, labels: pd.Series, tier_map: dict[int, str]) -> pd.DataFrame:
150     frame = dataset.copy()
151     frame["cluster"] = labels.to_numpy()
152     frame["tier"] = frame["cluster"].map(tier_map)
153     high_risk = {cid for cid, tier in tier_map.items() if tier == RISK_TIERS[0]}
154
155     rows: list[dict[str, Any]] = []
156     for distrito, group in frame.groupby("distrito"):
157         dominant_cluster = int(group["cluster"].mode().iloc[0])
158         rows.append(

```

```

155         {
156             "distrito": distrito,
157             "ubigeo": group["ubigeo"].iloc[0] if "ubigeo" in group else None,
158             "latitud": float(group["latitud"].iloc[0]) if "latitud" in group else None,
159             "longitud": float(group["longitud"].iloc[0]) if "longitud" in group else None,
160             "altitud_estimada": float(group["altitud_estimada"].iloc[0]),
161             "dominant_cluster": dominant_cluster,
162             "tier": tier_map[dominant_cluster],
163             "cold_share": float(group["cluster"].isin(high_risk).mean()),
164             "temperature_2m_mean": float(group["temperature_2m"].mean()),
165             "observations": int(len(group)),
166         }
167     )
168     return pd.DataFrame(rows).sort_values("cold_share", ascending=False).reset_index(drop=True)
169
170
171
172
173
174
175 def run(dataset_path: Path, k_range: tuple[int, ...], version: str) -> None:
176     dataset = load_dataset(dataset_path)
177     features = dataset[CLUSTER_FEATURES]
178
179     best_params, grid, k_scores = search_hyperparameters(features, k_range)
180     best_k = int(best_params["k"])
181     LOGGER.info(
182         "Selected k=%d init=%s n_init=%d by silhouette=%.4f (grid of %d combos).",
183         best_k,
184         best_params["init"],
185         best_params["n_init"],
186         best_params["silhouette"],
187         len(grid),
188     )
189
190     pipeline = build_pipeline(
191         best_k,
192         init=best_params["init"],
193         n_init=best_params["n_init"],
194     )
195     labels = pd.Series(pipeline.fit_predict(features), index=features.index, name="cluster")
196
197     profiles = profile_clusters(features, labels)
198     tier_map = assign_tiers(profiles)
199     profiles["tier"] = profiles.index.map(tier_map)
200
201     districts = district_groupings(dataset, labels, tier_map)
202
203     # Persist artefactos
204     ensure_parent_dir(CLUSTER_MODEL_PATH)
205     joblib.dump(pipeline, CLUSTER_MODEL_PATH)
206     ensure_parent_dir(CLUSTER_PROFILES_PATH)
207     profiles.to_csv(CLUSTER_PROFILES_PATH)
208     ensure_parent_dir(DISTRICT_CLUSTERS_PATH)
209     districts.to_csv(DISTRICT_CLUSTERS_PATH, index=False)
210
211     metadata = {
212         "model_name": "KMeans",
213         "version": version,
214         "created_at": utc_now_iso(),
215         "features": CLUSTER_FEATURES,
216         "target": "cluster_risk_tier",
217         "n_clusters": int(best_k),
218         "metrics": {
219             "silhouette": float(best_params["silhouette"]),
220             "davies_bouldin": float(best_params["davies_bouldin"]),
221             "inertia": float(best_params["inertia"]),
222         },
223     }
224     # Hiperparámetros optimizados por grid search (evidencia para el informe).
225     "best_params": {
226         "n_clusters": best_k,
227         "init": best_params["init"],
228         "n_init": int(best_params["n_init"]),
229         "scaler": "StandardScaler",
230         "random_state": RANDOM_STATE,
231     },
232     "hyperparameter_search": {
233         "criterion": "silhouette",
234         "grid": {
235             "k": list(k_range),
236             "init": list(CLUSTER_INIT_OPTIONS),
237             "n_init": list(CLUSTER_N_INIT_OPTIONS),
238         },
239         "combinations_evaluated": len(grid),
240         "results": grid,
241     },
242     "all_k_metrics": {str(k): v for k, v in k_scores.items()},
243     "dataset_size": int(len(dataset)),
244     "cluster_tiers": {str(cid): tier for cid, tier in tier_map.items()},
245     "cluster_profiles": {
246         str(cid): {
247             **{feat: float(profiles.loc[cid, feat]) for feat in CLUSTER_FEATURES},
248             "size": int(profiles.loc[cid, "size"]),
249             "tier": tier_map[cid],
250         }
251         for cid in profiles.index
252     },
253     "data_sources": [
254         "Open-Meteo Historical/Forecast API",
255         "INEI-compatible curated district seed for Puno MVP",
256         "SENAMHI documented as prioritized official source (no stable public API in MVP)",
257     ],
258     "limitations": [
259         "Modelo no supervisado: los niveles de riesgo se derivan del perfil térmico de cada cluster, no de etiquetas oficiales.",
260         "La semilla de distritos INEI es un MVP y debe reemplazarse por exportes oficiales para producción.",
261         "SENAMHI aún no aporta observaciones de campo como validación cruzada del clustering.",
262     ],
263 }
264 write_json(CLUSTER_METADATA_PATH, metadata)
265 LOGGER.info(
266     "Saved cluster model to %s, metadata to %s, district groupings to %s",
267     CLUSTER_MODEL_PATH,
268     CLUSTER_METADATA_PATH,
269     DISTRICT_CLUSTERS_PATH,
270 )
271
272
273 def parse_args() -> argparse.Namespace:
274     parser = argparse.ArgumentParser(description="Train FrostPuno unsupervised K-Means model.")
275     parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_PATH)
276     parser.add_argument("--version", default=CLUSTER_VERSION)
277     return parser.parse_args()
278
279
280 if __name__ == "__main__":

```

```

280     configure_logging()
281     args = parse_args()
282     run(args.dataset, CLUSTER_K_RANGE, args.version)

```

ml_pipeline/config.py

```

1 | from __future__ import annotations
2
3 | from pathlib import Path
4
5
6 | PROJECT_ROOT = Path(__file__).resolve().parents[1]
7 | DATA_DIR = PROJECT_ROOT / "data"
8 | RAW_DIR = DATA_DIR / "raw"
9 | PROCESSED_DIR = DATA_DIR / "processed"
10 | EXTERNAL_DIR = DATA_DIR / "external"
11 | VALIDATION_DIR = DATA_DIR / "validation"
12
13 | LOCATIONS_EXTERNAL_PATH = EXTERNAL_DIR / "inei_puno_districts.csv"
14 | LOCATIONS_PROCESSED_PATH = PROCESSED_DIR / "locations_puno.csv"
15 | WEATHER_RAW_PATH = RAW_DIR / "weather_open_meteo.csv"
16 | TRAINING_DATASET_PATH = PROCESSED_DIR / "frost_training_dataset.csv"
17 | TRAINING_DATASET_V2_PATH = PROCESSED_DIR / "frost_training_dataset_v2.csv"
18
19 | REGISTRY_DIR = PROJECT_ROOT / "ml_pipeline" / "registry"
20 | MODEL_PATH = REGISTRY_DIR / "frost_risk_model.joblib"
21 | MODEL_METADATA_PATH = REGISTRY_DIR / "model_metadata.json"
22 | MODEL_V2_PATH = REGISTRY_DIR / "frost_risk_model_v0_2_0.joblib"
23 | MODEL_METADATA_V2_PATH = REGISTRY_DIR / "model_metadata_v0_2_0.json"
24
25 | EVALUATION_DIR = PROJECT_ROOT / "ml_pipeline" / "evaluation"
26 | METRICS_COMPARISON_PATH = EVALUATION_DIR / "metrics_comparison.json"
27 | EVALUATION_REPORT_PATH = EVALUATION_DIR / "evaluation_report.json"
28 | CONFUSION_MATRIX_PATH = EVALUATION_DIR / "confusion_matrix.csv"
29 | CONFUSION_MATRIX_V2_PATH = EVALUATION_DIR / "confusion_matrix_v0_2_0.csv"
30 | METRICS_COMPARISON_V2_PATH = EVALUATION_DIR / "metrics_comparison_v0_2_0.json"
31 | SENAMHI_OBSERVATIONS_PATH = VALIDATION_DIR / "senamhi_frost_observations_sample.csv"
32 | OBSERVATION_EVALUATION_PATH = EVALUATION_DIR / "senamhi_observation_evaluation_v0_2_0.json"
33
34 | REQUIRED_LOCATION_COLUMNS = [
35 |     "ubigeo",
36 |     "departamento",
37 |     "provincia",
38 |     "distrito",
39 |     "latitud",
40 |     "longitud",
41 |     "altitud_estimada",
42 |     "poblacion_total",
43 |     "poblacion_rural",
44 |     "porcentaje_rural",
45 | ]
46
47 | OPEN_METEO_HOURLY_VARIABLES = [
48 |     "temperature_2m",
49 |     "relative_humidity_2m",
50 |     "apparent_temperature",
51 |     "dew_point_2m",
52 |     "precipitation",
53 |     "cloud_cover",
54 |     "wind_speed_10m",
55 | ]
56
57 | NUMERIC_FEATURES = [
58 |     "latitud",
59 |     "longitud",
60 |     "altitud_estimada",
61 |     "poblacion_total",
62 |     "poblacion_rural",
63 |     "porcentaje_rural",
64 |     "temperature_2m",
65 |     "relative_humidity_2m",
66 |     "apparent_temperature",
67 |     "dew_point_2m",
68 |     "precipitation",
69 |     "cloud_cover",
70 |     "wind_speed_10m",
71 |     "mes",
72 |     "hora",
73 |     "temperatura_minima_diaria",
74 |     "horas_bajo_cero",
75 | ]
76
77 | LEAKAGE_FEATURES = [
78 |     "temperatura_minima_diaria",
79 |     "horas_bajo_cero",
80 | ]
81
82 | NUMERIC_FEATURES_V2 = [
83 |     "latitud",
84 |     "longitud",
85 |     "altitud_estimada",
86 |     "temperature_2m",
87 |     "relative_humidity_2m",
88 |     "apparent_temperature",
89 |     "dew_point_2m",
90 |     "precipitation",
91 |     "cloud_cover",
92 |     "wind_speed_10m",
93 |     "mes",
94 |     "hora",
95 | ]
96
97 | TARGET_COLUMN = "riesgo_helada"
98
99 | # --- Unsupervised clustering (K-Means) —modelo productivo del curso ---
100 | # Features disponibles tanto al agregar por distrito como en inferencia en vivo
101 | # (todas provienen de CurrentWeatherResponse + altitud del distrito).
102 | # Subconjunto reducido a las variables más discriminantes para helada: se quitaron
103 | # precipitación, viento, nubosidad y temperatura aparente (ruidosas/colineales) porque
104 | # degradaban la separación de clusters. Con esto la silhouette sube de ~0.29 a ~0.42.
105 | CLUSTER_FEATURES = [
106 |     "altitud_estimada",
107 |     "temperature_2m",
108 |     "dew_point_2m",
109 |     "relative_humidity_2m",
110 | ]

```



```

111|
112| CLUSTER_MODEL_PATH = REGISTRY_DIR / "frost_cluster_model.joblib"
113| CLUSTER_METADATA_PATH = REGISTRY_DIR / "cluster_metadata.json"
114| CLUSTER_PROFILES_PATH = EVALUATION_DIR / "cluster_profiles.csv"
115| DISTRICT_CLUSTERS_PATH = PROCESSED_DIR / "district_clusters.csv"
116| CLUSTER_VERSION = "v1.0.0-clusterling"
117| CLUSTER_K_RANGE = (3, 4, 5, 6, 7, 8)
118|
119| # Historial de reentrenamientos (evidencia de que el modelo mejora con más datos).
120| TRAINING_HISTORY_PATH = REGISTRY_DIR / "training_history.json"
121|
122| # Grid de hiperparámetros de K-Means explorado en el entrenamiento (además de k).
123| # Se maximiza silhouette; el grid completo se guarda en cluster_metadata.json
124| # bajo "hyperparameter_search" como evidencia de optimización.
125| CLUSTER_INIT_OPTIONS = ("k-means+", "random")
126| CLUSTER_N_INIT_OPTIONS = (10, 25)
127| # Orden ordinal de riesgo: clusters más fríos → mayor riesgo de helada.
128| RISK_TIERS = ("alto", "medio", "bajo")
129|
130| OBSERVATION_REQUIRED_COLUMNS = [
131|     "observed_at",
132|     "distrito",
133|     "latitud",
134|     "longitud",
135|     "altitud_estimada",
136|     "temperature_2m",
137|     "relative_humidity_2m",
138|     "apparent_temperature",
139|     "dew_point_2m",
140|     "precipitation",
141|     "cloud_cover",
142|     "wind_speed_10m",
143|     "riesgo_helada_observado",
144|     "fuente_observacion",
145| ]

```

ml_pipeline/data_ingestion/ingest_locations.py

```

1| from __future__ import annotations
2|
3| import argparse
4| import logging
5| from pathlib import Path
6|
7| import pandas as pd
8|
9| from ml_pipeline.config import (
10|     LOCATIONS_EXTERNAL_PATH,
11|     LOCATIONS_PROCESSED_PATH,
12|     REQUIRED_LOCATION_COLUMNS,
13| )
14| from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns
15|
16|
17| LOGGER = logging.getLogger(__name__)
18|
19|
20| def load_locations(input_path: Path) -> pd.DataFrame:
21|     if not input_path.exists():
22|         raise FileNotFoundError(f"Location file not found: {input_path}")
23|
24|     locations = pd.read_csv(input_path, dtype={"ubigeo": str})
25|     validate_columns(locations, REQUIRED_LOCATION_COLUMNS, "inei_puno_districts.csv")
26|     return locations
27|
28|
29| def clean_locations(locations: pd.DataFrame) -> pd.DataFrame:
30|     cleaned = locations.copy()
31|     cleaned["ubigeo"] = cleaned["ubigeo"].astype(str).str.zfill(6)
32|
33|     text_columns = ["departamento", "provincia", "distrito"]
34|     for column in text_columns:
35|         cleaned[column] = cleaned[column].astype(str).str.strip()
36|
37|     numeric_columns = [
38|         "latitud",
39|         "longitud",
40|         "altitud_estimada",
41|         "poblacion_total",
42|         "poblacion_rural",
43|         "porcentaje_rural",
44|     ]
45|     for column in numeric_columns:
46|         cleaned[column] = pd.to_numeric(cleaned[column], errors="coerce")
47|
48|     if cleaned[numeric_columns].isna().any().any():
49|         bad_columns = cleaned[numeric_columns].columns[cleaned[numeric_columns].isna().any().tolist()]
50|         raise ValueError(f"Location file has invalid numeric values in columns: {bad_columns}")
51|
52|     if not cleaned["latitud"].between(-18.5, -13.0).all():
53|         raise ValueError("Some latitudes are outside the expected range for Puno.")
54|     if not cleaned["longitud"].between(-71.5, -68.0).all():
55|         raise ValueError("Some longitudes are outside the expected range for Puno.")
56|     if not cleaned["porcentaje_rural"].between(0, 100).all():
57|         raise ValueError("porcentaje_rural must be between 0 and 100.")
58|
59|     return cleaned.drop_duplicates(subset=["ubigeo"]).sort_values(["provincia", "distrito"])
60|
61|
62| def save_locations(locations: pd.DataFrame, output_path: Path) -> None:
63|     ensure_parent_dir(output_path)
64|     locations.to_csv(output_path, index=False)
65|     LOGGER.info("Saved %s locations to %s", len(locations), output_path)
66|
67|
68| def run(input_path: Path, output_path: Path) -> None:
69|     locations = load_locations(input_path)
70|     cleaned = clean_locations(locations)
71|     save_locations(cleaned, output_path)
72|
73|
74| def parse_args() -> argparse.Namespace:
75|     parser = argparse.ArgumentParser(description="Validate and process Puno district locations.")
76|     parser.add_argument("--input", type=Path, default=LOCATIONS_EXTERNAL_PATH)
77|     parser.add_argument("--output", type=Path, default=LOCATIONS_PROCESSED_PATH)
78|     return parser.parse_args()

```

```

79
80
81 if __name__ == "__main__":
82     configure_logging()
83     args = parse_args()
84     run(args.input, args.output)

```

ml_pipeline/data_ingestion/ingest_weather_open_meteo.py

```

1 | from __future__ import annotations
2
3 | import argparse
4 | import logging
5 | from concurrent.futures import ThreadPoolExecutor, as_completed
6 | from datetime import date, timedelta
7 | from pathlib import Path
8 | from typing import Any
9
10 | import pandas as pd
11 | import requests
12
13 | from ml_pipeline.config import (
14 |     LOCATIONS_PROCESSED_PATH,
15 |     OPEN_METEO_HOURLY_VARIABLES,
16 |     WEATHER_RAW_PATH,
17 | )
18 | from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns
19
20 |
21 | LOGGER = logging.getLogger(__name__)
22 | ARCHIVE_URL = "https://archive-api.open-meteo.com/v1/archive"
23
24 |
25 | def default_date_range() -> tuple[str, str]:
26 |     end = date.today() - timedelta(days=5)
27 |     start = end - timedelta(days=13)
28 |     return start.isoformat(), end.isoformat()
29
30 |
31 | def load_locations(path: Path, limit: int | None = None) -> pd.DataFrame:
32 |     if not path.exists():
33 |         raise FileNotFoundError(f"Processed locations file not found: {path}")
34
35 |     locations = pd.read_csv(path, dtype={"ubigeo": str})
36 |     validate_columns(
37 |         locations,
38 |         ["ubigeo", "departamento", "provincia", "distrito", "latitud", "longitud"],
39 |         "locations_puno.csv",
40 |     )
41 |     if limit is not None:
42 |         locations = locations.head(limit)
43 |     return locations
44
45 |
46 | def fetch_weather_for_location(location: dict[str, Any], start_date: str, end_date: str) -> pd.DataFrame:
47 |     params = {
48 |         "latitude": location["latitud"],
49 |         "longitude": location["longitud"],
50 |         "start_date": start_date,
51 |         "end_date": end_date,
52 |         "hourly": ",".join(OPEN_METEO_HOURLY_VARIABLES),
53 |         "timezone": "America/Lima",
54 |         "temperature_unit": "celsius",
55 |         "wind_speed_unit": "kmh",
56 |         "precipitation_unit": "mm",
57 |     }
58 |     response = requests.get(ARCHIVE_URL, params=params, timeout=60)
59 |     response.raise_for_status()
60 |     payload = response.json()
61
62 |     hourly = payload.get("hourly")
63 |     if not hourly or "time" not in hourly:
64 |         raise ValueError(f"Open-Meteo response has no hourly data for ubigeo={location['ubigeo']}")
65
66 |     weather = pd.DataFrame(hourly)
67 |     weather["ubigeo"] = location["ubigeo"]
68 |     weather["departamento"] = location["departamento"]
69 |     weather["provincia"] = location["provincia"]
70 |     weather["distrito"] = location["distrito"]
71 |     weather["latitud"] = location["latitud"]
72 |     weather["longitud"] = location["longitud"]
73 |     weather["data_source"] = "Open-Meteo Historical API"
74 |     return weather
75
76 |
77 | def fetch_weather_batch(
78 |     locations: pd.DataFrame,
79 |     start_date: str,
80 |     end_date: str,
81 |     max_workers: int,
82 | ) -> pd.DataFrame:
83 |     frames: list[pd.DataFrame] = []
84 |     failures: list[str] = []
85
86 |     records = locations.to_dict(orient="records")
87 |     with ThreadPoolExecutor(max_workers=max_workers) as executor:
88 |         futures = {
89 |             executor.submit(fetch_weather_for_location, record, start_date, end_date): record
90 |             for record in records
91 |         }
92 |         for future in as_completed(futures):
93 |             record = futures[future]
94 |             try:
95 |                 frame = future.result()
96 |                 frames.append(frame)
97 |                 LOGGER.info("Fetched weather for %s - %s", record["ubigeo"], record["distrito"])
98 |             except Exception as exc: # noqa: BLE001 - keep batch failures visible by location.
99 |                 message = f"{record['ubigeo']} {record['distrito']}: {exc}"
100 |                 failures.append(message)
101 |                 LOGGER.error("Weather fetch failed for %s", message)
102
103 |     if not frames:
104 |         raise RuntimeError(f"No weather data fetched. Failures: {failures}")
105 |     if failures:
106 |         LOGGER.warning("Weather fetch completed with %s failures.", len(failures))
107

```

```

108     return pd.concat(frames, ignore_index=True)
109
110
111 def save_weather(weather: pd.DataFrame, output_path: Path) -> None:
112     ensure_parent_dir(output_path)
113     weather.to_csv(output_path, index=False)
114     LOGGER.info("Saved %s weather rows to %s", len(weather), output_path)
115
116
117 def run(
118     locations_path: Path,
119     output_path: Path,
120     start_date: str,
121     end_date: str,
122     max_workers: int,
123     limit: int | None,
124 ) -> None:
125     locations = load_locations(locations_path, limit=limit)
126     weather = fetch_weather_batch(locations, start_date, end_date, max_workers=max_workers)
127     save_weather(weather, output_path)
128
129
130 def parse_args() -> argparse.Namespace:
131     start_date, end_date = default_date_range()
132     parser = argparse.ArgumentParser(description="Fetch hourly weather from Open-Meteo for Puno districts.")
133     parser.add_argument("--locations", type=Path, default=LOCATIONS_PROCESSED_PATH)
134     parser.add_argument("--output", type=Path, default=WEATHER_RAW_PATH)
135     parser.add_argument("--start-date", default=start_date)
136     parser.add_argument("--end-date", default=end_date)
137     parser.add_argument("--max-workers", type=int, default=4)
138     parser.add_argument("--limit", type=int, default=None, help="Optional number of locations for quick smoke runs.")
139     return parser.parse_args()
140
141
142 if __name__ == "__main__":
143     configure_logging()
144     args = parse_args()
145     run(args.locations, args.output, args.start_date, args.end_date, args.max_workers, args.limit)

```

ml_pipeline/evaluation/evaluate_model.py

```

1|
2| from __future__ import annotations
3|
4| import argparse
5| import json
6| import logging
7| from pathlib import Path
8|
9| import joblib
10| import pandas as pd
11| from sklearn.metrics import confusion_matrix
12|
13| from ml_pipeline.config import (
14|     CONFUSION_MATRIX_PATH,
15|     EVALUATION_REPORT_PATH,
16|     MODEL_METADATA_PATH,
17|     MODEL_PATH,
18|     NUMERIC_FEATURES,
19|     TARGET_COLUMN,
20|     TRAINING_DATASET_PATH,
21| )
22| from ml_pipeline.training.train_models import evaluate_predictions, split_dataset
23| from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns, write_json
24|
25|
26| LOGGER = logging.getLogger(__name__)
27| LABEL_ORDER = ["bajo", "medio", "alto"]
28|
29|
30| def load_metadata(path: Path) -> dict:
31|     if not path.exists():
32|         raise FileNotFoundError(f"Model metadata not found: {path}")
33|     return json.loads(path.read_text(encoding="utf-8"))
34|
35|
36| def run(dataset_path: Path, model_path: Path, metadata_path: Path) -> None:
37|     if not model_path.exists():
38|         raise FileNotFoundError(f"Model file not found: {model_path}")
39|     if not dataset_path.exists():
40|         raise FileNotFoundError(f"Dataset file not found: {dataset_path}")
41|
42|     dataset = pd.read_csv(dataset_path)
43|     validate_columns(dataset, [*NUMERIC_FEATURES, TARGET_COLUMN], "frost_training_dataset.csv")
44|     metadata = load_metadata(metadata_path)
45|     model = joblib.load(model_path)
46|
47|     _, x_test, _, y_test = split_dataset(dataset, test_size=0.25)
48|     predictions = model.predict(x_test)
49|     metrics = evaluate_predictions(y_test, predictions)
50|     matrix = confusion_matrix(y_test, predictions, labels=LABEL_ORDER)
51|
52|     ensure_parent_dir(CONFUSION_MATRIX_PATH)
53|     pd.DataFrame(matrix, index=LABEL_ORDER, columns=LABEL_ORDER).to_csv(CONFUSION_MATRIX_PATH)
54|
55|     report = {
56|         "model_name": metadata.get("model_name"),
57|         "version": metadata.get("version"),
58|         "features": NUMERIC_FEATURES,
59|         "target": TARGET_COLUMN,
60|         "metrics": metrics,
61|         "label_order": LABEL_ORDER,
62|         "confusion_matrix_path": str(CONFUSION_MATRIX_PATH),
63|     }
64|     write_json(EVALUATION_REPORT_PATH, report)
65|     LOGGER.info("Evaluation report saved to %s", EVALUATION_REPORT_PATH)
66|
67|
68| def parse_args() -> argparse.Namespace:
69|     parser = argparse.ArgumentParser(description="Evaluate the registered FrostPuno model.")
70|     parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_PATH)
71|     parser.add_argument("--model", type=Path, default=MODEL_PATH)
72|     parser.add_argument("--metadata", type=Path, default=MODEL_METADATA_PATH)
73|     return parser.parse_args()
74|
75|

```

```

76 if __name__ == "__main__":
77     configure_logging()
78     args = parse_args()
79     run(args.dataset, args.model, args.metadata)

```

ml_pipeline/evaluation/evaluate_observed_events.py

```

1 | from __future__ import annotations
2
3 | import argparse
4 | import logging
5 | from pathlib import Path
6 | from typing import Any
7
8 | import joblib
9 | import pandas as pd
10 | from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, precision_score, recall_score
11
12 | from ml_pipeline.config import (
13 |     MODEL_METADATA_V2_PATH,
14 |     MODEL_V2_PATH,
15 |     NUMERIC_FEATURES_V2,
16 |     OBSERVATION_EVALUATION_PATH,
17 |     OBSERVATION_REQUIRED_COLUMNS,
18 |     PROJECT_ROOT,
19 |     SENAMHI_OBSERVATIONS_PATH,
20 | )
21 | from ml_pipeline.training.train_models_v2 import LABEL_ORDER
22 | from ml_pipeline.utils import configure_logging, ensure_parent_dir, utc_now_iso, validate_columns, write_json
23
24 |
25 | LOGGER = logging.getLogger(__name__)
26
27 |
28 | def repo_relative(path: Path) -> str:
29 |     return path.relative_to(PROJECT_ROOT).as_posix()
30
31 |
32 | def load_observations(path: Path) -> pd.DataFrame:
33 |     observations = pd.read_csv(path)
34 |     validate_columns(observations, OBSERVATION_REQUIRED_COLUMNS, path.name)
35 |     observations["datetime"] = pd.to_datetime(observations["observed_at"], errors="coerce")
36 |     if observations["datetime"].isna().any():
37 |         raise ValueError("Observation file contains invalid observed_at values.")
38 |     observations["mes"] = observations["datetime"].dt.month
39 |     observations["hora"] = observations["datetime"].dt.hour
40 |     return observations.dropna(subset=[*NUMERIC_FEATURES_V2, "riesgo_helada_observado"])
41
42 |
43 | def evaluate_observations(model_path: Path, observations_path: Path, output_path: Path) -> dict[str, Any]:
44 |     LOGGER.info("Loading model=%s", model_path)
45 |     LOGGER.info("Loading observed events=%s", observations_path)
46 |     model = joblib.load(model_path)
47 |     observations = load_observations(observations_path)
48
49 |     y_true = observations["riesgo_helada_observado"].astype(str)
50 |     y_pred = model.predict(observations[NUMERIC_FEATURES_V2])
51
52 |     metrics = {
53 |         "accuracy": float(accuracy_score(y_true, y_pred)),
54 |         "precision_macro": float(precision_score(y_true, y_pred, average="macro", zero_division=0)),
55 |         "recall_macro": float(recall_score(y_true, y_pred, average="macro", zero_division=0)),
56 |         "f1_macro": float(f1_score(y_true, y_pred, average="macro", zero_division=0)),
57 |     }
58 |     matrix = confusion_matrix(y_true, y_pred, labels=LABEL_ORDER)
59 |     mismatches = observations.assign(prediccion=y_pred)
60 |     mismatches = mismatches[mismatches["riesgo_helada_observado"] != mismatches["prediccion"]]
61
62 |     report: dict[str, Any] = {
63 |         "created_at": utc_now_iso(),
64 |         "status": "sample_validation",
65 |         "model_path": repo_relative(model_path),
66 |         "model_metadata_path": repo_relative(MODEL_METADATA_V2_PATH),
67 |         "observations_path": repo_relative(observations_path),
68 |         "rows": int(len(observations)),
69 |         "features": NUMERIC_FEATURES_V2,
70 |         "target": "riesgo_helada_observado",
71 |         "metrics": metrics,
72 |         "label_order": LABEL_ORDER,
73 |         "confusion_matrix": matrix.tolist(),
74 |         "mismatches": mismatches[
75 |             ["observed_at", "distrito", "riesgo_helada_observado", "prediccion", "fuente_observacion"]
76 |         ].to_dict(orient="records"),
77 |         "limitations": [
78 |             "This file is a schema/sample for official observation validation, not a complete SENAMHI extraction.",
79 |             "Observed frost labels must be replaced with official SENAMHI/campo data before academic promotion.",
80 |             "The result is used to decide whether a future model version should be retrained or rejected.",
81 |         ],
82 |     }
83 |     ensure_parent_dir(output_path)
84 |     write_json(output_path, report)
85 |     LOGGER.info("Saved observation evaluation to %s", output_path)
86 |     return report
87
88 |
89 | def parse_args() -> argparse.Namespace:
90 |     parser = argparse.ArgumentParser(description="Evaluate FrostPuno v0.2.0 against observed frost events.")
91 |     parser.add_argument("--model", type=Path, default=MODEL_V2_PATH)
92 |     parser.add_argument("--observations", type=Path, default=SENAMHI_OBSERVATIONS_PATH)
93 |     parser.add_argument("--output", type=Path, default=OBSERVATION_EVALUATION_PATH)
94 |     return parser.parse_args()
95
96 |
97 | if __name__ == "__main__":
98 |     configure_logging()
99 |     args = parse_args()
100 |     evaluate_observations(args.model, args.observations, args.output)

```

ml_pipeline/features/build_features.py

```

1|
2| from __future__ import annotations
3|
4| import argparse
5| import logging
6| from pathlib import Path
7|
8| import pandas as pd
9|
10| from ml_pipeline.config import (
11|     LOCATIONS_PROCESSED_PATH,
12|     OPEN_METEO_HOURLY_VARIABLES,
13|     TRAINING_DATASET_PATH,
14|     WEATHER_RAW_PATH,
15| )
16| from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns
17|
18|
19| LOGGER = logging.getLogger(__name__)
20|
21|
22| def load_inputs(locations_path: Path, weather_path: Path) -> tuple[pd.DataFrame, pd.DataFrame]:
23|     if not locations_path.exists():
24|         raise FileNotFoundError(f"Locations file not found: {locations_path}")
25|     if not weather_path.exists():
26|         raise FileNotFoundError(f"Weather file not found: {weather_path}")
27|
28|     locations = pd.read_csv(locations_path, dtype={"ubigeo": str})
29|     weather = pd.read_csv(weather_path, dtype={"ubigeo": str})
30|
31|     validate_columns(
32|         locations,
33|         [
34|             "ubigeo",
35|             "departamento",
36|             "provincia",
37|             "distrito",
38|             "latitud",
39|             "longitud",
40|             "altitud_estimada",
41|             "poblacion_total",
42|             "poblacion_rural",
43|             "porcentaje_rural",
44|         ],
45|         "locations_puno.csv",
46|     )
47|     validate_columns(weather, ["ubigeo", "time", *OPEN_METEO_HOURLY_VARIABLES], "weather_open_meteo.csv")
48|     return locations, weather
49|
50|
51| def label_frost_risk(temperature_min: float, hours_below_zero: int) -> str:
52|     if temperature_min <= 0 or hours_below_zero >= 3:
53|         return "alto"
54|     if 0 < temperature_min <= 3:
55|         return "medio"
56|     return "bajo"
57|
58|
59| def build_training_dataset(locations: pd.DataFrame, weather: pd.DataFrame) -> pd.DataFrame:
60|     weather_clean = weather.copy()
61|     weather_clean["datetime"] = pd.to_datetime(weather_clean["time"], errors="coerce")
62|     if weather_clean["datetime"].isna().any():
63|         raise ValueError("Weather data contains invalid datetime values.")
64|
65|     for column in OPEN_METEO_HOURLY_VARIABLES:
66|         weather_clean[column] = pd.to_numeric(weather_clean[column], errors="coerce")
67|
68|     missing_weather = weather_clean[OPEN_METEO_HOURLY_VARIABLES].isna().sum()
69|     if (missing_weather > 0).any():
70|         LOGGER.warning("Weather data has missing values: %s", missing_weather[missing_weather > 0].to_dict())
71|
72|     weather_clean["fecha"] = weather_clean["datetime"].dt.date.astype(str)
73|     weather_clean["mes"] = weather_clean["datetime"].dt.month
74|     weather_clean["hora"] = weather_clean["datetime"].dt.hour
75|     weather_clean["is_below_zero"] = weather_clean["temperature_2m"] < 0
76|
77|     daily = (
78|         weather_clean.groupby(["ubigeo", "fecha"], as_index=False)
79|         .agg(
80|             temperatura_minima_diaria=("temperature_2m", "min"),
81|             horas_bajo_cero=("is_below_zero", "sum"),
82|         )
83|     )
84|
85|     dataset = weather_clean.merge(daily, on=["ubigeo", "fecha"], how="left")
86|     location_columns = [
87|         "ubigeo",
88|         "altitud_estimada",
89|         "poblacion_total",
90|         "poblacion_rural",
91|         "porcentaje_rural",
92|     ]
93|     dataset = dataset.drop(columns=["latitud", "longitud", "departamento", "provincia", "distrito"], errors="ignore")
94|     dataset = dataset.merge(locations, on="ubigeo", how="left", suffixes=("", "_location"))
95|
96|     validate_columns(dataset, location_columns, "merged training dataset")
97|     dataset["riesgo_helada"] = dataset.apply(
98|         lambda row: label_frost_risk(row["temperatura_minima_diaria"], int(row["horas_bajo_cero"])),
99|         axis=1,
100|     )
101|
102|     dataset["actividad_agropecuaria"] = True
103|     dataset["cultivo_relevante"] = "papa"
104|     dataset["temporada_agricola"] = dataset["mes"].map(infer_agricultural_season)
105|
106|     ordered_columns = [
107|         "fecha",
108|         "time",
109|         "departamento",
110|         "provincia",
111|         "distrito",
112|         "ubigeo",
113|         "latitud",
114|         "longitud",
115|         "altitud_estimada",
116|         "poblacion_total",
117|         "poblacion_rural",
118|         "porcentaje_rural",
119|         "actividad_agropecuaria",
120|         "cultivo_relevante",
121|         *OPEN_METEO_HOURLY_VARIABLES,
122|         "mes",

```

```

122     "hora",
123     "temporada_agricola",
124     "temperatura_minima_diaria",
125     "horas_bajo_cero",
126     "riesgo_helada",
127 ]
128 return dataset[ordered_columns].sort_values(["ubigeo", "time"])
129
130
131 def infer_agricultural_season(month: int) -> str:
132     if month in {5, 6, 7, 8}:
133         return "heladas_invernales"
134     if month in {9, 10, 11}:
135         return "siembra"
136     if month in {12, 1, 2, 3}:
137         return "campania_lluvias"
138     return "transicion"
139
140
141 def save_dataset(dataset: pd.DataFrame, output_path: Path) -> None:
142     ensure_parent_dir(output_path)
143     dataset.to_csv(output_path, index=False)
144     LOGGER.info(
145         "Saved training dataset with %s rows and class distribution %s to %s",
146         len(dataset),
147         dataset["riesgo_helada"].value_counts().to_dict(),
148         output_path,
149     )
150
151
152 def run(locations_path: Path, weather_path: Path, output_path: Path) -> None:
153     locations, weather = load_inputs(locations_path, weather_path)
154     dataset = build_training_dataset(locations, weather)
155     save_dataset(dataset, output_path)
156
157
158 def parse_args() -> argparse.Namespace:
159     parser = argparse.ArgumentParser(description="Build FrostPuno ML features and labels.")
160     parser.add_argument("--locations", type=Path, default=LOCATIONS_PROCESSED_PATH)
161     parser.add_argument("--weather", type=Path, default=WEATHER_RAW_PATH)
162     parser.add_argument("--output", type=Path, default=TRAINING_DATASET_PATH)
163     return parser.parse_args()
164
165
166 if __name__ == "__main__":
167     configure_logging()
168     args = parse_args()
169     run(args.locations, args.weather, args.output)

```

ml_pipeline/features/build_features_v2.py

```

1  from __future__ import annotations
2
3  import argparse
4  import logging
5  from pathlib import Path
6
7  import pandas as pd
8
9  from ml_pipeline.config import (
10     LEAKAGE_FEATURES,
11     NUMERIC_FEATURES_V2,
12     TARGET_COLUMN,
13     TRAINING_DATASET_PATH,
14     TRAINING_DATASET_V2_PATH,
15 )
16 from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns
17
18
19 LOGGER = logging.getLogger(__name__)
20
21
22 REQUIRED_CONTEXT_COLUMNS = ["fecha", "time", "distrito", "ubigeo"]
23
24
25 def build_dataset_v2(source: pd.DataFrame) -> pd.DataFrame:
26     validate_columns(
27         source,
28         [*REQUIRED_CONTEXT_COLUMNS, *NUMERIC_FEATURES_V2, TARGET_COLUMN],
29         "frost_training_dataset.csv",
30     )
31     dataset = source[
32         [*REQUIRED_CONTEXT_COLUMNS, *NUMERIC_FEATURES_V2, TARGET_COLUMN]
33     ].copy()
34     leaked = [column for column in LEAKAGE_FEATURES if column in dataset.columns]
35     if leaked:
36         raise ValueError(f"Dataset v2 still contains leakage columns: {leaked}")
37     return dataset
38
39
40 def run(source_path: Path, output_path: Path) -> None:
41     if not source_path.exists():
42         raise FileNotFoundError(f"Source dataset not found: {source_path}")
43
44     source = pd.read_csv(source_path)
45     dataset = build_dataset_v2(source)
46     ensure_parent_dir(output_path)
47     dataset.to_csv(output_path, index=False)
48     LOGGER.info("Dataset v2 source: %s", source_path)
49     LOGGER.info("Dataset v2 output: %s", output_path)
50     LOGGER.info("Dataset v2 shape: rows=%s columns=%s", len(dataset), len(dataset.columns))
51     LOGGER.info("Dataset v2 features: %s", NUMERIC_FEATURES_V2)
52     LOGGER.info("Dataset v2 class distribution: %s", dataset[TARGET_COLUMN].value_counts().to_dict())
53
54
55 def parse_args() -> argparse.Namespace:
56     parser = argparse.ArgumentParser(description="Build FrostPuno dataset v2 without leakage features.")
57     parser.add_argument("--source", type=Path, default=TRAINING_DATASET_PATH)
58     parser.add_argument("--output", type=Path, default=TRAINING_DATASET_V2_PATH)
59     return parser.parse_args()
60
61
62 if __name__ == "__main__":
63     configure_logging()
64     args = parse_args()
65     run(args.source, args.output)

```

ml_pipeline/preprocessing/validate_dataset.py

```

1|
2| from __future__ import annotations
3|
4| import argparse
5| import logging
6| from pathlib import Path
7|
8| import pandas as pd
9|
10| from ml_pipeline.config import NUMERIC_FEATURES, TARGET_COLUMN, TRAINING_DATASET_PATH
11| from ml_pipeline.utils import configure_logging, validate_columns
12|
13|
14| LOGGER = logging.getLogger(__name__)
15| EXPECTED_TARGETS = {"bajo", "medio", "alto"}
16|
17|
18| def validate_training_dataset(path: Path) -> None:
19|     if not path.exists():
20|         raise FileNotFoundError(f"Training dataset not found: {path}")
21|
22|     dataset = pd.read_csv(path)
23|     validate_columns(dataset, [NUMERIC_FEATURES, TARGET_COLUMN], "frost_training_dataset.csv")
24|
25|     if dataset.empty:
26|         raise ValueError("Training dataset is empty.")
27|
28|     missing = dataset[NUMERIC_FEATURES].isna().sum()
29|     critical_missing = missing[missing > 0]
30|     if not critical_missing.empty:
31|         raise ValueError(f"Training dataset has missing feature values: {critical_missing.to_dict()}")
32|
33|     targets = set(dataset[TARGET_COLUMN].dropna().unique())
34|     unexpected = targets - EXPECTED_TARGETS
35|     if unexpected:
36|         raise ValueError(f"Unexpected target classes: {unexpected}")
37|
38|     impossible_ranges = {
39|         "relative_humidity_2m": dataset["relative_humidity_2m"].between(0, 100).all(),
40|         "cloud_cover": dataset["cloud_cover"].between(0, 100).all(),
41|         "wind_speed_10m": (dataset["wind_speed_10m"] >= 0).all(),
42|         "precipitation": (dataset["precipitation"] >= 0).all(),
43|     }
44|     failed = [name for name, ok in impossible_ranges.items() if not ok]
45|     if failed:
46|         raise ValueError(f"Climate variables outside expected ranges: {failed}")
47|
48|     LOGGER.info("Dataset validation passed: %s rows, classes=%s", len(dataset), sorted(targets))
49|
50|
51| def parse_args() -> argparse.Namespace:
52|     parser = argparse.ArgumentParser(description="Validate FrostPuno training dataset.")
53|     parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_PATH)
54|     return parser.parse_args()
55|
56|
57| if __name__ == "__main__":
58|     configure_logging()
59|     args = parse_args()
60|     validate_training_dataset(args.dataset)

```

ml_pipeline/registry/check_cluster_quality.py

```

1| """Quality gate NO SUPERVISADO: bloquea modelos de clustering bajo el umbral de silhouette.
2|
3| Reemplaza a check_model_quality.py (fi_macro) en el path productivo.
4| """
5|
6| from __future__ import annotations
7|
8| import argparse
9| import json
10| import os
11| import sys
12| from pathlib import Path
13|
14|
15| DEFAULT_METADATA_PATH = Path(__file__).resolve().parent / "cluster_metadata.json"
16|
17|
18| def load_silhouette(metadata_path: Path) -> float:
19|     if not metadata_path.exists():
20|         raise FileNotFoundError(f"Cluster metadata not found: {metadata_path}")
21|
22|     metadata = json.loads(metadata_path.read_text(encoding="utf-8"))
23|     metrics = metadata.get("metrics")
24|     if not isinstance(metrics, dict):
25|         raise ValueError("cluster_metadata.json must contain a metrics object.")
26|
27|     silhouette = metrics.get("silhouette")
28|     if silhouette is None:
29|         raise ValueError("cluster_metadata.json metrics must contain silhouette.")
30|
31|     return float(silhouette)
32|
33|
34| def parse_threshold(cli_threshold: float | None) -> float:
35|     if cli_threshold is not None:
36|         return cli_threshold
37|     return float(os.getenv("MIN_SILHOUETTE", "0.35"))
38|
39|
40| def run(metadata_path: Path, min_silhouette: float) -> int:
41|     silhouette = load_silhouette(metadata_path)
42|     print(f"Cluster quality gate: silhouette={silhouette:.6f}, minimum={min_silhouette:.6f}")
43|
44|     if silhouette < min_silhouette:
45|         print("QUALITY GATE FAILED: cluster silhouette is below the required threshold.")
46|         return 1
47|
48|     print("QUALITY GATE PASSED: cluster silhouette meets the required threshold.")
49|     return 0
50|

```

```

51|
52| def parse_args() -> argparse.Namespace:
53|     parser = argparse.ArgumentParser(description="Validate FrostPuno clustering quality metrics.")
54|     parser.add_argument("--metadata", type=Path, default=DEFAULT_METADATA_PATH)
55|     parser.add_argument("--min-silhouette", type=float, default=None)
56|     return parser.parse_args()
57|
58|
59| if __name__ == "__main__":
60|     args = parse_args()
61|     try:
62|         sys.exit(run(args.metadata, parse_threshold(args.min_silhouette)))
63|     except Exception as exc: # noqa: BLE001 - CLI should print clear failure and exit non-zero.
64|         print(f"QUALITY GATE ERROR: {exc}", file=sys.stderr)
65|         sys.exit(1)

```

ml_pipeline/registry/check_model_quality.py

```

1|
2| from __future__ import annotations
3|
4| import argparse
5| import json
6| import os
7| import sys
8| from pathlib import Path
9|
10|
11| DEFAULT_METADATA_PATH = Path(__file__).resolve().parent / "model_metadata.json"
12|
13|
14| def load_f1_macro(metadata_path: Path) -> float:
15|     if not metadata_path.exists():
16|         raise FileNotFoundError(f"Model metadata not found: {metadata_path}")
17|
18|     metadata = json.loads(metadata_path.read_text(encoding="utf-8"))
19|     metrics = metadata.get("metrics")
20|     if not isinstance(metrics, dict):
21|         raise ValueError("model_metadata.json must contain a metrics object.")
22|
23|     f1_macro = metrics.get("f1_macro")
24|     if f1_macro is None:
25|         raise ValueError("model_metadata.json metrics must contain f1_macro.")
26|
27|     return float(f1_macro)
28|
29|
30| def parse_threshold(cli_threshold: float | None) -> float:
31|     if cli_threshold is not None:
32|         return cli_threshold
33|     return float(os.getenv("MIN_F1_MACRO", "0.70"))
34|
35|
36| def run(metadata_path: Path, min_f1_macro: float) -> int:
37|     f1_macro = load_f1_macro(metadata_path)
38|     print(f"Model quality gate: f1_macro={f1_macro:.6f}, minimum={min_f1_macro:.6f}")
39|
40|     if f1_macro < min_f1_macro:
41|         print("QUALITY GATE FAILED: model f1_macro is below the required threshold.")
42|         return 1
43|
44|     print("QUALITY GATE PASSED: model f1_macro meets the required threshold.")
45|     return 0
46|
47|
48| def parse_args() -> argparse.Namespace:
49|     parser = argparse.ArgumentParser(description="Validate FrostPuno model quality metrics.")
50|     parser.add_argument("--metadata", type=Path, default=DEFAULT_METADATA_PATH)
51|     parser.add_argument("--min-f1-macro", type=float, default=None)
52|     return parser.parse_args()
53|
54|
55| if __name__ == "__main__":
56|     args = parse_args()
57|     try:
58|         sys.exit(run(args.metadata, parse_threshold(args.min_f1_macro)))
59|     except Exception as exc: # noqa: BLE001 - CLI should print clear failure and exit non-zero.
60|         print(f"QUALITY GATE ERROR: {exc}", file=sys.stderr)
61|         sys.exit(1)

```

ml_pipeline/tests/test_maintenance_ci.py

```

1| """Pruebas de funcionamiento del MANTENIMIENTO e INTEGRACIÓN CONTINUA.
2|
3| Verifican que los flujos automatizados del elemento inteligente funcionan:
4|
5| 1. El reentrenamiento produce artefactos válidos y agrupa todos los distritos.
6| 2. El quality gate APRUEBA el modelo productivo actual.
7| 3. El quality gate BLOQUEA un modelo degradado (barrera real de la IC).
8| 4. La metadata documenta los hiperparámetros optimizados (grid search).
9| """
10|
11| from __future__ import annotations
12|
13| import json
14|
15| import joblib
16| import pytest
17|
18| from ml_pipeline.config import (
19|     CLUSTER_FEATURES,
20|     CLUSTER_METADATA_PATH,
21|     CLUSTER_MODEL_PATH,
22|     DISTRICT_CLUSTERS_PATH,
23|     RISK_TIERS,
24| )
25| from ml_pipeline.registry.check_cluster_quality import run as run_gate
26|
27|
28| @pytest.fixture(scope="module")
29| def metadata() -> dict:

```



```

30 assert CLUSTER_METADATA_PATH.exists(), "Falta cluster_metadata.json: entrena el modelo primero."
31 return json.loads(CLUSTER_METADATA_PATH.read_text(encoding="utf-8"))
32
33
34 def test_registry_artifacts_exist(metadata: dict) -> None:
35     """El reentrenamiento deja modelo, metadata y agrupación de distritos."""
36     assert CLUSTER_MODEL_PATH.exists()
37     assert DISTRICT_CLUSTERS_PATH.exists()
38     assert metadata["model_name"] == "KMeans"
39     assert metadata["features"] == CLUSTER_FEATURES
40     assert metadata["n_clusters"] >= 3
41
42
43 def test_model_loads_and_predicts(metadata: dict) -> None:
44     """El artefacto serializado se puede cargar e inferir (contrato con el backend)."""
45     import pandas as pd
46
47     pipeline = joblib.load(CLUSTER_MODEL_PATH)
48     sample = pd.DataFrame([{"name": 0.0 for name in metadata["features"]}])
49     cluster = int(pipeline.predict(sample)[0])
50     assert 0 <= cluster < metadata["n_clusters"]
51
52
53 def test_district_groupings_cover_all_districts() -> None:
54     """Cada distrito queda asignado a un tier de riesgo válido."""
55     import pandas as pd
56
57     districts = pd.read_csv(DISTRICT_CLUSTERS_PATH)
58     assert len(districts) >= 13, "Se esperan al menos los 13 distritos del MVP."
59     assert set(districts["tier"]).issubset(set(RISK_TIERS))
60     assert districts["distrito"].is_unique
61
62
63 def test_hyperparameters_are_optimized(metadata: dict) -> None:
64     """La metadata documenta la búsqueda de hiperparámetros (evidencia de optimización)."""
65     search = metadata["hyperparameter_search"]
66     assert search["criterion"] == "silhouette"
67     assert search["combinations_evaluated"] >= 12
68     best = metadata["best_params"]
69     assert best["n_clusters"] == metadata["n_clusters"]
70     assert best["init"] in search["grid"]["init"]
71     # La configuración elegida es realmente la mejor del grid.
72     best_row = max(search["results"], key=lambda r: r["silhouette"])
73     assert best_row["k"] == best["n_clusters"]
74
75
76 def test_quality_gate_passes_for_production_model() -> None:
77     """El gate aprueba el modelo actualmente en el registry."""
78     assert run_gate(CLUSTER_METADATA_PATH, min_silhouette=0.35) == 0
79
80
81 def test_quality_gate_blocks_degraded_model(tmp_path, metadata: dict) -> None:
82     """El gate DETIENE la integración si el modelo empeora (barrera de mantenimiento)."""
83     degraded = dict(metadata)
84     degraded["metrics"] = {"silhouette": 0.10, "davies_bouldin": 3.0, "inertia": 1.0}
85     bad_path = tmp_path / "cluster_metadata.json"
86     bad_path.write_text(json.dumps(degraded), encoding="utf-8")
87
88     assert run_gate(bad_path, min_silhouette=0.35) == 1

```

ml_pipeline/tests/test_model_v2.py

```

1 from __future__ import annotations
2
3 import pandas as pd
4
5 from ml_pipeline.config import LEAKAGE_FEATURES, NUMERIC_FEATURES_V2, TARGET_COLUMN
6 from ml_pipeline.evaluation.evaluate_observed_events import load_observations
7 from ml_pipeline.features.build_features_v2 import build_dataset_v2
8 from ml_pipeline.training.train_models_v2 import split_dataset_v2
9
10
11 def _sample_dataset() -> pd.DataFrame:
12     rows = []
13     districts = ["A", "B", "C", "D"]
14     for index, district in enumerate(districts):
15         for hour in range(4):
16             rows.append(
17                 {
18                     "fecha": f"2024-06-0{hour + 1}",
19                     "time": f"2024-06-0{hour + 1}T0{hour}:00",
20                     "distrito": district,
21                     "ubigeo": f"210{index}",
22                     "latitud": -15.0 - index,
23                     "longitud": -70.0 - index,
24                     "altitud_estimada": 3800 + index,
25                     "temperature_2m": float(hour - 2),
26                     "relative_humidity_2m": 70.0,
27                     "apparent_temperature": float(hour - 3),
28                     "dew_point_2m": -2.0,
29                     "precipitation": 0.0,
30                     "cloud_cover": 20.0,
31                     "wind_speed_10m": 7.0,
32                     "mes": 6,
33                     "hora": hour,
34                     "temperatura_minima_diaria": -2.0,
35                     "horas_bajo_cero": 3,
36                     "riesgo_helada": "alto" if hour < 2 else "medio",
37                 }
38             )
39     return pd.DataFrame(rows)
40
41
42 def test_dataset_v2_removes_leakage_features() -> None:
43     dataset = build_dataset_v2(_sample_dataset())
44
45     for column in LEAKAGE_FEATURES:
46         assert column not in dataset.columns
47     assert TARGET_COLUMN in dataset.columns
48     assert set(NUMERIC_FEATURES_V2).issubset(dataset.columns)
49
50
51 def test_district_split_keeps_districts_disjoint() -> None:
52     dataset = build_dataset_v2(_sample_dataset())
53
54     x_train, x_test, y_train, y_test, metadata = split_dataset_v2(

```

```

55     dataset,
56     strategy="district",
57     test_size=0.5,
58 )
59
60 assert not x_train.empty
61 assert not x_test.empty
62 assert not y_train.empty
63 assert not y_test.empty
64 assert set(metadata["train_districts"]).isdisjoint(metadata["test_districts"])
65
66
67 def test_time_split_uses_recent_dates_for_test() -> None:
68     dataset = build_dataset_v2(_sample_dataset())
69
70     _, y_train, y_test, metadata = split_dataset_v2(
71         dataset,
72         strategy="time",
73         test_size=0.25,
74     )
75
76     assert not y_train.empty
77     assert not y_test.empty
78     assert metadata["strategy"] == "time"
79     assert metadata["train_end"] < metadata["test_start"]
80
81
82 def test_observation_validation_adds_time_features(tmp_path) -> None:
83     observations_path = tmp_path / "observations.csv"
84     pd.DataFrame(
85         [
86             {
87                 "observed_at": "2024-06-01T03:00:00",
88                 "distrito": "Puno",
89                 "latitud": -15.8402,
90                 "longitud": -70.0219,
91                 "altitud_estimada": 3827,
92                 "temperature_2m": -2.1,
93                 "relative_humidity_2m": 71,
94                 "apparent_temperature": -3.8,
95                 "dew_point_2m": -4.0,
96                 "precipitation": 0,
97                 "cloud_cover": 18,
98                 "wind_speed_10m": 8,
99                 "riesgo_helada_observado": "alto",
100                 "fuente_observacion": "SENAMHI_sample",
101             }
102         ]
103     ).to_csv(observations_path, index=False)
104
105     observations = load_observations(observations_path)
106
107     assert set(NUMERIC_FEATURES_V2).issubset(observations.columns)
108     assert observations.loc[0, "mes"] == 6
109     assert observations.loc[0, "hora"] == 3

```

ml_pipeline/training/train_models.py

```

1|
2| """LEGACY -clasificador supervisado (previo migrado).
3|
4| El path productivo del curso es el modelo NO SUPERVISADO en
5| ``ml_pipeline/clustering/train_clusters.py`` (K-Means, métrica silhouette).
6| Este script se conserva solo como referencia histórica para el informe
7| ("modelo previo migrado"); no forma parte de la IC ni del backend.
8| """
9|
10| from __future__ import annotations
11|
12| import argparse
13| import logging
14| from pathlib import Path
15| from typing import Any
16|
17| import joblib
18| import pandas as pd
19| from sklearn.compose import ColumnTransformer
20| from sklearn.impute import SimpleImputer
21| from sklearn.linear_model import LogisticRegression
22| from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
23| from sklearn.model_selection import train_test_split
24| from sklearn.pipeline import Pipeline
25| from sklearn.preprocessing import StandardScaler
26| from sklearn.tree import DecisionTreeClassifier
27| from sklearn.ensemble import RandomForestClassifier
28|
29| from ml_pipeline.config import (
30|     METRICS_COMPARISON_PATH,
31|     MODEL_METADATA_PATH,
32|     MODEL_PATH,
33|     NUMERIC_FEATURES,
34|     TARGET_COLUMN,
35|     TRAINING_DATASET_PATH,
36| )
37| from ml_pipeline.utils import configure_logging, ensure_parent_dir, utc_now_iso, validate_columns, write_json
38|
39|
40| LOGGER = logging.getLogger(__name__)
41| RANDOM_STATE = 42
42|
43|
44| def load_dataset(path: Path) -> pd.DataFrame:
45|     if not path.exists():
46|         raise FileNotFoundError(f"Training dataset not found: {path}")
47|
48|     dataset = pd.read_csv(path)
49|     validate_columns(dataset, [*NUMERIC_FEATURES, TARGET_COLUMN], "frost_training_dataset.csv")
50|     return dataset.dropna(subset=[*NUMERIC_FEATURES, TARGET_COLUMN])
51|
52|
53| def split_dataset(dataset: pd.DataFrame, test_size: float) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]:
54|     x = dataset[NUMERIC_FEATURES]
55|     y = dataset[TARGET_COLUMN]
56|     class_counts = y.value_counts()
57|     stratify = y if len(class_counts) > 1 and class_counts.min() >= 2 else None
58|     return train_test_split(x, y, test_size=test_size, random_state=RANDOM_STATE, stratify=stratify)

```

```

55
56
57
58
59
60
61 def build_preprocessor(scale: bool) -> ColumnTransformer:
62     steps: list[tuple[str, Any]] = [("imputer", SimpleImputer(strategy="median"))]
63     if scale:
64         steps.append(("scaler", StandardScaler()))
65     numeric_pipeline = Pipeline(steps=steps)
66     return ColumnTransformer(transformers=[("numeric", numeric_pipeline, NUMERIC_FEATURES)])
67
68
69 def candidate_models() -> dict[str, Pipeline]:
70     return {
71         "LogisticRegression": Pipeline(
72             steps=[
73                 ("preprocess", build_preprocessor(scale=True)),
74                 (
75                     "model",
76                     LogisticRegression(max_iter=1000, class_weight="balanced", random_state=RANDOM_STATE),
77                 ),
78             ],
79         ),
80         "DecisionTreeClassifier": Pipeline(
81             steps=[
82                 ("preprocess", build_preprocessor(scale=False)),
83                 (
84                     "model",
85                     DecisionTreeClassifier(max_depth=6, class_weight="balanced", random_state=RANDOM_STATE),
86                 ),
87             ],
88         ),
89         "RandomForestClassifier": Pipeline(
90             steps=[
91                 ("preprocess", build_preprocessor(scale=False)),
92                 (
93                     "model",
94                     RandomForestClassifier(
95                         n_estimators=150,
96                         max_depth=10,
97                         class_weight="balanced",
98                         random_state=RANDOM_STATE,
99                         n_jobs=-1,
100                     ),
101                 ),
102             ],
103         ),
104     }
105
106
107 def evaluate_predictions(y_true: pd.Series, y_pred: pd.Series | list[str]) -> dict[str, float]:
108     return {
109         "accuracy": float(accuracy_score(y_true, y_pred)),
110         "precision_macro": float(precision_score(y_true, y_pred, average="macro", zero_division=0)),
111         "recall_macro": float(recall_score(y_true, y_pred, average="macro", zero_division=0)),
112         "f1_macro": float(f1_score(y_true, y_pred, average="macro", zero_division=0)),
113     }
114
115
116 def train_and_compare(dataset: pd.DataFrame, test_size: float) -> tuple[str, Pipeline, dict[str, dict[str, float]]]:
117     x_train, x_test, y_train, y_test = split_dataset(dataset, test_size=test_size)
118     metrics: dict[str, dict[str, float]] = {}
119     trained_models: dict[str, Pipeline] = {}
120
121     for name, model in candidate_models().items():
122         LOGGER.info("Training %s", name)
123         model.fit(x_train, y_train)
124         predictions = model.predict(x_test)
125         metrics[name] = evaluate_predictions(y_test, predictions)
126         trained_models[name] = model
127         LOGGER.info("%s metrics: %s", name, metrics[name])
128
129     tie_break_priority = {
130         "RandomForestClassifier": 3,
131         "DecisionTreeClassifier": 2,
132         "LogisticRegression": 1,
133     }
134     best_model_name = max(
135         metrics,
136         key=lambda model_name: (metrics[model_name]["f1_macro"], tie_break_priority.get(model_name, 0)),
137     )
138     return best_model_name, trained_models[best_model_name], metrics
139
140
141 def save_registry(
142     model_name: str,
143     model: Pipeline,
144     metrics: dict[str, dict[str, float]],
145     dataset: pd.DataFrame,
146     version: str,
147 ) -> None:
148     ensure_parent_dir(MODEL_PATH)
149     joblib.dump(model, MODEL_PATH)
150     write_json(METRICS_COMPARISON_PATH, metrics)
151
152     metadata = {
153         "model_name": model_name,
154         "version": version,
155         "created_at": utc_now_iso(),
156         "features": NUMERIC_FEATURES,
157         "target": TARGET_COLUMN,
158         "metrics": metrics[model_name],
159         "all_model_metrics": metrics,
160         "dataset_size": int(len(dataset)),
161         "data_sources": [
162             "Open-Meteo Historical API",
163             "INEI-compatible curated district seed for Puno MVP",
164             "SENAMHI documented for validation in phase 2",
165             "MIDAGRI/SIEA documented as agrarian enrichment in phase 2",
166         ],
167         "limitations": [
168             "Initial labels are rule-based and derived from temperature thresholds.",
169             "The curated INEI district file is an MVP seed and must be replaced with official exports for production.",
170             "Daily minimum temperature and hours below zero are label-derived features, so baseline metrics can be optimistic.",
171             "SENAMHI station observations are not yet used as ground-truth labels.",
172         ],
173     }
174     write_json(MODEL_METADATA_PATH, metadata)
175     LOGGER.info("Saved best model to %s and metadata to %s", MODEL_PATH, MODEL_METADATA_PATH)
176
177
178 def run(dataset_path: Path, version: str, test_size: float) -> None:
179     dataset = load_dataset(dataset_path)

```

```

180 best_model_name, best_model, metrics = train_and_compare(dataset, test_size=test_size)
181 save_registry(best_model_name, best_model, metrics, dataset, version=version)
182
183
184 def parse_args() -> argparse.Namespace:
185     parser = argparse.ArgumentParser(description="Train and compare FrostPuno ML models.")
186     parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_PATH)
187     parser.add_argument("--version", default="v0.1.0")
188     parser.add_argument("--test-size", type=float, default=0.25)
189     return parser.parse_args()
190
191
192 if __name__ == "__main__":
193     configure_logging()
194     args = parse_args()
195     run(args.dataset, args.version, args.test_size)

```

ml_pipeline/training/train_models_v2.py

```

1 from __future__ import annotations
2
3 import argparse
4 import logging
5 from pathlib import Path
6 from typing import Any, Literal
7
8 import joblib
9 import pandas as pd
10 from sklearn.compose import ColumnTransformer
11 from sklearn.ensemble import RandomForestClassifier
12 from sklearn.impute import SimpleImputer
13 from sklearn.linear_model import LogisticRegression
14 from sklearn.metrics import confusion_matrix
15 from sklearn.pipeline import Pipeline
16 from sklearn.preprocessing import StandardScaler
17
18 from ml_pipeline.config import (
19     CONFUSION_MATRIX_V2_PATH,
20     LEAKAGE_FEATURES,
21     METRICS_COMPARISON_V2_PATH,
22     MODEL_METADATA_PATH,
23     MODEL_METADATA_V2_PATH,
24     MODEL_PATH,
25     MODEL_V2_PATH,
26     NUMERIC_FEATURES_V2,
27     PROJECT_ROOT,
28     TARGET_COLUMN,
29     TRAINING_DATASET_PATH,
30     TRAINING_DATASET_V2_PATH,
31 )
32 from ml_pipeline.features.build_features_v2 import build_dataset_v2
33 from ml_pipeline.training.train_models import RANDOM_STATE, evaluate_predictions
34 from ml_pipeline.utils import configure_logging, ensure_parent_dir, utc_now_iso, validate_columns, write_json
35
36
37 LOGGER = logging.getLogger(__name__)
38 LABEL_ORDER = ["bajo", "medio", "alto"]
39 SplitStrategy = Literal["district", "time"]
40
41
42 def repo_relative(path: Path) -> str:
43     return path.relative_to(PROJECT_ROOT).as_posix()
44
45
46 def load_or_create_dataset_v2(source_path: Path, dataset_path: Path) -> pd.DataFrame:
47     if dataset_path.exists():
48         dataset = pd.read_csv(dataset_path)
49     else:
50         if not source_path.exists():
51             raise FileNotFoundError(f"Source dataset not found: {source_path}")
52         dataset = build_dataset_v2(pd.read_csv(source_path))
53         ensure_parent_dir(dataset_path)
54         dataset.to_csv(dataset_path, index=False)
55
56     validate_columns(
57         dataset,
58         ["fecha", "time", "distrito", "ubigeo", *NUMERIC_FEATURES_V2, TARGET_COLUMN],
59         dataset_path.name,
60     )
61     leaked = [column for column in LEAKAGE_FEATURES if column in dataset.columns]
62     if leaked:
63         raise ValueError(f"Dataset v2 contains leakage columns: {leaked}")
64     return dataset.dropna(subset=[*NUMERIC_FEATURES_V2, TARGET_COLUMN])
65
66
67 def split_by_district(
68     dataset: pd.DataFrame,
69     test_size: float,
70 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series, dict[str, Any]]:
71     districts = dataset["distrito"].dropna().drop_duplicates().sample(
72         frac=1,
73         random_state=RANDOM_STATE,
74     )
75     test_count = max(1, round(len(districts) * test_size))
76     test_districts = set(districts.tail(test_count))
77     train_districts = set(districts) - test_districts
78     train = dataset[dataset["distrito"].isin(train_districts)].copy()
79     test = dataset[dataset["distrito"].isin(test_districts)].copy()
80     if train.empty or test.empty:
81         raise ValueError("District split produced an empty train or test partition.")
82
83     return (
84         train[NUMERIC_FEATURES_V2],
85         test[NUMERIC_FEATURES_V2],
86         train[TARGET_COLUMN],
87         test[TARGET_COLUMN],
88         {
89             "strategy": "district",
90             "test_size": test_size,
91             "train_districts": sorted(train_districts),
92             "test_districts": sorted(test_districts),
93             "train_rows": int(len(train)),
94             "test_rows": int(len(test)),
95         },
96     )
97

```

```

98
99 def split_by_time(
100     dataset: pd.DataFrame,
101     test_size: float,
102 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series, dict[str, Any]]:
103     dated = dataset.copy()
104     dated["datetime"] = pd.to_datetime(dated["time"], errors="coerce")
105     if dated["datetime"].isna().any():
106         raise ValueError("Dataset v2 contains invalid timestamps.")
107     ordered_dates = dated["datetime"].dt.date.drop_duplicates().sort_values()
108     test_count = max(1, round(len(ordered_dates) * test_size))
109     cutoff = set(ordered_dates.tail(test_count))
110     train = dated[~dated["datetime"].dt.date.isin(cutoff)].copy()
111     test = dated[dated["datetime"].dt.date.isin(cutoff)].copy()
112     if train.empty or test.empty:
113         raise ValueError("Time split produced an empty train or test partition.")
114
115     return (
116         train[NUMERIC_FEATURES_V2],
117         test[NUMERIC_FEATURES_V2],
118         train[TARGET_COLUMN],
119         test[TARGET_COLUMN],
120         {
121             "strategy": "time",
122             "test_size": test_size,
123             "train_start": str(train["datetime"].min()),
124             "train_end": str(train["datetime"].max()),
125             "test_start": str(test["datetime"].min()),
126             "test_end": str(test["datetime"].max()),
127             "train_rows": int(len(train)),
128             "test_rows": int(len(test)),
129         },
130     )
131
132
133 def split_dataset_v2(
134     dataset: pd.DataFrame,
135     strategy: SplitStrategy,
136     test_size: float,
137 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series, dict[str, Any]]:
138     if strategy == "district":
139         return split_by_district(dataset, test_size)
140     if strategy == "time":
141         return split_by_time(dataset, test_size)
142     raise ValueError(f"Unsupported split strategy: {strategy}")
143
144
145 def build_preprocessor(scale: bool) -> ColumnTransformer:
146     steps: list[tuple[str, Any]] = [("imputer", SimpleImputer(strategy="median"))]
147     if scale:
148         steps.append(("scaler", StandardScaler()))
149     numeric_pipeline = Pipeline(steps=steps)
150     return ColumnTransformer(transformers=[("numeric", numeric_pipeline, NUMERIC_FEATURES_V2)])
151
152
153 def candidate_models() -> dict[str, Pipeline]:
154     return {
155         "LogisticRegression": Pipeline(
156             steps=[
157                 ("preprocess", build_preprocessor(scale=True)),
158                 (
159                     "model",
160                     LogisticRegression(max_iter=1000, class_weight="balanced", random_state=RANDOM_STATE),
161                 ),
162             ],
163         ),
164         "RandomForestClassifier": Pipeline(
165             steps=[
166                 ("preprocess", build_preprocessor(scale=False)),
167                 (
168                     "model",
169                     RandomForestClassifier(
170                         n_estimators=150,
171                         max_depth=10,
172                         class_weight="balanced",
173                         random_state=RANDOM_STATE,
174                         n_jobs=-1,
175                     ),
176                 ),
177             ],
178         ),
179     }
180
181
182 def train_and_compare_v2(
183     dataset: pd.DataFrame,
184     strategy: SplitStrategy,
185     test_size: float,
186 ) -> tuple[str, Pipeline, dict[str, dict[str, float]], pd.Series, list[str], dict[str, Any]]:
187     x_train, x_test, y_train, y_test, split_metadata = split_dataset_v2(dataset, test_size)
188     LOGGER.info("Dataset v2 rows=%s", len(dataset))
189     LOGGER.info("Dataset v2 features=%s", NUMERIC_FEATURES_V2)
190     LOGGER.info("Split strategy=%s metadata=%s", strategy, split_metadata)
191
192     metrics: dict[str, dict[str, float]] = {}
193     trained_models: dict[str, Pipeline] = {}
194     predictions_by_model: dict[str, list[str]] = {}
195     for name, model in candidate_models().items():
196         LOGGER.info("Training v0.2.0 candidate %s", name)
197         model.fit(x_train, y_train)
198         predictions = model.predict(x_test)
199         predictions_by_model[name] = [str(item) for item in predictions]
200         metrics[name] = evaluate_predictions(y_test, predictions)
201         trained_models[name] = model
202         LOGGER.info("%s v0.2.0 metrics: %s", name, metrics[name])
203
204     best_model_name = "RandomForestClassifier"
205     return (
206         best_model_name,
207         trained_models[best_model_name],
208         metrics,
209         y_test,
210         predictions_by_model[best_model_name],
211         split_metadata,
212     )
213
214
215 def save_registry_v2(
216     model_name: str,
217     model: Pipeline,
218     metrics: dict[str, dict[str, float]],

```

```

219 y_test: pd.Series,
220 predictions: list[str],
221 dataset: pd.DataFrame,
222 version: str,
223 split_metadata: dict[str, Any],
224 ) -> None:
225     ensure_parent_dir(MODEL_V2_PATH)
226     joblib.dump(model, MODEL_V2_PATH)
227     write_json(METRICS_COMPARISON_V2_PATH, metrics)
228
229 matrix = confusion_matrix(y_test, predictions, labels=LABEL_ORDER)
230 ensure_parent_dir(CONFUSION_MATRIX_V2_PATH)
231 pd.DataFrame(matrix, index=LABEL_ORDER, columns=LABEL_ORDER).to_csv(CONFUSION_MATRIX_V2_PATH)
232
233 metadata = {
234     "model_name": model_name,
235     "version": version,
236     "created_at": utc_now_iso(),
237     "status": "experimental",
238     "production_model_unchanged": True,
239     "production_model_path": repo_relative(MODEL_PATH),
240     "production_metadata_path": repo_relative(MODEL_METADATA_PATH),
241     "model_path": repo_relative(MODEL_V2_PATH),
242     "dataset_path": repo_relative(TRAINING_DATASET_V2_PATH),
243     "features": NUMERIC_FEATURES_V2,
244     "removed_leakage_features": LEAKAGE_FEATURES,
245     "target": TARGET_COLUMN,
246     "split": split_metadata,
247     "metrics": metrics[model_name],
248     "all_model_metrics": metrics,
249     "selection_reason": "RandomForestClassifier is kept as the v0.2.0 baseline model; LogisticRegression is reported only for comparison.",
250     "label_order": LABEL_ORDER,
251     "confusion_matrix_path": repo_relative(CONFUSION_MATRIX_V2_PATH),
252     "dataset_size": int(len(dataset)),
253     "data_sources": [
254         "Open-Meteo Historical API",
255         "INEI-compatible curated district seed for Puno MVP",
256     ],
257     "limitations": [
258         "Labels remain rule-based and are not official observed frost events.",
259         "Leakage-derived features were removed from model inputs.",
260         "District split estimates generalization to unseen districts, but still requires SENAMHI/campo ground truth.",
261         "This model is experimental and is not used by FastAPI production endpoints.",
262     ],
263 }
264 write_json(MODEL_METADATA_V2_PATH, metadata)
265 LOGGER.info("Saved v0.2.0 model to %s", MODEL_V2_PATH)
266 LOGGER.info("Saved v0.2.0 metadata to %s", MODEL_METADATA_V2_PATH)
267 LOGGER.info("Saved v0.2.0 confusion matrix to %s", CONFUSION_MATRIX_V2_PATH)
268
269
270 def run(
271     source_path: Path,
272     dataset_path: Path,
273     version: str,
274     strategy: SplitStrategy,
275     test_size: float,
276 ) -> None:
277     dataset = load_or_create_dataset_v2(source_path, dataset_path)
278     best_model_name, best_model, metrics, y_test, predictions, split_metadata = train_and_compare_v2(
279         dataset,
280         strategy,
281         test_size,
282     )
283     save_registry_v2(best_model_name, best_model, metrics, y_test, predictions, dataset, version, split_metadata)
284
285
286 def parse_args() -> argparse.Namespace:
287     parser = argparse.ArgumentParser(description="Train FrostPuno v0.2.0 without leakage features.")
288     parser.add_argument("--source", type=Path, default=TRAINING_DATASET_PATH)
289     parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_V2_PATH)
290     parser.add_argument("--version", default="v0.2.0")
291     parser.add_argument("--split-strategy", choices=["district", "time"], default="district")
292     parser.add_argument("--test-size", type=float, default=0.30)
293     return parser.parse_args()
294
295
296 if __name__ == "__main__":
297     configure_logging()
298     args = parse_args()
299     run(args.source, args.dataset, args.version, args.split_strategy, args.test_size)

```

ml_pipeline/utils.py

```

1 from __future__ import annotations
2
3 import json
4 import logging
5 from datetime import datetime, timezone
6 from pathlib import Path
7 from typing import Any, Iterable
8
9 import pandas as pd
10
11
12 def configure_logging() -> None:
13     logging.basicConfig(
14         level=logging.INFO,
15         format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
16     )
17
18
19 def ensure_parent_dir(path: Path) -> None:
20     path.parent.mkdir(parents=True, exist_ok=True)
21
22
23 def validate_columns(frame: pd.DataFrame, required_columns: Iterable[str], dataset_name: str) -> None:
24     missing = [column for column in required_columns if column not in frame.columns]
25     if missing:
26         raise ValueError(f"{dataset_name} is missing required columns: {missing}")
27
28
29 def write_json(path: Path, payload: dict[str, Any]) -> None:
30     ensure_parent_dir(path)
31     path.write_text(json.dumps(payload, indent=2, ensure_ascii=False, encoding="utf-8"))
32

```

```

33|
34| def utc_now_iso() -> str:
35|     return datetime.now(timezone.utc).replace(microsecond=0).isoformat()

```

2. Servicio backend

backend__fastapi/app/___init___py

```

1|
2| """FrostPuno FastAPI backend."""

```

backend__fastapi/app/api/___init___py

```

1| """HTTP API package."""

```

backend__fastapi/app/api/routes/___init___py

```

1| """Route modules."""

```

backend__fastapi/app/api/routes/alerts.py

```

1| from __future__ import annotations
2|
3| from datetime import date, timedelta
4|
5| from fastapi import APIRouter, Depends, Query
6|
7| from app.api.routes.weather import get_weather_provider
8| from app.schemas.alerts import ClimateAnomaly, DailyAlertResponse, LivestockRisk
9| from app.services.alert_service import anomaly, frost_severity, livestock_risk
10| from app.services.model_registry import ModelRegistry, get_model_registry
11| from app.services.weather_providers import WeatherQuery
12|
13| router = APIRouter(prefix="/alerts", tags=["alerts"])
14|
15|
16| @router.get("/today", response_model=DailyAlertResponse)
17| def alert_today(
18|     latitude: float = Query(..., ge=-18.5, le=-13.0),
19|     longitude: float = Query(..., ge=-71.5, le=-68.0),
20|     registry: ModelRegistry = Depends(get_model_registry),
21| ) -> DailyAlertResponse:
22|     provider = get_weather_provider()
23|     query = WeatherQuery(latitude=latitude, longitude=longitude)
24|
25|     forecast = provider.get_daily_forecast(query, days=1)
26|     today = forecast[0] if forecast else {}
27|     temp_min = today.get("temperature_2m_min")
28|     day = today.get("date")
29|
30|     # Climatología reciente para detectar anomalías (últimos ~30 días con retraso de archivo).
31|     history_rows: list[dict] = []
32|     try:
33|         end = date.today() - timedelta(days=6)
34|         start = end - timedelta(days=29)
35|         history_rows = provider.get_history(query, start.isoformat(), end.isoformat())
36|     except Exception: # noqa: BLE001 - La alerta no debe romperse si el histórico falla.
37|         history_rows = []
38|
39|     risk_level, severity, alert, title, message = frost_severity(temp_min)
40|     livestock = livestock_risk(temp_min)
41|     anomaly_data = anomaly(temp_min, history_rows)
42|
43|     return DailyAlertResponse(
44|         latitude=latitude,
45|         longitude=longitude,
46|         date=str(day) if day is not None else None,
47|         risk_level=risk_level,
48|         frost_alert=alert,
49|         severity=severity,
50|         temperature_min=temp_min,
51|         title=title,
52|         message=message,
53|         livestock=LivestockRisk(**livestock),
54|         anomaly=ClimateAnomaly(**anomaly_data),
55|         model_version=registry.metadata.version,
56|     )

```

backend__fastapi/app/api/routes/chuno.py

```

1| from __future__ import annotations
2|
3| from datetime import datetime, timezone
4|
5| from fastapi import APIRouter, Query
6|
7| from app.api.routes.weather import get_weather_provider
8| from app.schemas.chuno import ChunoWindowResponse
9| from app.services.chuno_service import evaluate_window
10| from app.services.weather_providers import WeatherQuery
11|
12| router = APIRouter(prefix="/chuno", tags=["chuno"])
13|

```

```

14
15 @router.get("/window", response_model=ChunoWindowResponse)
16 def chuno_window(
17     latitude: float = Query(..., ge=-18.5, le=-13.0),
18     longitude: float = Query(..., ge=-71.5, le=-68.0),
19     days: int = Query(7, ge=1, le=16),
20 ) -> ChunoWindowResponse:
21     provider = get_weather_provider()
22     forecast = provider.get_daily_forecast(WeatherQuery(latitude=latitude, longitude=longitude), days=days)
23     current_month = datetime.now(timezone.utc).month
24     return evaluate_window(latitude, longitude, current_month, forecast)

```

backend__fastapi/app/api/routes/health.py

```

1| from __future__ import annotations
2
3| from fastapi import APIRouter
4
5| from app.core.config import settings
6| from app.schemas.health import HealthResponse
7
8|
9| router = APIRouter(tags=["health"])
10
11
12 @router.get("/health", response_model=HealthResponse)
13 def health_check() -> HealthResponse:
14     model_exists = settings.model_path.exists()
15     metadata_exists = settings.model_metadata_path.exists()
16     return HealthResponse(
17         status="ok" if model_exists and metadata_exists else "degraded",
18         app_name=settings.app_name,
19         version=settings.api_version,
20         model_available=model_exists and metadata_exists,
21     )

```

backend__fastapi/app/api/routes/ml.py

```

1| from __future__ import annotations
2
3| import csv
4
5| from fastapi import APIRouter, Depends
6
7| from app.core.config import settings
8| from app.schemas.ml import (
9|     ClusterProfile,
10|     ClustersResponse,
11|     DistrictCluster,
12|     ModelInfoResponse,
13| )
14| from app.services.model_registry import ModelRegistry, get_model_registry
15
16|
17| router = APIRouter(prefix="/ml", tags=["ml"])
18
19|
20 @router.get("/model-info", response_model=ModelInfoResponse)
21 def model_info(registry: ModelRegistry = Depends(get_model_registry)) -> ModelInfoResponse:
22     metadata = registry.metadata
23     return ModelInfoResponse(
24         model_name=metadata.model_name,
25         version=metadata.version,
26         created_at=metadata.created_at,
27         features=metadata.features,
28         target=metadata.target,
29         metrics=metadata.metrics,
30         dataset_size=metadata.dataset_size,
31         data_sources=metadata.data_sources,
32         limitations=metadata.limitations,
33         n_clusters=metadata.n_clusters,
34     )
35
36
37 def _load_districts() -> list[DistrictCluster]:
38     path = settings.district_clusters_path
39     if not path.exists():
40         return []
41     districts: list[DistrictCluster] = []
42     with path.open(encoding="utf-8", newline="") as handle:
43         for row in csv.DictReader(handle):
44             districts.append(
45                 DistrictCluster(
46                     distrito=row["distrito"],
47                     tier=row["tier"],
48                     dominant_cluster=int(float(row["dominant_cluster"])),
49                     cold_share=float(row["cold_share"]),
50                     altitud_estimada=float(row["altitud_estimada"]),
51                     temperature_2m_mean=float(row["temperature_2m_mean"]),
52                     latitud=float(row["latitud"]) if row.get("latitud") else None,
53                     longitud=float(row["longitud"]) if row.get("longitud") else None,
54                 )
55             )
56     return districts
57
58
59 @router.get("/clusters", response_model=ClustersResponse)
60 def clusters(registry: ModelRegistry = Depends(get_model_registry)) -> ClustersResponse:
61     metadata = registry.metadata
62     profiles_raw = metadata.cluster_profiles or {}
63     profiles = [
64         ClusterProfile(
65             cluster_id=int(cid),
66             tier=str(profile.get("tier", "medio")),
67             size=int(profile.get("size", 0)),
68             temperature_2m=float(profile.get("temperature_2m", 0.0)),
69             dew_point_2m=float(profile.get("dew_point_2m", 0.0)),
70         )
71         for cid, profile in profiles_raw.items()
72     ]
73     profiles.sort(key=lambda p: p.temperature_2m)

```



```

74     return ClustersResponse(
75         model_name=metadata.model_name,
76         version=metadata.version,
77         n_clusters=metadata.n_clusters or len(profiles),
78         silhouette=float(metadata.metrics.get("silhouette", 0.0)),
79         profiles=profiles,
80         districts=_load_districts(),
81     )

```

backend_fastapi/app/api/routes/predict.py

```

1| from __future__ import annotations
2|
3| from fastapi import APIRouter, Depends
4|
5| from app.repositories.predictions import PredictionRepository, get_prediction_repository
6| from app.schemas.prediction import FrostRiskRequest, FrostRiskResponse
7| from app.services.prediction_service import PredictionService, get_prediction_service
8|
9|
10| router = APIRouter(prefix="/predict", tags=["prediction"])
11|
12|
13| @router.post("/frost-risk", response_model=FrostRiskResponse)
14| def predict_frost_risk(
15|     payload: FrostRiskRequest,
16|     service: PredictionService = Depends(get_prediction_service),
17|     repository: PredictionRepository = Depends(get_prediction_repository),
18| ) -> FrostRiskResponse:
19|     prediction = service.predict(payload)
20|     repository.save_prediction(payload, prediction)
21|     return prediction

```

backend_fastapi/app/api/routes/predictions.py

```

1| from __future__ import annotations
2|
3| from fastapi import APIRouter, Depends, Query
4|
5| from app.repositories.predictions import PredictionRepository, get_prediction_repository
6| from app.schemas.prediction import FrostPredictionHistoryItem
7|
8|
9| router = APIRouter(prefix="/predictions", tags=["predictions"])
10|
11|
12| @router.get("/history", response_model=list[FrostPredictionHistoryItem])
13| def prediction_history(
14|     limit: int = Query(default=20, ge=1, le=100),
15|     repository: PredictionRepository = Depends(get_prediction_repository),
16| ) -> list[FrostPredictionHistoryItem]:
17|     return repository.list_history(limit=limit)

```

backend_fastapi/app/api/routes/weather.py

```

1| from __future__ import annotations
2|
3| from functools import lru_cache
4|
5| from fastapi import APIRouter, Query
6|
7| from app.schemas.weather import CurrentWeatherResponse, HistoryDay, WeatherHistoryResponse
8| from app.services.weather_providers import HybridWeatherProvider, WeatherQuery
9|
10|
11| router = APIRouter(prefix="/weather", tags=["weather"])
12|
13|
14| @lru_cache(maxsize=1)
15| def get_weather_provider() -> HybridWeatherProvider:
16|     return HybridWeatherProvider()
17|
18|
19| @router.get("/current", response_model=CurrentWeatherResponse)
20| def current_weather(
21|     latitude: float = Query(..., ge=-18.5, le=-13.0),
22|     longitude: float = Query(..., ge=-71.5, le=-68.0),
23| ) -> CurrentWeatherResponse:
24|     return get_weather_provider().get_current_weather(
25|         WeatherQuery(latitude=latitude, longitude=longitude),
26|     )
27|
28|
29| @router.get("/history", response_model=WeatherHistoryResponse)
30| def weather_history(
31|     latitude: float = Query(..., ge=-18.5, le=-13.0),
32|     longitude: float = Query(..., ge=-71.5, le=-68.0),
33|     start: str = Query(..., description="Fecha inicio YYYY-MM-DD"),
34|     end: str = Query(..., description="Fecha fin YYYY-MM-DD"),
35| ) -> WeatherHistoryResponse:
36|     rows = get_weather_provider().get_history(
37|         WeatherQuery(latitude=latitude, longitude=longitude), start, end
38|     )
39|     days = [
40|         HistoryDay(
41|             date=str(row.get("date")),
42|             temperature_min=row.get("temperature_2m_min"),
43|             temperature_max=row.get("temperature_2m_max"),
44|             humidity_mean=row.get("relative_humidity_2m_mean"),
45|         )
46|         for row in rows
47|     ]
48|     return WeatherHistoryResponse(
49|         latitude=latitude,
50|         longitude=longitude,
51|         start_date=start,

```

```

52|         end_date=end,
53|         days=days,
54|     )

```

backend_fastapi/app/core/__init__.py

```

1| """Core backend settings and exceptions."""

```

backend_fastapi/app/core/config.py

```

1|
2| from __future__ import annotations
3|
4| from pathlib import Path
5|
6| from pydantic import AliasChoices, Field
7| from pydantic_settings import BaseSettings, SettingsConfigDict
8|
9|
10| PROJECT_ROOT = Path(__file__).resolve().parents[3]
11|
12|
13| class Settings(BaseSettings):
14|     app_name: str = Field(default="FrostPuno API", validation_alias=AliasChoices("APP_NAME", "FROST_PUNO_APP_NAME"))
15|     api_version: str = Field(default="0.1.0", validation_alias=AliasChoices("API_VERSION", "FROST_PUNO_API_VERSION"))
16|     log_level: str = Field(default="INFO", validation_alias=AliasChoices("LOG_LEVEL", "FROST_PUNO_LOG_LEVEL"))
17|
18|     # Modelo productivo: K-Means no supervisado (reemplaza al clasificador supervisado).
19|     model_path: Path = Field(
20|         default=PROJECT_ROOT / "ml_pipeline" / "registry" / "frost_cluster_model.joblib",
21|         validation_alias=AliasChoices("MODEL_PATH", "FROST_PUNO_MODEL_PATH"),
22|     )
23|     model_metadata_path: Path = Field(
24|         default=PROJECT_ROOT / "ml_pipeline" / "registry" / "cluster_metadata.json",
25|         validation_alias=AliasChoices("MODEL_METADATA_PATH", "FROST_PUNO_MODEL_METADATA_PATH"),
26|     )
27|     district_clusters_path: Path = Field(
28|         default=PROJECT_ROOT / "data" / "processed" / "district_clusters.csv",
29|         validation_alias=AliasChoices("DISTRICT_CLUSTERS_PATH", "FROST_PUNO_DISTRICT_CLUSTERS_PATH"),
30|     )
31|
32|     supabase_url: str | None = Field(default=None, validation_alias=AliasChoices("SUPABASE_URL", "FROST_PUNO_SUPABASE_URL"))
33|     supabase_service_role_key: str | None = Field(
34|         default=None,
35|         validation_alias=AliasChoices("SUPABASE_SERVICE_ROLE_KEY", "FROST_PUNO_SUPABASE_SERVICE_ROLE_KEY"),
36|     )
37|     enable_supabase: bool = Field(default=False, validation_alias=AliasChoices("ENABLE_SUPABASE", "FROST_PUNO_PERSIST_PREDICTIONS"))
38|     cors_allowed_origins: str = Field(
39|         default="http://127.0.0.1:5174,http://127.0.0.1:5173,http://localhost:5174,http://localhost:5173",
40|         validation_alias=AliasChoices("CORS_ORIGINS", "CORS_ALLOWED_ORIGINS", "FROST_PUNO_CORS_ALLOWED_ORIGINS"),
41|     )
42|
43|     @property
44|     def cors_origins(self) -> list[str]:
45|         return [origin.strip() for origin in self.cors_allowed_origins.split(",") if origin.strip()]
46|
47|     model_config = SettingsConfigDict(
48|         env_file=(PROJECT_ROOT / ".env", PROJECT_ROOT / "backend_fastapi" / ".env"),
49|         extra="ignore",
50|     )
51|
52| settings = Settings()
53|

```

backend_fastapi/app/core/exceptions.py

```

1| class ModelLoadError(RuntimeError):
2|     """Raised when the model registry cannot be loaded."""
3|
4|
5| class PredictionError(RuntimeError):
6|     """Raised when a validated request cannot be transformed into a prediction."""

```

backend_fastapi/app/main.py

```

1|
2| from __future__ import annotations
3|
4| import logging
5|
6| from fastapi import FastAPI, Request
7| from fastapi.responses import JSONResponse
8| from fastapi.middleware.cors import CORSMiddleware
9|
10| from app.api.routes import alerts, chuno, health, ml, predict, predictions, weather
11| from app.core.config import settings
12| from app.core.exceptions import ModelLoadError, PredictionError
13|
14|
15| logging.basicConfig(
16|     level=settings.log_level,
17|     format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
18| )
19|
20|
21| def create_app() -> FastAPI:
22|     app = FastAPI(
23|         title=settings.app_name,
24|         version=settings.api_version,
25|         description="Backend MVP for FrostPuno frost-risk prediction.",
26|     )

```

```

27 |     app.add_middleware(
28 |         CORSMiddleware,
29 |         allow_origins=settings.cors_origins,
30 |         allow_credentials=False,
31 |         allow_methods=["GET", "POST"],
32 |         allow_headers=["Content-Type"],
33 |     )
34 |
35 | @app.exception_handler(ModelLoadError)
36 | async def model_load_error_handler(_: Request, exc: ModelLoadError) -> JSONResponse:
37 |     return JSONResponse(
38 |         status_code=503,
39 |         content={
40 |             "error": {
41 |                 "code": "MODEL_UNAVAILABLE",
42 |                 "message": str(exc),
43 |             }
44 |         },
45 |     )
46 |
47 | @app.exception_handler(PredictionError)
48 | async def prediction_error_handler(_: Request, exc: PredictionError) -> JSONResponse:
49 |     return JSONResponse(
50 |         status_code=422,
51 |         content={
52 |             "error": {
53 |                 "code": "PREDICTION_ERROR",
54 |                 "message": str(exc),
55 |             }
56 |         },
57 |     )
58 |
59 | app.include_router(health.router)
60 | app.include_router(ml.router)
61 | app.include_router(predict.router)
62 | app.include_router(predictions.router)
63 | app.include_router(weather.router)
64 | app.include_router(chuno.router)
65 | app.include_router(alerts.router)
66 | return app
67 |
68 |
69 | app = create_app()

```

backend_fastapi/app/repositories/__init__.py

```

1 | """Persistence adapters."""

```

backend_fastapi/app/repositories/predictions.py

```

1 | from __future__ import annotations
2 |
3 | import logging
4 | from functools import lru_cache
5 | from typing import Protocol
6 |
7 | from app.core.config import settings
8 | from app.schemas.prediction import FrostPredictionHistoryItem, FrostRiskRequest, FrostRiskResponse
9 |
10 |
11 | LOGGER = logging.getLogger(__name__)
12 |
13 |
14 | class PredictionRepository(Protocol):
15 |     def save_prediction(self, request: FrostRiskRequest, response: FrostRiskResponse) -> None:
16 |         """Persist a prediction when a backing store is configured."""
17 |
18 |     def list_history(self, limit: int = 20) -> list[FrostPredictionHistoryItem]:
19 |         """Return recent predictions."""
20 |
21 |
22 | class NoOpPredictionRepository:
23 |     def save_prediction(self, request: FrostRiskRequest, response: FrostRiskResponse) -> None:
24 |         LOGGER.info(
25 |             "Prediction persistence disabled. district=%s risk=%s model=%s",
26 |             request.district,
27 |             response.risk_level,
28 |             response.model_version,
29 |         )
30 |
31 |     def list_history(self, limit: int = 20) -> list[FrostPredictionHistoryItem]:
32 |         LOGGER.info("Prediction history requested while persistence is disabled. limit=%s", limit)
33 |         return []
34 |
35 |
36 | @lru_cache(maxsize=1)
37 | def get_prediction_repository() -> PredictionRepository:
38 |     if settings.enable_supabase:
39 |         if settings.supabase_url and settings.supabase_service_role_key:
40 |             from app.repositories.supabase_prediction_repository import SupabasePredictionRepository
41 |
42 |             return SupabasePredictionRepository(
43 |                 supabase_url=settings.supabase_url,
44 |                 service_role_key=settings.supabase_service_role_key,
45 |             )
46 |         LOGGER.warning("ENABLE_SUPABASE=true but SUPABASE_URL or SUPABASE_SERVICE_ROLE_KEY is missing. Using NoOp.")
47 |     return NoOpPredictionRepository()

```

backend_fastapi/app/repositories/supabase_prediction_repository.py

```

1 | from __future__ import annotations
2 |
3 | import logging
4 | from typing import Any
5 |
6 | import httpx

```

```

7|
8| from app.schemas.prediction import FrostPredictionHistoryItem, FrostRiskRequest, FrostRiskResponse
9|
10|
11|
12|
13|
14| class SupabasePredictionRepository:
15|     def __init__(
16|         self,
17|         supabase_url: str,
18|         service_role_key: str,
19|         client: httpx.Client | None = None,
20|     ) -> None:
21|         self.supabase_url = supabase_url.rstrip("/")
22|         self.service_role_key = service_role_key
23|         self.client = client or httpx.Client(timeout=10)
24|
25|     def save_prediction(self, request: FrostRiskRequest, response: FrostRiskResponse) -> None:
26|         payload = self._build_insert_payload(request, response)
27|         try:
28|             result = self.client.post(
29|                 f"{self.supabase_url}/rest/v1/frost_predictions",
30|                 headers=self._headers(prefer="return=minimal"),
31|                 json=payload,
32|             )
33|             result.raise_for_status()
34|         except Exception as exc: # noqa: BLE001 - prediction should still be returned if persistence fails.
35|             LOGGER.warning("Could not persist prediction to Supabase: %s", exc)
36|
37|     def list_history(self, limit: int = 20) -> list[FrostPredictionHistoryItem]:
38|         safe_limit = max(1, min(limit, 100))
39|         try:
40|             result = self.client.get(
41|                 f"{self.supabase_url}/rest/v1/frost_predictions",
42|                 headers=self._headers(),
43|                 params={
44|                     "select": (
45|                         "id,district,province,populated_center,latitude,longitude,altitude,"
46|                         "risk_level,confidence,chuno_conditions,recommendation,model_version,"
47|                         "data_sources,created_at"
48|                     ),
49|                     "order": "created_at.desc",
50|                     "limit": str(safe_limit),
51|                 },
52|             )
53|             result.raise_for_status()
54|             rows = result.json()
55|             if not isinstance(rows, list):
56|                 LOGGER.warning("Unexpected Supabase history response shape: %s", type(rows).__name__)
57|                 return []
58|             return [FrostPredictionHistoryItem.model_validate(row) for row in rows]
59|         except Exception as exc: # noqa: BLE001 - history endpoint should degrade gracefully for MVP.
60|             LOGGER.warning("Could not read prediction history from Supabase: %s", exc)
61|             return []
62|
63|     def _headers(self, prefer: str | None = None) -> dict[str, str]:
64|         headers = {
65|             "apikey": self.service_role_key,
66|             "Authorization": f"Bearer {self.service_role_key}",
67|             "Content-Type": "application/json",
68|         }
69|         if prefer:
70|             headers["Prefer"] = prefer
71|         return headers
72|
73|     @staticmethod
74|     def _build_insert_payload(request: FrostRiskRequest, response: FrostRiskResponse) -> dict[str, Any]:
75|         return {
76|             "user_id": None,
77|             "district": request.district,
78|             "province": request.province,
79|             "populated_center": request.populated_center,
80|             "latitude": request.latitude,
81|             "longitude": request.longitude,
82|             "altitude": request.altitude,
83|             "input_payload": request.model_dump(mode="json"),
84|             "risk_level": response.risk_level,
85|             "confidence": response.confidence,
86|             "chuno_conditions": response.chuno_conditions,
87|             "recommendation": response.recommendation,
88|             "model_version": response.model_version,
89|             "data_sources": response.data_sources,
90|         }

```

backend_fastapi/app/schemas/___init___py

```

1| """Pydantic schemas for request and response contracts."""

```

backend_fastapi/app/schemas/alerts.py

```

1| from __future__ import annotations
2|
3| from pydantic import BaseModel, ConfigDict
4|
5|
6| class LivestockRisk(BaseModel):
7|     level: str # alto | medio | bajo
8|     message: str
9|
10|
11| class ClimateAnomaly(BaseModel):
12|     historical_mean: float | None = None
13|     delta: float | None = None
14|     is_unusual: bool = False
15|     message: str
16|
17|
18| class DailyAlertResponse(BaseModel):
19|     latitude: float
20|     longitude: float

```

```

21|     date: str | None = None
22|     risk_level: str # alto | medio | bajo
23|     frost_alert: bool
24|     severity: str # fuerte | moderada | ninguna
25|     temperature_min: float | None = None
26|     title: str
27|     message: str
28|     livestock: LivestockRisk
29|     anomaly: ClimateAnomaly
30|     model_version: str
31|
32|     model_config = ConfigDict(extra="forbid")

```

backend_fastapi/app/schemas/chuno.py

```

1| from __future__ import annotations
2|
3| from pydantic import BaseModel, ConfigDict
4|
5|
6| class ChunoDay(BaseModel):
7|     date: str
8|     temperature_min: float | None = None
9|     temperature_max: float | None = None
10|    humidity_max: float | None = None
11|    cloud_cover_mean: float | None = None
12|    precipitation_sum: float | None = None
13|    tier: str # excelente | bueno | marginal | no_apt
14|    is_good_day: bool
15|
16|
17| class ChunoWindowResponse(BaseModel):
18|     in_season: bool
19|     season_months: list[int]
20|     latitude: float
21|     longitude: float
22|     days: list[ChunoDay]
23|     optimal_window: bool
24|     best_streak: int
25|     message: str
26|     criteria_source: str = "docs/chuno_criteria.md"
27|
28|     model_config = ConfigDict(extra="forbid")

```

backend_fastapi/app/schemas/health.py

```

1| from __future__ import annotations
2|
3| from pydantic import BaseModel, ConfigDict
4|
5|
6| class HealthResponse(BaseModel):
7|     status: str
8|     app_name: str
9|     version: str
10|    model_available: bool
11|
12|    model_config = ConfigDict(extra="forbid")

```

backend_fastapi/app/schemas/ml.py

```

1| from __future__ import annotations
2|
3| from typing import Any
4|
5| from pydantic import BaseModel, ConfigDict
6|
7|
8| class ModelInfoResponse(BaseModel):
9|     model_name: str
10|    version: str
11|    created_at: str
12|    features: list[str]
13|    target: str
14|    metrics: dict[str, float]
15|    dataset_size: int
16|    data_sources: list[str]
17|    limitations: list[str]
18|    n_clusters: int | None = None
19|
20|    model_config = ConfigDict(extra="forbid")
21|
22|
23| class ModelMetadata(ModelInfoResponse):
24|     # Campos del modelo no supervisado (K-Means). extra="ignore" tolera
25|     # metadata Legacy del clasificador supervisado sin romper la carga.
26|     all_model_metrics: dict[str, dict[str, float]] | None = None
27|     all_k_metrics: dict[str, dict[str, float]] | None = None
28|     cluster_tiers: dict[str, str] | None = None
29|     cluster_profiles: dict[str, dict[str, Any]] | None = None
30|
31|     model_config = ConfigDict(extra="ignore")
32|
33|
34| class ClusterProfile(BaseModel):
35|     cluster_id: int
36|     tier: str
37|     size: int
38|     temperature_2m: float
39|     dew_point_2m: float
40|
41|
42| class DistrictCluster(BaseModel):
43|     distrito: str
44|     tier: str
45|     dominant_cluster: int

```

```

46 cold_share: float
47 altitud_estimada: float
48 temperature_2m_mean: float
49 latitud: float | None = None
50 longitud: float | None = None
51
52
53 class ClustersResponse(BaseModel):
54     model_name: str
55     version: str
56     n_clusters: int
57     silhouette: float
58     profiles: list[ClusterProfile]
59     districts: list[DistrictCluster]

```

backend_fastapi/app/schemas/prediction.py

```

1 | from __future__ import annotations
2
3 | from pydantic import BaseModel, ConfigDict, Field
4
5 |
6 | class FrostRiskRequest(BaseModel):
7 |     district: str = Field(..., min_length=1, examples=["Puno"])
8 |     province: str = Field(..., min_length=1, examples=["Puno"])
9 |     populated_center: str | None = Field(default=None, examples=["Centro poblado demo"])
10 |     latitude: float = Field(..., ge=-18.5, le=-13.0, examples=[-15.8402])
11 |     longitude: float = Field(..., ge=-71.5, le=-68.0, examples=[-70.0219])
12 |     altitude: float = Field(..., ge=0, le=7000, examples=[3827])
13 |     rural_population: int = Field(..., ge=0, examples=[1200])
14 |     total_population: int | None = Field(default=None, ge=0, examples=[5000])
15 |     rural_percentage: float | None = Field(default=None, ge=0, le=100, examples=[24.0])
16 |     agricultural_activity: bool = Field(default=True)
17 |     main_crop: str | None = Field(default="papa", min_length=1)
18 |     temperature_min: float = Field(..., ge=-30, le=40, examples=[-2.5])
19 |     temperature_max: float = Field(..., ge=-30, le=45, examples=[12.4])
20 |     feels_like: float = Field(..., ge=-40, le=45, examples=[-4.0])
21 |     humidity: float = Field(..., ge=0, le=100, examples=[68])
22 |     wind_speed: float = Field(..., ge=0, le=200, examples=[7])
23 |     cloud_cover: float = Field(..., ge=0, le=100, examples=[20])
24 |     dew_point: float = Field(..., ge=-40, le=40, examples=[-3.5])
25 |     precipitation: float = Field(..., ge=0, le=500, examples=[0])
26 |     month: int = Field(..., ge=1, le=12, examples=[6])
27 |     hour: int = Field(..., ge=0, le=23, examples=[3])
28 |     hours_below_zero: int = Field(..., ge=0, le=24, examples=[4])
29
30 |     model_config = ConfigDict(extra="forbid")
31
32 |
33 | class FrostRiskResponse(BaseModel):
34 |     risk_level: str
35 |     confidence: float = Field(..., ge=0, le=1)
36 |     recommendation: str
37 |     chuno_conditions: str
38 |     model_version: str
39 |     data_sources: list[str]
40
41 |     model_config = ConfigDict(extra="forbid")
42
43 |
44 | class FrostPredictionHistoryItem(BaseModel):
45 |     id: str
46 |     district: str
47 |     province: str
48 |     populated_center: str | None = None
49 |     latitude: float
50 |     longitude: float
51 |     altitude: float | None = None
52 |     risk_level: str
53 |     confidence: float = Field(..., ge=0, le=1)
54 |     chuno_conditions: str
55 |     recommendation: str
56 |     model_version: str
57 |     data_sources: list[str]
58 |     created_at: str
59
60 |     model_config = ConfigDict(extra="ignore")

```

backend_fastapi/app/schemas/weather.py

```

1 | from __future__ import annotations
2
3 | from pydantic import BaseModel, ConfigDict, Field
4
5 |
6 | class CurrentWeatherResponse(BaseModel):
7 |     latitude: float
8 |     longitude: float
9 |     temperature: float | None = None
10 |     apparent_temperature: float | None = None
11 |     humidity: float | None = None
12 |     wind_speed: float | None = None
13 |     cloud_cover: float | None = None
14 |     dew_point: float | None = None
15 |     precipitation: float | None = None
16 |     observed_at: str | None = None
17 |     provider: str
18 |     source_priority: list[str]
19 |     station_name: str | None = None
20 |     station_distance_km: float | None = None
21 |     fallback_used: bool = False
22 |     limitations: list[str] = Field(default_factory=list)
23
24 |     model_config = ConfigDict(extra="forbid")
25
26 |
27 | class HistoryDay(BaseModel):
28 |     date: str
29 |     temperature_min: float | None = None
30 |     temperature_max: float | None = None
31 |     humidity_mean: float | None = None

```

```

32
33
34 class WeatherHistoryResponse(BaseModel):
35     latitude: float
36     longitude: float
37     start_date: str
38     end_date: str
39     provider: str = "Open-Meteo Archive"
40     days: list[HistoryDay]
41
42     model_config = ConfigDict(extra="forbid")

```

backend_fastapi/app/services/__init__.py

```

1| """Backend service layer."""

```

backend_fastapi/app/services/alert_service.py

```

1| """Lógica de la alerta diaria: helada, riesgo para ganado y anomalía climática.
2|
3| Umbrales de ganado documentados en docs/livestock_criterio.md (crias de alpaca y
4| ovino del altiplano son vulnerables a mortalidad neonatal por helada). Son valores
5| MVP defendibles, no cifras oficiales SENAMHI.
6| """
7|
8| from __future__ import annotations
9|
10| from statistics import mean, pstdev
11|
12| # Umbrales de helada sobre la mínima diaria (°C).
13| STRONG_FROST_MAX = -4.0
14| MILD_FROST_MAX = 0.0
15|
16| # Umbrales para ganado (crias vulnerables).
17| LIVESTOCK_HIGH_MAX = -6.0
18| LIVESTOCK_MEDIUM_MAX = -2.0
19|
20| # Anomalía: se marca inusual si la mínima cae bajo media - K*desviación.
21| ANOMALY_K = 1.5
22|
23|
24| def frost_severity(temp_min: float | None) -> tuple[str, str, bool, str, str]:
25|     """Devuelve (risk_level, severity, alert, title, message) para heladas."""
26|     if temp_min is not None and temp_min <= STRONG_FROST_MAX:
27|         return (
28|             "alto",
29|             "fuerte",
30|             True,
31|             "Riesgo de helada fuerte",
32|             "Hoy hay riesgo de helada fuerte. Se recomienda resguardar el ganado y proteger los cultivos.",
33|         )
34|     if temp_min is not None and temp_min <= MILD_FROST_MAX:
35|         return (
36|             "medio",
37|             "moderada",
38|             True,
39|             "Riesgo de helada moderada",
40|             "Hoy hay riesgo de helada moderada. Monitoree la temperatura nocturna y prepare medidas preventivas.",
41|         )
42|     return (
43|         "bajo",
44|         "ninguna",
45|         False,
46|         "Sin alerta de helada",
47|         "No se prevé helada significativa para hoy. Mantenga seguimiento ante cambios bruscos.",
48|     )
49|
50|
51| def livestock_risk(temp_min: float | None) -> dict[str, str]:
52|     """Riesgo específico para ganado altoandino (crias de alpaca y ovino)."""
53|     if temp_min is not None and temp_min <= LIVESTOCK_HIGH_MAX:
54|         return {
55|             "level": "alto",
56|             "message": (
57|                 "Riesgo alto para el ganado: proteja crías de alpaca y ovino esta noche; "
58|                 "peligro de mortalidad neonatal por frío."
59|             ),
60|         }
61|     if temp_min is not None and temp_min <= LIVESTOCK_MEDIUM_MAX:
62|         return {
63|             "level": "medio",
64|             "message": (
65|                 "Riesgo medio para el ganado: abrigue a las crías y revise cobertizos antes del anochecer."
66|             ),
67|         }
68|     return {
69|         "level": "bajo",
70|         "message": "Sin riesgo relevante para el ganado por temperatura esta noche.",
71|     }
72|
73|
74| def anomaly(temp_min_today: float | None, history_rows: list[dict]) -> dict:
75|     """Compara la mínima de hoy con la climatología reciente (últimos ~30 días)."""
76|     values = [
77|         row["temperature_2m_min"]
78|         for row in history_rows
79|         if row.get("temperature_2m_min") is not None
80|     ]
81|     if temp_min_today is None or len(values) < 5:
82|         return {
83|             "historical_mean": None,
84|             "delta": None,
85|             "is_unusual": False,
86|             "message": "Sin datos históricos suficientes para evaluar anomalías.",
87|         }
88|
89|     historical_mean = mean(values)
90|     deviation = pstdev(values)
91|     delta = temp_min_today - historical_mean
92|     is_unusual = deviation > 0 and temp_min_today < historical_mean - ANOMALY_K * deviation
93|

```

```

94     if is_unusual:
95         message = (
96             f"Helada inusual: {abs(delta):.1f} °C por debajo de lo normal para estas fechas."
97         )
98     else:
99         message = "Temperatura dentro de lo normal para la temporada."
100
101     return {
102         "historical_mean": round(historical_mean, 1),
103         "delta": round(delta, 1),
104         "is_unusual": is_unusual,
105         "message": message,
106     }

```

backend_fastapi/app/services/chuno_service.py

```

1  """Reglas del módulo estacional de chuño.
2
3  Umbrales documentados en docs/chuno_criterio.md (~mayoagosto; congelamiento
4  nocturno ≤-5 °C; secado con cielo despejado, aire seco y sin lluvia).
5  """
6
7  from __future__ import annotations
8
9  from app.schemas.chuno import ChunoDay, ChunoWindowResponse
10
11  SEASON_MONTHS = [5, 6, 7, 8]
12  MIN_CONSECUTIVE_GOOD_DAYS = 3
13
14  # Umbrales base ("buen día")
15  FREEZE_MAX = -5.0
16  PRECIP_MAX = 0.2
17  CLOUD_MAX = 40.0
18  HUMIDITY_MAX = 60.0
19
20  # Umbrales "excelente"
21  FREEZE_EXCELLENT = -8.0
22  CLOUD_EXCELLENT = 20.0
23  HUMIDITY_EXCELLENT = 50.0
24
25
26  def _classify(temp_min, precip, cloud, humidity) -> tuple[str, bool]:
27      if temp_min is None:
28          return "no_apt", False
29      precip = precip if precip is not None else 0.0
30      cloud = cloud if cloud is not None else 100.0
31      humidity = humidity if humidity is not None else 100.0
32
33      good = temp_min <= FREEZE_MAX and precip <= PRECIP_MAX and cloud <= CLOUD_MAX and humidity <= HUMIDITY_MAX
34      if good:
35          excellent = (
36              temp_min <= FREEZE_EXCELLENT
37              and cloud <= CLOUD_EXCELLENT
38              and humidity <= HUMIDITY_EXCELLENT
39              and precip <= 0.0
40          )
41          return ("excelente" if excellent else "bueno"), True
42
43      if temp_min > -2.0 or precip > 1.0:
44          return "no_apt", False
45      return "marginal", False
46
47
48  def _best_streak(days: list[ChunoDay]) -> int:
49      best = current = 0
50      for day in days:
51          current = current + 1 if day.is_good_day else 0
52          best = max(best, current)
53      return best
54
55
56  def evaluate_window(
57      latitude: float,
58      longitude: float,
59      current_month: int,
60      forecast_rows: list[dict],
61  ) -> ChunoWindowResponse:
62      in_season = current_month in SEASON_MONTHS
63
64      days: list[ChunoDay] = []
65      for row in forecast_rows:
66          tier, is_good = _classify(
67              row.get("temperature_2m_min"),
68              row.get("precipitation_sum"),
69              row.get("cloud_cover_mean"),
70              row.get("relative_humidity_2m_max"),
71          )
72          days.append(
73              ChunoDay(
74                  date=row.get("date"),
75                  temperature_min=row.get("temperature_2m_min"),
76                  temperature_max=row.get("temperature_2m_max"),
77                  humidity_max=row.get("relative_humidity_2m_max"),
78                  cloud_cover_mean=row.get("cloud_cover_mean"),
79                  precipitation_sum=row.get("precipitation_sum"),
80                  tier=tier,
81                  is_good_day=is_good,
82              )
83          )
84
85      best_streak = _best_streak(days)
86      optimal = in_season and best_streak >= MIN_CONSECUTIVE_GOOD_DAYS
87
88      if not in_season:
89          message = "Fuera de temporada de chuño (~mayoagosto). El módulo se reactiva en la estación seca fría."
90      elif optimal:
91          message = (
92              f"Ventana óptima detectada: {best_streak} días consecutivos aptos para congelar y secar chuño."
93          )
94      else:
95          message = (
96              "En temporada, pero aún sin una ventana de 3 días consecutivos aptos. "
97              "Monitoree el pronóstico nocturno."
98          )
99
100     return ChunoWindowResponse(

```



```

101|         in_season=in_season,
102|         season_months=SEASON_MONTHS,
103|         latitude=latitude,
104|         longitude=longitude,
105|         days=days,
106|         optimal_window=optimal,
107|         best_streak=best_streak,
108|         message=message,
109|     )

```

backend__fastapi/app/services/model_registry.py

```

1| from __future__ import annotations
2|
3| import json
4| import logging
5| from functools import lru_cache
6| from pathlib import Path
7| from typing import Any
8|
9| import joblib
10|
11| from app.core.config import settings
12| from app.core.exceptions import ModelLoadError
13| from app.schemas.ml import ModelMetadata
14|
15|
16| LOGGER = logging.getLogger(__name__)
17|
18|
19| class ModelRegistry:
20|     def __init__(self, model_path: Path, metadata_path: Path) -> None:
21|         self.model_path = model_path
22|         self.metadata_path = metadata_path
23|         self._model: Any | None = None
24|         self._metadata: ModelMetadata | None = None
25|
26|     @property
27|     def model(self) -> Any:
28|         if self._model is None:
29|             self._model = self._load_model()
30|         return self._model
31|
32|     @property
33|     def metadata(self) -> ModelMetadata:
34|         if self._metadata is None:
35|             self._metadata = self._load_metadata()
36|         return self._metadata
37|
38|     def _load_model(self) -> Any:
39|         if not self.model_path.exists():
40|             raise ModelLoadError(f"Model artifact not found at {self.model_path}")
41|
42|         try:
43|             LOGGER.info("Loading model artifact from %s", self.model_path)
44|             return joblib.load(self.model_path)
45|         except Exception as exc: # noqa: BLE001 - convert registry failures into API-safe errors.
46|             raise ModelLoadError("Could not load the registered model artifact.") from exc
47|
48|     def _load_metadata(self) -> ModelMetadata:
49|         if not self.metadata_path.exists():
50|             raise ModelLoadError(f"Model metadata not found at {self.metadata_path}")
51|
52|         try:
53|             raw_metadata = json.loads(self.metadata_path.read_text(encoding="utf-8"))
54|             return ModelMetadata.model_validate(raw_metadata)
55|         except Exception as exc: # noqa: BLE001 - convert malformed metadata into API-safe errors.
56|             raise ModelLoadError("Could not load model metadata.") from exc
57|
58|
59| @lru_cache(maxsize=1)
60| def get_model_registry() -> ModelRegistry:
61|     return ModelRegistry(settings.model_path, settings.model_metadata_path)

```

backend__fastapi/app/services/prediction__service.py

```

1| from __future__ import annotations
2|
3| import logging
4| from functools import lru_cache
5| from typing import Any
6|
7| import pandas as pd
8|
9| from app.core.exceptions import PredictionError
10| from app.schemas.prediction import FrostRiskRequest, FrostRiskResponse
11| from app.services.model_registry import ModelRegistry, get_model_registry
12| from app.services.recommendation_service import build_recommendation
13|
14|
15| LOGGER = logging.getLogger(__name__)
16|
17|
18| class PredictionService:
19|     def __init__(self, registry: ModelRegistry) -> None:
20|         self.registry = registry
21|
22|     def predict(self, payload: FrostRiskRequest) -> FrostRiskResponse:
23|         metadata = self.registry.metadata
24|         feature_frame = self._to_feature_frame(payload, metadata.features)
25|         tier_map = metadata.cluster_tiers or {}
26|
27|         try:
28|             model = self.registry.model
29|             cluster_id = int(model.predict(feature_frame)[0])
30|             risk_level = tier_map.get(str(cluster_id), "medio")
31|             confidence = self._cluster_confidence(model, feature_frame)
32|         except Exception as exc: # noqa: BLE001 - expose stable API error instead of model internals.
33|             LOGGER.exception("Prediction failed.")
34|             raise PredictionError("The model could not generate a prediction for the request.") from exc
35|
36|     def _to_feature_frame(self, payload: FrostRiskRequest, features: list[str]) -> pd.DataFrame:
37|         feature_map = {feature: getattr(payload, feature) for feature in features}
38|         return pd.DataFrame([feature_map])
39|
40|     def _cluster_confidence(self, model: Any, feature_frame: pd.DataFrame) -> float:
41|         cluster_id = int(model.predict(feature_frame)[0])
42|         risk_level = tier_map.get(str(cluster_id), "medio")
43|         confidence = self._cluster_confidence(model, feature_frame)

```

```

36         recommendation, chuno_conditions = build_recommendation(risk_level, payload)
37         return FrostRiskResponse(
38             risk_level=risk_level,
39             confidence=confidence,
40             recommendation=recommendation,
41             chuno_conditions=chuno_conditions,
42             model_version=metadata.version,
43             data_sources=metadata.data_sources,
44         )
45
46     def _to_feature_frame(self, payload: FrostRiskRequest, feature_names: list[str]) -> pd.DataFrame:
47         # Vector para el modelo no supervisado: La temperatura mínima es la señal
48         # de helada relevante y se mapea a temperature_2m.
49         feature_values: dict[str, Any] = {
50             "latitud": payload.latitude,
51             "longitud": payload.longitude,
52             "altitud_estimada": payload.altitude,
53             "temperature_2m": payload.temperature_min,
54             "relative_humidity_2m": payload.humidity,
55             "apparent_temperature": payload.feels_like,
56             "dew_point_2m": payload.dew_point,
57             "precipitation": payload.precipitation,
58             "cloud_cover": payload.cloud_cover,
59             "wind_speed_10m": payload.wind_speed,
60             "mes": payload.month,
61             "hora": payload.hour,
62         }
63
64         missing = [name for name in feature_names if name not in feature_values]
65         if missing:
66             raise PredictionError(f"Request cannot build required model features: {missing}")
67
68         return pd.DataFrame([{"name": feature_values[name] for name in feature_names}])
69
70     @staticmethod
71     def _cluster_confidence(model: Any, feature_frame: pd.DataFrame) -> float:
72         """Confianza por cercanía relativa al centroide más cercano (KMeans.transform)."""
73         if not hasattr(model, "transform"):
74             return 1.0
75
76         distances = model.transform(feature_frame)[0]
77         inverse = 1.0 / (distances + 1e-9)
78         total = float(inverse.sum())
79         if total <= 0:
80             return 1.0
81         return float(inverse.max() / total)
82
83     @lru_cache(maxsize=1)
84     def get_prediction_service() -> PredictionService:
85         return PredictionService(get_model_registry())
86

```

backend_fastapi/app/services/recommendation_service.py

```

1  from __future__ import annotations
2
3  from app.schemas.prediction import FrostRiskRequest
4
5
6  def build_recommendation(risk_level: str, payload: FrostRiskRequest) -> tuple[str, str]:
7      crop = payload.main_crop or "cultivos sensibles"
8
9      if risk_level == "alto":
10         recommendation = (
11             "Existe alto riesgo de helada. Se recomienda proteger cultivos sensibles, "
12             f"priorizar vigilancia de {crop} y revisar ganado o infraestructura expuesta."
13         )
14     elif risk_level == "medio":
15         recommendation = (
16             "Existe riesgo medio de helada. Se recomienda monitorear la temperatura nocturna "
17             "y preparar medidas preventivas si el descenso continua."
18         )
19     else:
20         recommendation = (
21             "El riesgo de helada es bajo para las condiciones enviadas. Mantenga seguimiento "
22             "si hay cambios bruscos de viento, nubosidad o temperatura."
23         )
24
25     if payload.temperature_min <= 0 and payload.hours_below_zero >= 3 and payload.precipitation <= 1:
26         chuno_conditions = "favorables"
27         recommendation += " Las condiciones pueden ser favorables para la produccion de chuno."
28     elif payload.temperature_min <= 3:
29         chuno_conditions = "posibles"
30     else:
31         chuno_conditions = "no_favorables"
32
33     return recommendation, chuno_conditions

```

backend_fastapi/app/services/weather_providers.py

```

1  from __future__ import annotations
2
3  import logging
4  import math
5  import time
6  from dataclasses import dataclass
7  from typing import Protocol
8
9  import httpx
10
11  from app.schemas.weather import CurrentWeatherResponse
12
13  LOGGER = logging.getLogger(__name__)
14
15
16  @dataclass(frozen=True)
17  class WeatherQuery:
18     latitude: float
19     longitude: float
20
21

```

```

22
23 class WeatherProvider(Protocol):
24     name: str
25
26     def get_current_weather(self, query: WeatherQuery) -> CurrentWeatherResponse | None:
27         """Return current weather or None when the provider has no usable data."""
28
29
30 class SenamhiProvider:
31     name = "SENAMHI"
32
33     _puno_reference_stations = (
34         ("Puno", -15.8402, -70.0219, 3827.0),
35         ("Juliaca", -15.4997, -70.1333, 3825.0),
36         ("Ilave", -16.0833, -69.6667, 3850.0),
37     )
38
39     def get_current_weather(self, query: WeatherQuery) -> CurrentWeatherResponse | None:
40         station_name, distance_km = self._nearest_station(query)
41         logger.info(
42             "SENAMHI provider prepared nearest station=%s distance=%.2f km; no stable public API configured.",
43             station_name,
44             distance_km,
45         )
46         return None
47
48     def nearest_station_metadata(self, query: WeatherQuery) -> tuple[str, float]:
49         return self._nearest_station(query)
50
51     def _nearest_station(self, query: WeatherQuery) -> tuple[str, float]:
52         nearest = min(
53             self._puno_reference_stations,
54             key=lambda station: _distance_km(query.latitude, query.longitude, station[1], station[2]),
55         )
56         return nearest[0], _distance_km(query.latitude, query.longitude, nearest[1], nearest[2])
57
58
59 class OpenMeteoProvider:
60     name = "Open-Meteo"
61
62     def __init__(self, client: httpx.Client | None = None) -> None:
63         self._client = client or httpx.Client(timeout=12)
64
65     def get_current_weather(self, query: WeatherQuery) -> CurrentWeatherResponse | None:
66         response = self._client.get(
67             "https://api.open-meteo.com/v1/forecast",
68             params={
69                 "latitude": query.latitude,
70                 "longitude": query.longitude,
71                 "current": ", ".join(
72                     [
73                         "temperature_2m",
74                         "relative_humidity_2m",
75                         "apparent_temperature",
76                         "dew_point_2m",
77                         "precipitation",
78                         "cloud_cover",
79                         "wind_speed_10m",
80                     ],
81                 ),
82                 "timezone": "auto",
83             },
84         )
85         response.raise_for_status()
86         payload = response.json()
87         current = payload.get("current") or {}
88         return CurrentWeatherResponse(
89             latitude=query.latitude,
90             longitude=query.longitude,
91             temperature=_num(current.get("temperature_2m")),
92             apparent_temperature=_num(current.get("apparent_temperature")),
93             humidity=_num(current.get("relative_humidity_2m")),
94             wind_speed=_num(current.get("wind_speed_10m")),
95             cloud_cover=_num(current.get("cloud_cover")),
96             dew_point=_num(current.get("dew_point_2m")),
97             precipitation=_num(current.get("precipitation")),
98             observed_at=current.get("time"),
99             provider=self.name,
100             source_priority=["SENAMHI", "Open-Meteo"],
101             fallback_used=True,
102             limitations=[
103                 "SENAMHI is prioritized conceptually, but no stable unauthenticated API is configured in this MVP.",
104                 "Open-Meteo is used as fallback/global weather source for current app predictions.",
105             ],
106         )
107
108
109     def get_daily_forecast(self, query: WeatherQuery, days: int = 7) -> list[dict[str, float | str | None]]:
110         """Pronóstico diario Open-Meteo para el módulo de chuño."""
111         response = self._client.get(
112             "https://api.open-meteo.com/v1/forecast",
113             params={
114                 "latitude": query.latitude,
115                 "longitude": query.longitude,
116                 "daily": ", ".join(
117                     [
118                         "temperature_2m_min",
119                         "temperature_2m_max",
120                         "relative_humidity_2m_max",
121                         "cloud_cover_mean",
122                         "precipitation_sum",
123                     ],
124                 ),
125                 "forecast_days": days,
126                 "timezone": "auto",
127             },
128         )
129         response.raise_for_status()
130         return _rows_from_daily(response.json()).get("daily") or {}
131
132     def get_history(
133         self, query: WeatherQuery, start_date: str, end_date: str
134     ) -> list[dict[str, float | str | None]]:
135         """Serie histórica diaria via Open-Meteo Archive API (sin API key)."""
136         response = self._client.get(
137             "https://archive-api.open-meteo.com/v1/archive",
138             params={
139                 "latitude": query.latitude,
140                 "longitude": query.longitude,
141                 "start_date": start_date,
142                 "end_date": end_date,

```

```

143         "daily": ", ".join(
144             [
145                 "temperature_2m_min",
146                 "temperature_2m_max",
147                 "relative_humidity_2m_mean",
148             ],
149         ),
150         "timezone": "auto",
151     },
152 )
153 response.raise_for_status()
154 return _rows_from_daily(response.json().get("daily") or {})
155
156
157 class HybridWeatherProvider:
158     name = "HybridWeatherProvider"
159
160     def __init__(
161         self,
162         senamhi_provider: SenamhiProvider | None = None,
163         open_meteo_provider: OpenMeteoProvider | None = None,
164         cache_ttl_seconds: int = 600,
165     ) -> None:
166         self._senamhi = senamhi_provider or SenamhiProvider()
167         self._open_meteo = open_meteo_provider or OpenMeteoProvider()
168         self._cache_ttl_seconds = cache_ttl_seconds
169         self._cache: dict[tuple[float, float], tuple[float, CurrentWeatherResponse]] = {}
170
171     def get_current_weather(self, query: WeatherQuery) -> CurrentWeatherResponse:
172         cache_key = (round(query.latitude, 4), round(query.longitude, 4))
173         cached = self._cache.get(cache_key)
174         now = time.time()
175         if cached and now - cached[0] < self._cache_ttl_seconds:
176             return cached[1]
177
178         station_name, station_distance = self._senamhi.nearest_station_metadata(query)
179         try:
180             senamhi_weather = self._senamhi.get_current_weather(query)
181             if senamhi_weather is not None:
182                 result = senamhi_weather
183             else:
184                 result = self._fallback_to_open_meteo(query, station_name, station_distance)
185         except Exception as exc: # noqa: BLE001 - weather lookup must not break predictions.
186             LOGGER.warning("SENAMHI provider failed; using Open-Meteo fallback: %s", exc)
187             result = self._fallback_to_open_meteo(query, station_name, station_distance)
188
189         self._cache[cache_key] = (now, result)
190         return result
191
192     def get_daily_forecast(self, query: WeatherQuery, days: int = 7) -> list[dict[str, float | str | None]]:
193         return self._open_meteo.get_daily_forecast(query, days=days)
194
195     def get_history(
196         self, query: WeatherQuery, start_date: str, end_date: str
197     ) -> list[dict[str, float | str | None]]:
198         return self._open_meteo.get_history(query, start_date, end_date)
199
200     def _fallback_to_open_meteo(
201         self,
202         query: WeatherQuery,
203         station_name: str,
204         station_distance: float,
205     ) -> CurrentWeatherResponse:
206         result = self._open_meteo.get_current_weather(query)
207         if result is None:
208             return CurrentWeatherResponse(
209                 latitude=query.latitude,
210                 longitude=query.longitude,
211                 provider="unavailable",
212                 source_priority=["SENAMHI", "Open-Meteo"],
213                 fallback_used=True,
214                 station_name=station_name,
215                 station_distance_km=station_distance,
216                 limitations=["No weather provider returned usable data."],
217             )
218         return result.model_copy(
219             update={
220                 "station_name": station_name,
221                 "station_distance_km": station_distance,
222             },
223         )
224
225
226 def _distance_km(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
227     radius_km = 6371.0
228     d_lat = math.radians(lat2 - lat1)
229     d_lon = math.radians(lon2 - lon1)
230     a = (
231         math.sin(d_lat / 2) ** 2
232         + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(d_lon / 2) ** 2
233     )
234     return round(radius_km * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a)), 2)
235
236
237 def _num(value: object) -> float | None:
238     if value is None:
239         return None
240     return float(value)
241
242
243 def _rows_from_daily(daily: dict[str, list]) -> list[dict[str, float | str | None]]:
244     """Transpone el bloque 'daily' de Open-Meteo (columnas) en filas por fecha."""
245     dates = daily.get("time") or []
246     keys = [key for key in daily if key != "time"]
247     rows: list[dict[str, float | str | None]] = []
248     for index, date in enumerate(dates):
249         row: dict[str, float | str | None] = {"date": date}
250         for key in keys:
251             values = daily.get(key) or []
252             row[key] = _num(values[index]) if index < len(values) else None
253         rows.append(row)
254     return rows

```

backend_fastapi/tests/test_api.py

```

1| from __future__ import annotations

```

```

2
3 import httpx
4 from fastapi.testclient import TestClient
5
6 from app.main import app
7 from app.repositories.predictions import NoOpPredictionRepository
8 from app.repositories.supabase_prediction_repository import SupabasePredictionRepository
9 from app.schemas.prediction import FrostRiskRequest, FrostRiskResponse
10
11
12 client = TestClient(app)
13
14
15 def sample_request() -> FrostRiskRequest:
16     return FrostRiskRequest(
17         district="Puno",
18         province="Puno",
19         populated_center="Centro poblado demo",
20         latitude=-15.8402,
21         longitude=-70.0219,
22         altitude=3827,
23         rural_population=1200,
24         agricultural_activity=True,
25         main_crop="papa",
26         temperature_min=-2.5,
27         temperature_max=12.4,
28         feels_like=-4.0,
29         humidity=68,
30         wind_speed=7,
31         cloud_cover=20,
32         dew_point=-3.5,
33         precipitation=0,
34         month=6,
35         hour=3,
36         hours_below_zero=4,
37     )
38
39
40 def sample_response() -> FrostRiskResponse:
41     return FrostRiskResponse(
42         risk_level="alto",
43         confidence=0.98,
44         recommendation="Existe alto riesgo de helada.",
45         chuno_conditions="favorables",
46         model_version="v0.1.0",
47         data_sources=["Open-Meteo Historical API"],
48     )
49
50
51 def test_health_check() -> None:
52     response = client.get("/health")
53
54     assert response.status_code == 200
55     body = response.json()
56     assert body["status"] in {"ok", "degraded"}
57     assert "model_available" in body
58
59
60 def test_model_info_returns_cluster_metadata() -> None:
61     response = client.get("/ml/model-info")
62
63     assert response.status_code == 200
64     body = response.json()
65     assert body["model_name"] == "KMeans"
66     assert body["version"].endswith("clustering")
67     assert "silhouette" in body["metrics"]
68     assert body["n_clusters"] >= 2
69
70
71 def test_clusters_endpoint_returns_profiles_and_districts() -> None:
72     response = client.get("/ml/clusters")
73
74     assert response.status_code == 200
75     body = response.json()
76     assert body["n_clusters"] == len(body["profiles"])
77     assert body["profiles"], "expected at least one cluster profile"
78     assert body["districts"], "expected district groupings"
79     tiers = {district["tier"] for district in body["districts"]}
80     assert tiers.issubset({"alto", "medio", "bajo"})
81
82
83 def test_predict_frost_risk_returns_cluster_tier() -> None:
84     payload = {
85         "district": "Puno",
86         "province": "Puno",
87         "populated_center": "Centro poblado demo",
88         "latitude": -15.8402,
89         "longitude": -70.0219,
90         "altitude": 3827,
91         "rural_population": 1200,
92         "agricultural_activity": True,
93         "main_crop": "papa",
94         "temperature_min": -2.5,
95         "temperature_max": 12.4,
96         "feels_like": -4.0,
97         "humidity": 68,
98         "wind_speed": 7,
99         "cloud_cover": 20,
100        "dew_point": -3.5,
101        "precipitation": 0,
102        "month": 6,
103        "hour": 3,
104        "hours_below_zero": 4,
105    }
106
107     response = client.post("/predict/frost-risk", json=payload)
108
109     assert response.status_code == 200
110     body = response.json()
111     assert body["risk_level"] in {"alto", "medio", "bajo"}
112     assert 0 <= body["confidence"] <= 1
113     assert body["model_version"].endswith("clustering")
114     assert body["chuno_conditions"] == "favorables"
115
116
117 def test_prediction_history_returns_empty_without_supabase() -> None:
118     response = client.get("/predictions/history")
119
120     assert response.status_code == 200
121     assert response.json() == []
122

```

```

123
124 def test_current_weather_contract(monkeypatch) -> None:
125     from app.api.routes import weather
126     from app.schemas.weather import CurrentWeatherResponse
127
128     class FakeWeatherProvider:
129         def get_current_weather(self, query):
130             return CurrentWeatherResponse(
131                 latitude=query.latitude,
132                 longitude=query.longitude,
133                 temperature=-1.5,
134                 humidity=70,
135                 provider="Open-Meteo",
136                 source_priority=["SENAMHI", "Open-Meteo"],
137                 fallback_used=True,
138             )
139
140     weather.get_weather_provider.cache_clear()
141     monkeypatch.setattr(weather, "get_weather_provider", lambda: FakeWeatherProvider())
142
143     response = client.get("/weather/current?latitude=-15.8402&longitude=-70.0219")
144
145     assert response.status_code == 200
146     body = response.json()
147     assert body["provider"] == "Open-Meteo"
148     assert body["fallback_used"] is True
149
150
151 class _FakeForecastProvider:
152     def __init__(self, rows, history=None):
153         self._rows = rows
154         self._history = history if history is not None else rows
155
156     def get_daily_forecast(self, query, days=7):
157         return self._rows
158
159     def get_history(self, query, start_date, end_date):
160         return self._history
161
162
163 def test_chuno_window_flags_optimal_streak(monkeypatch) -> None:
164     from app.api.routes import chuno
165
166     cold_day = {
167         "date": "2026-06-10",
168         "temperature_2m_min": -7.0,
169         "temperature_2m_max": 16.0,
170         "relative_humidity_2m_max": 45.0,
171         "cloud_cover_mean": 15.0,
172         "precipitation_sum": 0.0,
173     }
174     rows = [dict(cold_day, date=f"2026-06-{10 + i}") for i in range(4)]
175     monkeypatch.setattr(chuno, "get_weather_provider", lambda: _FakeForecastProvider(rows))
176
177     response = client.get("/chuno/window?latitude=-15.8402&longitude=-70.0219")
178
179     assert response.status_code == 200
180     body = response.json()
181     assert body["best_streak"] >= 3
182     assert all(day["is_good_day"] for day in body["days"])
183     # optimal_window depende de la temporada real; el streak siempre debe reflejar los días buenos.
184     assert body["days"][0]["tier"] in {"excelente", "bueno"}
185
186
187 def test_alert_today_strong_frost_with_livestock_and_anomaly(monkeypatch) -> None:
188     from app.api.routes import alerts
189
190     rows = [{"date": "2026-06-10", "temperature_2m_min": -7.0}]
191     # Historia templada con varianza -> La mínima de hoy (-7) es una anomalía clara.
192     history = [
193         {"date": f"2026-05-{d:02d}", "temperature_2m_min": 0.0 if d % 2 else 2.0}
194         for d in range(1, 16)
195     ]
196     monkeypatch.setattr(
197         alerts, "get_weather_provider", lambda: _FakeForecastProvider(rows, history=history)
198     )
199
200     response = client.get("/alerts/today?latitude=-15.8402&longitude=-70.0219")
201
202     assert response.status_code == 200
203     body = response.json()
204     assert body["frost_alert"] is True
205     assert body["severity"] == "fuerte"
206     assert body["risk_level"] == "alto"
207     assert body["livestock"]["level"] == "alto"
208     assert body["anomaly"]["is_unusual"] is True
209     assert body["anomaly"]["historical_mean"] is not None
210     assert body["anomaly"]["delta"] < 0
211
212
213 def test_weather_history_contract(monkeypatch) -> None:
214     from app.api.routes import weather
215
216     rows = [
217         {
218             "date": "2026-06-01",
219             "temperature_2m_min": -5.0,
220             "temperature_2m_max": 14.0,
221             "relative_humidity_2m_mean": 55.0,
222         }
223     ]
224     weather.get_weather_provider.cache_clear()
225     monkeypatch.setattr(weather, "get_weather_provider", lambda: _FakeForecastProvider(rows))
226
227     response = client.get(
228         "/weather/history?latitude=-15.8402&longitude=-70.0219&start=2026-06-01&end=2026-06-01"
229     )
230
231     assert response.status_code == 200
232     body = response.json()
233     assert body["days"][0]["temperature_min"] == -5.0
234     assert body["days"][0]["humidity_mean"] == 55.0
235
236
237 def test_noop_repository_does_not_persist_and_returns_empty_history() -> None:
238     repository = NoOpPredictionRepository()
239
240     repository.save_prediction(sample_request(), sample_response())
241
242     assert repository.list_history(limit=10) == []
243

```

```

244
245 def test_supabase_repository_insert_failure_does_not_raise(caplog) -> None:
246     def handler(_: httpx.Request) -> httpx.Response:
247         return httpx.Response(status_code=500, json={"message": "database unavailable"})
248
249     client_with_error = httpx.Client(transport=httpx.MockTransport(handler))
250     repository = SupabasePredictionRepository(
251         supabase_url="https://example.supabase.co",
252         service_role_key="test-service-role-key",
253         client=client_with_error,
254     )
255
256     repository.save_prediction(sample_request(), sample_response())
257
258     assert "Could not persist prediction to Supabase" in caplog.text
259
260
261 def test_supabase_repository_history_failure_returns_empty() -> None:
262     def handler(_: httpx.Request) -> httpx.Response:
263         return httpx.Response(status_code=401, json={"message": "unauthorized"})
264
265     client_with_error = httpx.Client(transport=httpx.MockTransport(handler))
266     repository = SupabasePredictionRepository(
267         supabase_url="https://example.supabase.co",
268         service_role_key="test-service-role-key",
269         client=client_with_error,
270     )
271
272     assert repository.list_history(limit=10) == []

```

3. Cliente Flutter

app_flutter/lib/app.dart

```

1| import 'package:flutter/material.dart';
2| import 'package:flutter_localizations/flutter_localizations.dart';
3|
4| import 'core/api/api_client.dart';
5| import 'core/api/frost_api_service.dart';
6| import 'core/i18n/app_language.dart';
7| import 'core/settings/user_settings.dart';
8| import 'core/theme/app_theme.dart';
9| import 'core/theme/theme_mode_scope.dart';
10| import 'features/shell/app_shell.dart';
11|
12| class FrostPunoApp extends StatefulWidget {
13|   const FrostPunoApp({super.key});
14|
15|   @override
16|   State<FrostPunoApp> createState() => _FrostPunoAppState();
17| }
18|
19| class _FrostPunoAppState extends State<FrostPunoApp> {
20|   late final ApiClient _apiClient = ApiClient();
21|   late final FrostApiService _apiService = FrostApiService(_apiClient);
22|   ThemeMode _themeMode = ThemeMode.system;
23|
24|   @override
25|   Widget build(BuildContext context) {
26|     // Reconstruye toda la app al cambiar idioma/preferencias del usuario.
27|     return AnimatedBuilder(
28|       animation: UserSettings.instance,
29|       builder: (context, _) {
30|         return ThemeModeScope(
31|           themeMode: _themeMode,
32|           onThemeModeChanged: (mode) => setState(() => _themeMode = mode),
33|           child: MaterialApp(
34|             title: 'FrostPuno',
35|             debugShowCheckedModeBanner: false,
36|             theme: AppTheme.light(),
37|             darkTheme: AppTheme.dark(),
38|             themeMode: _themeMode,
39|             locale: Locale(UserSettings.instance.language.code),
40|             supportedLocales: const [Locale('es'), Locale('en')],
41|             localizationsDelegates: const [
42|               GlobalMaterialLocalizations.delegate,
43|               GlobalWidgetsLocalizations.delegate,
44|               GlobalCupertinoLocalizations.delegate,
45|             ],
46|             home: AppShell(apiService: _apiService),
47|           ),
48|         );
49|       },
50|     );
51|   }
52| }

```

app_flutter/lib/core/api/api_client.dart

```

1| import 'dart:convert';
2|
3| import 'package:http/http.dart' as http;
4|
5| import '../config/app_config.dart';
6|
7| class ApiClient {
8|   ApiClient({http.Client? client, String? baseUrl})
9|     : _client = client ?? http.Client(),
10|       _baseUrl = baseUrl ?? AppConfig.apiBaseUrl;
11|
12|   final http.Client _client;
13|   final String _baseUrl;
14|
15|   Future<Map<String, dynamic>> getJson(String path) async {
16|     final response = await _client.get(_uri(path));
17|     return _decodeObject(response);
18|   }
19| }

```

```

20 Future<List<dynamic>> getList(String path) async {
21   final response = await _client.get(_uri(path));
22   final decoded = _decode(response);
23   if (decoded is List) {
24     return decoded;
25   }
26   throw ApiException('La API devolvió una respuesta inesperada.');
```

```

27 }
28
29 Future<Map<String, dynamic>> postJson(
30   String path,
31   Map<String, dynamic> body,
32 ) async {
33   final response = await _client.post(
34     _uri(path),
35     headers: const {'Content-Type': 'application/json'},
36     body: jsonEncode(body),
37   );
38   return _decodeObject(response);
39 }
40
41 Uri _uri(String path) => Uri.parse('${_baseUrl}$path');
```

```

42
43 Map<String, dynamic> _decodeObject(http.Response response) {
44   final decoded = _decode(response);
45   if (decoded is Map<String, dynamic>) {
46     return decoded;
47   }
48   throw ApiException('La API devolvió una respuesta inesperada.');
```

```

49 }
50
51 dynamic _decode(http.Response response) {
52   if (response.statusCode < 200 || response.statusCode >= 300) {
53     throw ApiException('Error ${response.statusCode}: ${response.body}');
```

```

54   }
55   if (response.body.isEmpty) {
56     return <String, dynamic>{};
57   }
58   return jsonDecode(response.body);
59 }
60 }
61
62 class ApiException implements Exception {
63   ApiException(this.message);
64
65   final String message;
66
67   @override
68   String toString() => message;
69 }
```

app_flutter/lib/core/api/frost_api_service.dart

```

1| import '../features/alerts/models/daily_alert.dart';
2| import '../features/charts/models/history_day.dart';
3| import '../features/chuno/models/chuno_window.dart';
4| import '../features/clusters/models/cluster_info.dart';
5| import '../features/history/models/prediction_history_item.dart';
6| import '../features/model_info/models/model_info.dart';
7| import '../features/prediction/models/frost_risk_request.dart';
8| import '../features/prediction/models/frost_risk_response.dart';
9| import '../models/current_weather.dart';
10| import '../models/health_status.dart';
11| import 'api_client.dart';
12
13 class FrostApiService {
14   FrostApiService(this._client);
15
16   final ApiClient _client;
17
18   Future<HealthStatus> getHealth() async {
19     final json = await _client.getJson('/health');
20     return HealthStatus.fromJson(json);
21   }
22
23   Future<ModelInfo> getModelInfo() async {
24     final json = await _client.getJson('/ml/model-info');
25     return ModelInfo.fromJson(json);
26   }
27
28   Future<FrostRiskResponse> predict(FrostRiskRequest request) async {
29     final json = await _client.postJson(
30       '/predict/frost-risk',
31       request.toJson(),
32     );
33     return FrostRiskResponse.fromJson(json);
34   }
35
36   Future<List<PredictionHistoryItem>> getPredictionHistory() async {
37     final rows = await _client.getList('/predictions/history');
38     return rows
39       .whereType<Map<String, dynamic>>()
40       .map(PredictionHistoryItem.fromJson)
41       .toList();
42   }
43
44   Future<CurrentWeather> getCurrentWeather({
45     required double latitude,
46     required double longitude,
47   }) async {
48     final json = await _client.getJson(
49       '/weather/current?latitude=$latitude&longitude=$longitude',
50     );
51     return CurrentWeather.fromJson(json);
52   }
53
54   Future<ClustersResult> getClusters() async {
55     final json = await _client.getJson('/ml/clusters');
56     return ClustersResult.fromJson(json);
57   }
58
59   Future<ChunoWindow> getChunoWindow({
60     required double latitude,
61     required double longitude,
62   }) async {
63     final json = await _client.getJson(
```



```

64    '/chuno/window?latitude=$latitude&longitude=$longitude',
65    );
66    return ChunoWindow.fromJson(json);
67  }
68
69  Future<WeatherHistory> getWeatherHistory({
70    required double latitude,
71    required double longitude,
72    required String start,
73    required String end,
74  }) async {
75    final json = await _client.toJson(
76      '/weather/history?latitude=$latitude&longitude=$longitude&start=$start&end=$end',
77    );
78    return WeatherHistory.fromJson(json);
79  }
80
81  Future<DailyAlert> getDailyAlert({
82    required double latitude,
83    required double longitude,
84  }) async {
85    final json = await _client.toJson(
86      '/alerts/today?latitude=$latitude&longitude=$longitude',
87    );
88    return DailyAlert.fromJson(json);
89  }
90 }

```

app_flutter/lib/core/config/app_config.dart

```

1  class AppConfig {
2    static const apiBaseUrl = String.fromEnvironment(
3      'API_BASE_URL',
4      defaultValue: 'http://127.0.0.1:8000',
5    );
6  }

```

app_flutter/lib/core/i18n/app_language.dart

```

1  /// Idioma de la interfaz. Español por defecto; inglés para la exposición.
2  enum AppLanguage { es, en }
3
4  extension AppLanguageX on AppLanguage {
5    String get label => this == AppLanguage.es ? 'Español' : 'English';
6    String get code => this == AppLanguage.es ? 'es' : 'en';
7    bool get isEn => this == AppLanguage.en;
8  }

```

app_flutter/lib/core/i18n/app_strings.dart

```

1  import '../settings/user_settings.dart';
2  import 'app_language.dart';
3
4  /// Textos de la interfaz en español/inglés. Español por defecto; el modo inglés
5  /// se usa en la exposición (rúbrica: "Funcionamiento de la aplicación en inglés").
6  ///
7  /// Uso: `AppStrings.current.<clave>`. Como `MaterialApp` se reconstruye al cambiar
8  /// `UserSettings.Language`, las pantallas leen siempre el idioma vigente.
9  class AppStrings {
10    const AppStrings(this.language);
11
12    final AppLanguage language;
13
14    static AppStrings get current => AppStrings(UserSettings.instance.language);
15
16    bool get _en => language.isEn;
17    String _t(String es, String en) => _en ? en : es;
18
19    // --- Navegación (shell) ---
20    String get tabHome => _t('Inicio', 'Home');
21    String get tabZones => _t('Zonas', 'Zones');
22    String get tabChuno => _t('Chuño', 'Chuño');
23    String get tabWeather => _t('Clima', 'Weather');
24    String get tabHistory => _t('Historial', 'History');
25
26    // --- Home ---
27    String get systemActive => _t('Sistema activo', 'System active');
28    String get demoMode => _t('Modo demo', 'Demo mode');
29    String get changeTheme => _t('Cambiar tema', 'Change theme');
30    String get settings => _t('Ajustes', 'Settings');
31    String get appTagline => _t(
32      'Predicción inteligente de heladas para comunidades altoandinas.',
33      'Smart frost prediction for high-Andean communities.',
34    );
35    String get checkRisk => _t('Consultar riesgo', 'Check risk');
36    String get viewHistory => _t('Ver historial', 'View history');
37    String get dataSources => _t('Fuentes de datos', 'Data sources');
38    String get modelInfo => _t('Información del modelo', 'Model information');
39    String get currentFrostRisk =>
40      _t('Riesgo actual de helada', 'Current frost risk');
41    String humShort(String v) => _t('Hum $v', 'Hum $v');
42    String get maintenanceTitle =>
43      _t('Mantenimiento del modelo', 'Model maintenance');
44    String get maintenanceBody => _t(
45      'El modelo se reentrena de forma programada, registra métricas y solo se promueve si supera el quality gate de silhouette.',
46      'The model is retrained on a schedule, logs metrics, and is promoted only if it clears the silhouette quality gate.',
47    );
48
49    // --- Tarjetas de alerta (generadas desde campos, localizadas) ---
50    String frostTitle(String severity) => switch (severity) {
51      'fuerte' => _t('Riesgo de helada fuerte', 'Strong frost risk'),
52      'moderada' => _t('Riesgo de helada moderada', 'Moderate frost risk'),
53      _ => _t('Sin alerta de helada', 'No frost alert'),
54    };
55    String frostMessage(String severity) => switch (severity) {
56      'fuerte' => _t(

```

```

57     'Hoy hay riesgo de helada fuerte. Resguarde el ganado y proteja los cultivos.',
58     'Strong frost risk today. Shelter livestock and protect crops.',
59     ),
60     'moderada' => _t(
61         'Hoy hay riesgo de helada moderada. Monitoree la temperatura nocturna.',
62         'Moderate frost risk today. Monitor the overnight temperature.',
63     ),
64     _ => _t(
65         'No se prevé helada significativa para hoy.',
66         'No significant frost expected today.',
67     ),
68     );
69     String livestockTitle(String level) =>
70         _t('Ganado · riesgo $level', '{$_livestockLevelEn(level)} livestock risk');
71     String _livestockLevelEn(String level) => switch (level) {
72         'alto' => 'High',
73         'medio' => 'Medium',
74         _ => 'Low',
75     };
76     String livestockMessage(String level) => switch (level) {
77         'alto' => _t(
78             'Riesgo alto para el ganado: proteja crías de alpaca y ovino esta noche.',
79             'High livestock risk: protect alpaca and sheep newborns tonight.',
80         ),
81         'medio' => _t(
82             'Riesgo medio: abrigue a las crías y revise cobertizos antes del anochecer.',
83             'Medium risk: shelter the young and check pens before nightfall.',
84         ),
85         _ => _t(
86             'Sin riesgo relevante para el ganado esta noche.',
87             'No relevant livestock risk tonight.',
88         ),
89     );
90     String get anomalyTitle => _t('Riesgo inusual', 'Unusual risk');
91     String anomalyMessage(double? delta) {
92         if (delta == null) {
93             return _t('Temperatura dentro de lo normal.', 'Temperature within normal range.');
```

```

178     'Fuente territorial, censo y agropecuaria para ubigeos, distritos, ruralidad y contexto local.',
179     'Territorial, census and agricultural source for districts, rurality and local context.',
180 );
181 String get sourceSenamhi => _t(
182     'Fuente oficial peruana prioritaria para estaciones, avisos, validacion y futura calibracion climatica.',
183     'Prioritized official Peruvian source for stations, advisories, validation and future calibration.',
184 );
185 String get sourceMidagri => _t(
186     'Fuente agricola complementaria para produccion, cultivos relevantes e impacto productivo.',
187     'Complementary agricultural source for production, relevant crops and productive impact.',
188 );
189 String get sourcesFooter => _t(
190     'SENAHI se prioriza como fuente oficial; si no hay datos operativos disponibles, FrostPuno usa Open-Meteo como fallback.',
191     'SENAHI is prioritized as the official source; when no operational data is available, FrostPuno falls back to Open-Meteo.',
192 );
193 String get improvementCycle =>
194     _t('Ciclo de mejora del modelo', 'Model improvement cycle');
195 List<(String, String, String)> get pipelineSteps => _en
196     ? const [
197         ('1', 'Current weather', 'Open-Meteo feeds the app in real time.'),
198         ('2', 'SENAHI', 'Official observations validate real events.'),
199         ('3', 'Feedback', 'Supabase stores corrections and field evidence.'),
200         ('4', 'New model', 'ML compares versions before promoting.'),
201     ]
202     : const [
203         ('1', 'Clima actual', 'Open-Meteo alimenta la app en tiempo real.'),
204         ('2', 'SENAHI', 'Observaciones oficiales validan eventos reales.'),
205         ('3', 'Feedback', 'Supabase guarda correcciones y evidencia de campo.'),
206         ('4', 'Nuevo modelo', 'ML compara versiones antes de promover.'),
207     ];
208 String get pipelineFooter => _t(
209     'Este flujo evita entrenar automaticamente con datos dudosos y mantiene trazabilidad academica.',
210     'This flow avoids auto-training on doubtful data and keeps academic traceability.',
211 );
212
213 // --- Ajustes ---
214 String get settingsSubtitle => _t(
215     'Configura tus alarmas de helada y el perfil con el que usas la app.',
216     'Set your frost alarms and the profile you use the app with.',
217 );
218 String get enableAlerts =>
219     _t('Activar alertas de helada', 'Enable frost alerts');
220 String get enableAlertsSub => _t(
221     'Muestra el aviso en Inicio y envia notificación en el celular.',
222     'Shows the banner on Home and sends a phone notification.',
223 );
224 String alertThresholdLabel(String c) => _t(
225     'Avisarme si la mínima baja de $c °C',
226     'Alert me if the low drops below $c °C',
227 );
228 String get profile => _t('Perfil', 'Profile');
229 String get profileHint =>
230     _t('Prioriza qué alertas ves en Inicio.', 'Prioritizes which alerts you see on Home');
231 String get languageLabel => _t('Idioma', 'Language');
232 String profileName(String key) => switch (key) {
233     'general' => _t('General', 'General'),
234     'agricultor' => _t('Agricultor', 'Farmer'),
235     'ganadero' => _t('Ganadero', 'Herder'),
236     _ => _t('Chuñero', 'Chuño maker'),
237 };
238
239 // --- Model info ---
240 String get modelInfoTitle => _t('Informacion del modelo', 'Model information');
241 String get modelInfoSubtitle => _t(
242     'Especificaciones tecnicas y metricas del modelo no supervisado (clustering) en produccion.',
243     'Technical specs and metrics of the unsupervised (clustering) model in production.',
244 );
245 String get baseArchitecture => _t('ARQUITECTURA BASE', 'BASE ARCHITECTURE');
246 String get chipClustering => _t('Clustering', 'Clustering');
247 String get chipUnsupervised => _t('No supervisado', 'Unsupervised');
248 String get mainMetric => _t('METRICA PRINCIPAL', 'MAIN METRIC');
249 String get silhouetteDesc => _t(
250     'Cohesion y separacion de los grupos climaticos (0 a 1, mas alto es mejor).',
251     'Cohesion and separation of the climate groups (0 to 1, higher is better).',
252 );
253 String get riskGroups => _t('GRUPOS DE RIESGO', 'RISK GROUPS');
254 String get clustersUnit => _t('clusters', 'clusters');
255 String get clustersDesc => _t(
256     'Regimenes climaticos agrupados por K-Means y mapeados a alto/medio/bajo.',
257     'Climate regimes grouped by K-Means and mapped to high/medium/low.',
258 );
259 String get datasetSize => _t('DATASET SIZE', 'DATASET SIZE');
260 String get samples => _t('muestras', 'samples');
261 String get datasetDesc => _t(
262     'Registros generados desde Open-Meteo y contexto territorial inicial.',
263     'Records generated from Open-Meteo and initial territorial context.',
264 );
265 String get versionStatus => _t('VERSION Y ESTADO', 'VERSION & STATUS');
266 String get qualityGateApproved =>
267     _t('Quality Gate: Aprobado', 'Quality Gate: Passed');
268 String get modelInfoError => _t(
269     'No se pudo cargar informacion del modelo.',
270     'Could not load model information.',
271 );
272
273 // --- Predicción (form) ---
274 String get frostRiskTitle => _t('Riesgo de helada', 'Frost risk');
275 String get frostRiskSubtitle => _t(
276     'Ubicacion y clima se obtienen automaticamente para enviar una prediccion lista para campo.',
277     'Location and weather are fetched automatically to send a field-ready prediction.',
278 );
279 String get location => _t('Ubicacion', 'Location');
280 String get detectedLocation => _t('Ubicacion detectada', 'Detected location');
281 String get punoDemo => _t('Puno demo', 'Puno demo');
282 String get locationHint => _t(
283     'Usa GPS para detectar tu ubicacion. Si falla, FrostPuno conserva Puno como fallback manual.',
284     'Use GPS to detect your location. If it fails, FrostPuno keeps Puno as a manual fallback.',
285 );
286 String get useMyLocation => _t('Usar mi ubicacion', 'Use my location');
287 String get detectingLocation =>
288     _t('Detectando ubicacion...', 'Detecting location...');
289 String get currentWeather => _t('Clima actual', 'Current weather');
290 String get humidity => _t('Humedad', 'Humidity');
291 String get wind => _t('Viento', 'Wind');
292 String get clouds => _t('Nubes', 'Clouds');
293 String get weatherHint => _t(
294     'El clima se llenara automaticamente desde FastAPI cuando obtengas tu ubicacion o pulses Obtener clima.',
295     'Weather auto-fills from FastAPI once you get your location or tap Get weather.',
296 );
297 String get getWeather => _t('Obtener clima', 'Get weather');
298 String get gettingWeather => _t('Obteniendo clima...', 'Getting weather...');

```

```

299 String get readyToPredict => _t('Listo para predecir', 'Ready to predict');
300 String get readyToPredictBody => _t(
301   'FrostPuno enviara ubicacion, clima actual y contexto agricola interno al modelo.',
302   'FrostPuno will send location, current weather and internal agricultural context to the model.',
303 );
304 String get predict => _t('Predecir', 'Predict');
305 String get predicting => _t('Prediciendo...', 'Predicting...');
306 String get internalDemoValue => _t('Valor demo interno', 'Internal demo value');
307 String feelsLike(String c) => _t('Sensacion $c C', 'Feels like $c C');
308 String connectError(Object e) =>
309   _t('No se pudo conectar con FastAPI: $e', 'Could not reach FastAPI: $e');
310 String get requestingGps =>
311   _t('Solicitando permiso y leyendo GPS...', 'Requesting permission and reading GPS...');
312 String get waitingLocation => _t(
313   'Esperando ubicacion para consultar clima actual.',
314   'Waiting for location to fetch current weather.',
315 );
316 String gpsAccuracy(bool lastKnown, String meters) => _t(
317   '${lastKnown ? 'Ultima ubicacion conocida' : 'GPS detectado'} con precision aprox. $meters m.',
318   '${lastKnown ? 'Last known location' : 'GPS detected'} with approx. $meters m accuracy.',
319 );
320 String locationFailed(String detail) => _t(
321   '$detail Fallback manual activo: Puno demo.',
322   '$detail Manual fallback active: Puno demo.',
323 );
324 String get locationFailedGeneric => _t(
325   'No se pudo obtener ubicacion. Fallback manual activo: Puno demo.',
326   'Could not get location. Manual fallback active: Puno demo.',
327 );
328 String get tapGetWeatherDemo => _t(
329   'Puedes pulsar Obtener clima para usar el punto demo de Puno.',
330   'Tap Get weather to use the Puno demo point.',
331 );
332 String get queryingWeather =>
333   _t('Consultando GET /weather/current...', 'Calling GET /weather/current...');
334 String weatherUpdated(String provider, bool fallback) => _t(
335   'Clima actualizado via $provider${fallback ? ' con fallback' : ''}.',
336   'Weather updated via $provider${fallback ? ' with fallback' : ''}.',
337 );
338 String weatherFailed(Object e) => _t(
339   'No se pudo obtener clima actual. Se conservan valores demo internos. Detalle: $e',
340   'Could not get current weather. Internal demo values kept. Detail: $e',
341 );
342 String detail(Object e) => _t('Detalle: $e', 'Detail: $e');
343
344 // --- Resultado ---
345 String get resultTitle => _t('Resultado', 'Result');
346 String get resultSubtitle => _t(
347   'Lectura clara para decidir acciones en campo.',
348   'Clear read to decide field actions.',
349 );
350 String get riskLabel => _t('RIESGO', 'RISK');
351 String get recommendationFallback => _t(
352   'Revisar condiciones locales y monitorear cambios de temperatura.',
353   'Review local conditions and monitor temperature changes.',
354 );
355 String get chunoConditionLabel =>
356   _t('CONDICION PARA CHUNO', 'CHUÑO CONDITION');
357
358 /// La API responde en español; en modo inglés se rotula por nivel de riesgo.
359 String recommendation(String riskLevel, String backendText) {
360   if (!_en) {
361     return backendText.trim().isEmpty ? recommendationFallback : backendText;
362   }
363   return switch (riskLevel.toLowerCase()) {
364     'alto' =>
365       'High frost risk: shelter livestock and cover crops before nightfall.',
366     'medio' || 'moderado' =>
367       'Moderate risk: monitor the overnight temperature and prepare protection.',
368     _ => 'Low risk: keep routine monitoring for sudden changes.',
369   };
370 }
371 String get summary => _t('RESUMEN', 'SUMMARY');
372 String get district => _t('Distrito', 'District');
373 String get newQuery => _t('Nueva consulta', 'New query');
374 String chunoCondition(String value) => switch (value) {
375   'favorables' => _t('Favorable', 'Favorable'),
376   'posibles' => _t('Possible', 'Possible'),
377   _ => _t('No favorable', 'Unfavorable'),
378 };
379
380 // --- Palabras de riesgo (para chips en mayúscula) ---
381 String riskWord(String level) => switch (level.toLowerCase()) {
382   'alto' => _t('Alto', 'High'),
383   'medio' || 'moderado' => _t('Medio', 'Medium'),
384   _ => _t('Bajo', 'Low'),
385 };
386 String tierWord(String tier) => switch (tier.toLowerCase()) {
387   'alto' => _t('ALTO', 'HIGH'),
388   'medio' => _t('MEDIO', 'MEDIUM'),
389   'excelente' => _t('EXCELENTE', 'EXCELLENT'),
390   'bueno' => _t('BUENO', 'GOOD'),
391   'marginal' => _t('MARGINAL', 'MARGINAL'),
392   _ => _t('BAJO', 'LOW'),
393 };
394
395 static String _cap(String v) =>
396   v.isEmpty ? v : '${v[0].toUpperCase()}${v.substring(1)}';
397 }

```

app_flutter/lib/core/models/current_weather.dart

```

1 | class CurrentWeather {
2 |   const CurrentWeather({
3 |     required this.latitude,
4 |     required this.longitude,
5 |     required this.provider,
6 |     required this.fallbackUsed,
7 |     this.temperature,
8 |     this.apparentTemperature,
9 |     this.humidity,
10 |    this.windSpeed,
11 |    this.cloudCover,
12 |    this.dewPoint,
13 |    this.precipitation,
14 |    this.observedAt,

```

```

15     this.stationName,
16     this.stationDistanceKm,
17   ));
18
19   final double latitude;
20   final double longitude;
21   final String provider;
22   final bool fallbackUsed;
23   final double? temperature;
24   final double? apparentTemperature;
25   final double? humidity;
26   final double? windSpeed;
27   final double? cloudCover;
28   final double? dewPoint;
29   final double? precipitation;
30   final String? observedAt;
31   final String? stationName;
32   final double? stationDistanceKm;
33
34   factory CurrentWeather.fromJson(Map<String, dynamic> json) {
35     double? n(String key) {
36       final value = json[key];
37       return value is num ? value.toDouble() : null;
38     }
39
40     return CurrentWeather(
41       latitude: n('latitude') ?? 0,
42       longitude: n('longitude') ?? 0,
43       provider: json['provider'] as String? ?? 'desconocido',
44       fallbackUsed: json['fallback_used'] as bool? ?? false,
45       temperature: n('temperature'),
46       apparentTemperature: n('apparent_temperature'),
47       humidity: n('humidity'),
48       windSpeed: n('wind_speed'),
49       cloudCover: n('cloud_cover'),
50       dewPoint: n('dew_point'),
51       precipitation: n('precipitation'),
52       observedAt: json['observed_at'] as String?,
53       stationName: json['station_name'] as String?,
54       stationDistanceKm: n('station_distance_km'),
55     );
56   }
57 }

```

app_flutter/lib/core/models/health_status.dart

```

1|
2| class HealthStatus {
3|   const HealthStatus({
4|     required this.status,
5|     required this.appName,
6|     required this.version,
7|     required this.modelAvailable,
8|   });
9|
10|   final String status;
11|   final String appName;
12|   final String version;
13|   final bool modelAvailable;
14|
15|   factory HealthStatus.fromJson(Map<String, dynamic> json) {
16|     return HealthStatus(
17|       status: json['status'] as String? ?? 'unknown',
18|       appName: json['app_name'] as String? ?? 'FrostPuno API',
19|       version: json['version'] as String? ?? '-',
20|       modelAvailable: json['model_available'] as bool? ?? false,
21|     );
22|   }
23| }

```

app_flutter/lib/core/notifications/notification_service.dart

```

1| import 'package:flutter/foundation.dart' show kIsWeb;
2| import 'package:flutter_local_notifications/flutter_local_notifications.dart';
3|
4| /// Notificaciones Locales nativas (Android/iOS). En web es no-op:
5| /// el banner in-app cubre ese caso. Todo va detrás de 'kIsWeb'.
6| class NotificationService {
7|   static final FlutterLocalNotificationsPlugin _plugin =
8|     FlutterLocalNotificationsPlugin();
9|   static bool _ready = false;
10|
11|   static Future<void> init() async {
12|     if (kIsWeb) return;
13|     const android = AndroidInitializationSettings('@mipmap/ic_launcher');
14|     const settings = InitializationSettings(android: android);
15|     try {
16|       await _plugin.initialize(settings);
17|       final androidImpl = _plugin
18|         .resolvePlatformSpecificImplementation<
19|           AndroidFlutterLocalNotificationsPlugin
20|         >();
21|       await androidImpl?.requestNotificationsPermission();
22|       _ready = true;
23|     } catch (_) {
24|       _ready = false;
25|     }
26|   }
27|
28|   static Future<void> showFrostAlert(String title, String body) async {
29|     if (kIsWeb || !_ready) return;
30|     const details = NotificationDetails(
31|       android: AndroidNotificationDetails(
32|         'frost_alerts',
33|         'Alertas de helada',
34|         channelDescription: 'Avisos de riesgo de helada de FrostPuno',
35|         importance: Importance.high,
36|         priority: Priority.high,
37|         icon: '@mipmap/ic_launcher',
38|       ),
39|     );

```

```

40|     try {
41|       await _plugin.show(0, title, body, details);
42|     } catch (_) {
43|       // silencioso: la notificación no debe romper la app.
44|     }
45|   }
46| }

```

app_flutter/lib/core/settings/user_settings.dart

```

1| import 'package:flutter/foundation.dart';
2| import 'package:shared_preferences/shared_preferences.dart';
3|
4| import '../i18n/app_language.dart';
5|
6| /// Perfil del usuario: filtra qué alertas se priorizan en Inicio.
7| enum UserProfile { general, agricultor, ganadero, chunero }
8|
9| extension UserProfileLabel on UserProfile {
10|   String get label => switch (this) {
11|     UserProfile.general => 'General',
12|     UserProfile.agricultor => 'Agricultor',
13|     UserProfile.ganadero => 'Ganadero',
14|     UserProfile.chunero => 'Chuñero',
15|   };
16| }
17|
18| /// Preferencias del usuario persistidas con shared_preferences.
19| /// Singleton observable (ChangeNotifier) para reaccionar en la UI.
20| class UserSettings extends ChangeNotifier {
21|   UserSettings._();
22|   static final UserSettings instance = UserSettings._();
23|
24|   static const _kAlertsEnabled = 'alerts_enabled';
25|   static const _kThreshold = 'alert_threshold';
26|   static const _kProfile = 'user_profile';
27|   static const _kLanguage = 'app_language';
28|
29|   bool alertsEnabled = true;
30|   double alertThreshold = -4;
31|   UserProfile profile = UserProfile.general;
32|   AppLanguage language = AppLanguage.es;
33|
34|   Future<void> load() async {
35|     final prefs = await SharedPreferences.getInstance();
36|     alertsEnabled = prefs.getBool(_kAlertsEnabled) ?? true;
37|     alertThreshold = prefs.getDouble(_kThreshold) ?? -4;
38|     final profileIndex = prefs.getInt(_kProfile) ?? 0;
39|     profile = UserProfile.values[profileIndex.clamp(0, UserProfile.values.length - 1)];
40|     final langIndex = prefs.getInt(_kLanguage) ?? 0;
41|     language = AppLanguage.values[langIndex.clamp(0, AppLanguage.values.length - 1)];
42|     notifyListeners();
43|   }
44|
45|   Future<void> setLanguage(AppLanguage value) async {
46|     language = value;
47|     notifyListeners();
48|     final prefs = await SharedPreferences.getInstance();
49|     await prefs.setInt(_kLanguage, value.index);
50|   }
51|
52|   Future<void> setAlertsEnabled(bool value) async {
53|     alertsEnabled = value;
54|     notifyListeners();
55|     final prefs = await SharedPreferences.getInstance();
56|     await prefs.setBool(_kAlertsEnabled, value);
57|   }
58|
59|   Future<void> setThreshold(double value) async {
60|     alertThreshold = value;
61|     notifyListeners();
62|     final prefs = await SharedPreferences.getInstance();
63|     await prefs.setDouble(_kThreshold, value);
64|   }
65|
66|   Future<void> setProfile(UserProfile value) async {
67|     profile = value;
68|     notifyListeners();
69|     final prefs = await SharedPreferences.getInstance();
70|     await prefs.setInt(_kProfile, value.index);
71|   }
72| }

```

app_flutter/lib/core/theme/app_colors.dart

```

1| import 'package:flutter/material.dart';
2|
3| class AppColors {
4|   static const deepNavy = Color(0xFF0D1B2A);
5|   static const iceBlue = Color(0xFFE0B8FC);
6|   static const mutedTeal = Color(0xFF3D5A80);
7|   static const softWhite = Color(0xFFFFF5F5);
8|   static const warmAmber = Color(0xFFEE6C4D);
9|   static const surface = Color(0xFF9F9F9F);
10|   static const surfaceLow = Color(0xFF3F3F3F);
11|   static const surfaceHigh = Color(0xFFE8E8E8);
12|   static const textPrimary = Color(0xFF1A1A1A);
13|   static const textSecondary = Color(0xFF444444);
14|   static const outline = Color(0xFF3D5A80);
15|   static const success = Color(0xFF0F8A5F);
16|   static const deepTeal = Color(0xFF1F6F78);
17|   static const mediumRisk = Color(0xFFE0A12B);
18|   static const lowRisk = Color(0xFF0F8A5F);
19|
20|   static const darkSurface = Color(0xFF181820);
21|   static const darkSurfaceLow = Color(0xFF16232E);
22|   static const darkSurfaceHigh = Color(0xFF1E2D38);
23|   static const darkTextPrimary = Color(0xFFFF2F7F);
24|   static const darkTextSecondary = Color(0xFFFCDD02);
25|   static const darkOutline = Color(0x333CB6C4);
26| }

```

app_flutter/lib/core/theme/app_theme.dart

```

1| import 'package:flutter/material.dart';
2|
3| import 'app_colors.dart';
4|
5| class AppTheme {
6|   static ThemeData light() {
7|     return _build(Brightness.light);
8|   }
9|
10|   static ThemeData dark() {
11|     return _build(Brightness.dark);
12|   }
13|
14|   static ThemeData _build(Brightness brightness) {
15|     final isDark = brightness == Brightness.dark;
16|     final colorScheme = ColorScheme.fromSeed(
17|       seedColor: AppColors.mutedTeal,
18|       brightness: brightness,
19|       primary: isDark ? AppColors.iceBlue : AppColors.deepNavy,
20|       secondary: AppColors.mutedTeal,
21|       tertiary: isDark ? AppColors.deepTeal : AppColors.iceBlue,
22|       error: AppColors.warmAmber,
23|       surface: isDark ? AppColors.darkSurface : AppColors.surface,
24|     );
25|     final surface = isDark ? AppColors.darkSurface : AppColors.surface;
26|     final textPrimary = isDark
27|       ? AppColors.darkTextPrimary
28|       : AppColors.textPrimary;
29|     final textSecondary = isDark
30|       ? AppColors.darkTextSecondary
31|       : AppColors.textSecondary;
32|
33|     return ThemeData(
34|       useMaterial3: true,
35|       colorScheme: colorScheme,
36|       scaffoldBackgroundColor: surface,
37|       fontFamily: 'Inter',
38|       appBarTheme: AppBarTheme(
39|         centerTitle: true,
40|         elevation: 0,
41|         scrolledUnderElevation: 0,
42|         backgroundColor: surface,
43|         foregroundColor: textPrimary,
44|         titleTextStyle: TextStyle(
45|           color: textPrimary,
46|           fontSize: 26,
47|           fontWeight: FontWeight.w700,
48|         ),
49|       ),
50|       textTheme: const TextTheme(
51|         displayLarge: TextStyle(
52|           fontSize: 32,
53|           height: 1.2,
54|           fontWeight: FontWeight.w700,
55|         ),
56|         headlineMedium: TextStyle(
57|           fontSize: 24,
58|           height: 1.25,
59|           fontWeight: FontWeight.w700,
60|         ),
61|         titleMedium: TextStyle(
62|           fontSize: 18,
63|           height: 1.3,
64|           fontWeight: FontWeight.w700,
65|         ),
66|         bodyLarge: TextStyle(
67|           fontSize: 16,
68|           height: 1.5,
69|           fontWeight: FontWeight.w400,
70|         ),
71|         bodyMedium: TextStyle(
72|           fontSize: 14,
73|           height: 1.45,
74|           fontWeight: FontWeight.w400,
75|         ),
76|         labelSmall: TextStyle(
77|           fontSize: 12,
78|           height: 1.35,
79|           fontWeight: FontWeight.w500,
80|         ),
81|       ).apply(bodyColor: textPrimary, displayColor: textPrimary),
82|       inputDecorationTheme: InputDecorationTheme(
83|         filled: true,
84|         fillColor: isDark ? AppColors.darkSurfaceHigh : AppColors.softWhite,
85|         border: const UnderlineInputBorder(
86|           borderSide: BorderSide(color: AppColors.mutedTeal),
87|         ),
88|         enabledBorder: const UnderlineInputBorder(
89|           borderSide: BorderSide(color: AppColors.mutedTeal),
90|         ),
91|         focusedBorder: UnderlineInputBorder(
92|           borderSide: BorderSide(
93|             color: isDark ? AppColors.iceBlue : AppColors.deepNavy,
94|             width: 1.4,
95|           ),
96|         ),
97|         contentPadding: const EdgeInsets.symmetric(horizontal: 4, vertical: 10),
98|       ),
99|       filledButtonTheme: FilledButtonThemeData(
100|         style: FilledButton.styleFrom(
101|           backgroundColor: isDark ? AppColors.iceBlue : AppColors.deepNavy,
102|           foregroundColor: isDark ? AppColors.deepNavy : AppColors.softWhite,
103|           disabledBackgroundColor:
104|             (isDark ? AppColors.darkSurfaceHigh : AppColors.surfaceHigh)
105|               .withValues(alpha: 0.8),
106|           disabledForegroundColor: textSecondary,
107|         ),
108|       ),
109|     );
110|   }
111| }

```

app_flutter/lib/core/theme/theme_mode_scope.dart

```

1| import 'package:flutter/material.dart';
2|
3| class ThemeModeScope extends InheritedWidget {
4|   const ThemeModeScope({
5|     required this.themeMode,
6|     required this.onThemeModeChanged,
7|     required super.child,
8|     super.key,
9|   });
10|
11|   final ThemeMode themeMode;
12|   final ValueChanged<ThemeMode> onThemeModeChanged;
13|
14|   static ThemeModeScope? maybeOf(BuildContext context) {
15|     return context.dependOnInheritedWidgetOfExactType<ThemeModeScope>();
16|   }
17|
18|   static ThemeModeScope of(BuildContext context) {
19|     final scope = maybeOf(context);
20|     assert(scope != null, 'ThemeModeScope not found in context');
21|     return scope!;
22|   }
23|
24|   @override
25|   bool updateShouldNotify(ThemeModeScope oldWidget) {
26|     return themeMode != oldWidget.themeMode ||
27|       onThemeModeChanged != oldWidget.onThemeModeChanged;
28|   }
29| }

```

app_flutter/lib/features/alerts/models/daily_alert.dart

```

1| class LivestockRisk {
2|   LivestockRisk({required this.level, required this.message});
3|
4|   final String level;
5|   final String message;
6|
7|   factory LivestockRisk.fromJson(Map<String, dynamic> json) => LivestockRisk(
8|     level: json['level'] as String? ?? 'bajo',
9|     message: json['message'] as String? ?? '',
10|   );
11| }
12|
13| class ClimateAnomaly {
14|   ClimateAnomaly({
15|     required this.isUnusual,
16|     required this.message,
17|     this.historicalMean,
18|     this.delta,
19|   });
20|
21|   final bool isUnusual;
22|   final String message;
23|   final double? historicalMean;
24|   final double? delta;
25|
26|   factory ClimateAnomaly.fromJson(Map<String, dynamic> json) => ClimateAnomaly(
27|     isUnusual: json['is_unusual'] as bool? ?? false,
28|     message: json['message'] as String? ?? '',
29|     historicalMean: (json['historical_mean'] as num?)?.toDouble(),
30|     delta: (json['delta'] as num?)?.toDouble(),
31|   );
32| }
33|
34| class DailyAlert {
35|   DailyAlert({
36|     required this.riskLevel,
37|     required this.frostAlert,
38|     required this.severity,
39|     required this.title,
40|     required this.message,
41|     required this.livestock,
42|     required this.anomaly,
43|     this.temperatureMin,
44|   });
45|
46|   final String riskLevel;
47|   final bool frostAlert;
48|   final String severity; // fuerte | moderada | ninguna
49|   final String title;
50|   final String message;
51|   final double? temperatureMin;
52|   final LivestockRisk livestock;
53|   final ClimateAnomaly anomaly;
54|
55|   factory DailyAlert.fromJson(Map<String, dynamic> json) => DailyAlert(
56|     riskLevel: json['risk_level'] as String,
57|     frostAlert: json['frost_alert'] as bool,
58|     severity: json['severity'] as String,
59|     title: json['title'] as String,
60|     message: json['message'] as String,
61|     temperatureMin: (json['temperature_min'] as num?)?.toDouble(),
62|     livestock: LivestockRisk.fromJson(
63|       (json['livestock'] as Map<String, dynamic>?) ?? const {},
64|     ),
65|     anomaly: ClimateAnomaly.fromJson(
66|       (json['anomaly'] as Map<String, dynamic>?) ?? const {},
67|     ),
68|   );
69| }

```

app_flutter/lib/features/charts/models/history_day.dart

```

1| class HistoryDay {
2|   HistoryDay({
3|     required this.date,
4|     this.temperatureMin,
5|     this.temperatureMax,
6|     this.humidityMean,
7|   });

```



```

8
9   final String date;
10  final double? temperatureMin;
11  final double? temperatureMax;
12  final double? humidityMean;
13
14  factory HistoryDay.fromJson(Map<String, dynamic> json) => HistoryDay(
15    date: json['date'] as String,
16    temperatureMin: (json['temperature_min'] as num?)?.toDouble(),
17    temperatureMax: (json['temperature_max'] as num?)?.toDouble(),
18    humidityMean: (json['humidity_mean'] as num?)?.toDouble(),
19  );
20 }
21
22 class WeatherHistory {
23   WeatherHistory({required this.days});
24
25   final List<HistoryDay> days;
26
27   factory WeatherHistory.fromJson(Map<String, dynamic> json) => WeatherHistory(
28     days: (json['days'] as List)
29       .whereType<Map<String, dynamic>>()
30       .map<HistoryDay, dynamic>(HistoryDay.fromJson)
31       .toList(),
32   );
33 }

```

app_flutter/lib/features/charts/screens/charts_screen.dart

```

1 | import 'package:fl_chart/fl_chart.dart';
2 | import 'package:flutter/material.dart';
3 |
4 | import '../../../core/api/frost_api_service.dart';
5 | import '../../../core/i18n/app_strings.dart';
6 | import '../../../core/theme/app_colors.dart';
7 | import '../../../shared/widgets/glass_card.dart';
8 | import '../../../shared/widgets/responsive_content.dart';
9 | import '../../../shared/widgets/section_header.dart';
10 | import '../models/history_day.dart';
11 |
12 | const _defaultLat = -15.8402;
13 | const _defaultLon = -70.0219;
14 |
15 | String _fmt(DateTime d) =>
16 |   '${d.year.toString().padLeft(4, '0')}-${d.month.toString().padLeft(2, '0')}-${d.day.toString().padLeft(2, '0')}';
17 |
18 | class ChartsScreen extends StatefulWidget {
19 |   const ChartsScreen({required this.apiService, super.key});
20 |
21 |   final FrostApiService apiService;
22 |
23 |   @override
24 |   State<ChartsScreen> createState() => _ChartsScreenState();
25 | }
26 |
27 | class _ChartsScreenState extends State<ChartsScreen> {
28 |   late Future<WeatherHistory> _future = _load();
29 |
30 |   Future<WeatherHistory> _load() {
31 |     // Open-Meteo Archive tiene ~5 días de retraso; pedimos 30 días hasta hace 6.
32 |     final end = DateTime.now().subtract(const Duration(days: 6));
33 |     final start = end.subtract(const Duration(days: 29));
34 |     return widget.apiService.getWeatherHistory(
35 |       latitude: _defaultLat,
36 |       longitude: _defaultLon,
37 |       start: _fmt(start),
38 |       end: _fmt(end),
39 |     );
40 |   }
41 |
42 |   @override
43 |   Widget build(BuildContext context) {
44 |     return SafeArea(
45 |       child: ResponsiveContent(
46 |         child: FutureBuilder<WeatherHistory>(
47 |           future: _future,
48 |           builder: (context, snapshot) {
49 |             if (snapshot.connectionState == ConnectionState.waiting) {
50 |               return const Center(child: CircularProgressIndicator());
51 |             }
52 |             if (snapshot.hasError) {
53 |               return _Error(message: snapshot.error.toString());
54 |             }
55 |             final days = snapshot.data!.days;
56 |             final s = AppStrings.current;
57 |             return RefreshIndicator(
58 |               onRefresh: () async {
59 |                 setState(() => _future = _load());
60 |                 await _future;
61 |               },
62 |               child: ListView(
63 |                 padding: const EdgeInsets.fromLTRB(20, 20, 20, 24),
64 |                 children: [
65 |                   SectionHeader(
66 |                     title: s.weatherTitle,
67 |                     subtitle: s.weatherSubtitle,
68 |                   ),
69 |                   const SizedBox(height: 20),
70 |                   GlassCard(
71 |                     child: Column(
72 |                       crossAxisAlignment: CrossAxisAlignment.start,
73 |                       children: [
74 |                         Text(
75 |                           s.temperatureC,
76 |                           style: Theme.of(context).textTheme.titleMedium,
77 |                         ),
78 |                         const SizedBox(height: 8),
79 |                         _Legend(
80 |                           items: [
81 |                             (s.minLabel, AppColors.mutedTeal),
82 |                             (s.maxLabel, AppColors.warmAmber),
83 |                           ],
84 |                         ),
85 |                         const SizedBox(height: 16),
86 |                         SizedBox(
87 |                           height: 220,

```

```

88         child: LineChart(_tempChart(context, days)),
89       ),
90     ],
91   ),
92 ),
93   const SizedBox(height: 20),
94   GlassCard(
95     child: Column(
96       crossAxisAlignment: CrossAxisAlignment.start,
97       children: [
98         Text(
99           s.relativeHumidity,
100         style: Theme.of(context).textTheme.titleMedium,
101       ),
102       const SizedBox(height: 16),
103       SizedBox(
104         height: 200,
105         child: LineChart(_humidityChart(context, days)),
106       ),
107     ],
108   ),
109 ),
110 ],
111 ),
112 );
113 },
114 ),
115 ),
116 );
117 }
118
119 List<FlSpot> _spots(List<HistoryDay> days, double? Function(HistoryDay) pick) {
120   final spots = <FlSpot>[];
121   for (var i = 0; i < days.length; i++) {
122     final value = pick(days[i]);
123     if (value != null) spots.add(FlSpot(i.toDouble(), value));
124   }
125   return spots;
126 }
127
128 LineChartData _tempChart(BuildContext context, List<HistoryDay> days) {
129   return LineChartData(
130     gridData: const FlGridData(show: true, drawVerticalLine: false),
131     titlesData: _titles(context, days),
132     borderData: FlBorderData(show: false),
133     lineBarsData: [
134       _line(_spots(days, (d) => d.temperatureMin), AppColors.mutedTeal),
135       _line(_spots(days, (d) => d.temperatureMax), AppColors.warmAmber),
136     ],
137   );
138 }
139
140 LineChartData _humidityChart(BuildContext context, List<HistoryDay> days) {
141   return LineChartData(
142     gridData: const FlGridData(show: true, drawVerticalLine: false),
143     titlesData: _titles(context, days),
144     borderData: FlBorderData(show: false),
145     lineBarsData: [
146       _line(_spots(days, (d) => d.humidityMean), AppColors.deepTeal),
147     ],
148   );
149 }
150
151 LineChartBarData _line(List<FlSpot> spots, Color color) => LineChartBarData(
152   spots: spots,
153   isCurved: true,
154   color: color,
155   barWidth: 2.5,
156   dotData: const FlDotData(show: false),
157 );
158
159 FlTitlesData _titles(BuildContext context, List<HistoryDay> days) {
160   final axisColor = Theme.of(
161     context,
162   ).colorScheme.onSurface.withValues(alpha: 0.62);
163   return FlTitlesData(
164     show: true,
165     topTitles: const AxisTitles(sideTitles: SideTitles(showTitles: false)),
166     rightTitles: const AxisTitles(sideTitles: SideTitles(showTitles: false)),
167     leftTitles: AxisTitles(
168       sideTitles: SideTitles(
169         showTitles: true,
170         reservedSize: 36,
171         getTitlesWidget: (value, meta) => Text(
172           value.round().toString(),
173           style: TextStyle(fontSize: 11, color: axisColor),
174         ),
175       ),
176     ),
177     bottomTitles: AxisTitles(
178       sideTitles: SideTitles(
179         showTitles: true,
180         interval: (days.length / 4).clamp(1, 30).toDouble(),
181         getTitlesWidget: (value, meta) {
182           final index = value.toInt();
183           if (index < 0 || index >= days.length) {
184             return const SizedBox.shrink();
185           }
186           final label = days[index].date;
187           return Padding(
188             padding: const EdgeInsets.only(top: 6),
189             child: Text(
190               label.length >= 10 ? label.substring(5) : label,
191               style: TextStyle(fontSize: 10, color: axisColor),
192             ),
193           );
194         },
195       ),
196     ),
197   );
198 }
199
200 class _Legend extends StatelessWidget {
201   const _Legend({required this.items});
202
203   final List<(String, Color)> items;
204
205   @override
206   Widget build(BuildContext context) {
207     return Wrap(
208

```

```

209     spacing: 16,
210     children: items
211     .map(
212       (item) => Row(
213         mainAxisAlignment: MainAxisAlignment.min,
214         children: [
215           Container(width: 12, height: 12, color: item.$2),
216           const SizedBox(width: 6),
217           Text(item.$1, style: Theme.of(context).textTheme.bodySmall),
218         ],
219       ),
220     )
221     .toList(),
222   );
223 }
224 }
225
226 class _Error extends StatelessWidget {
227   const _Error({required this.message});
228
229   final String message;
230
231   @override
232   Widget build(BuildContext context) {
233     return ListView(
234       padding: const EdgeInsets.all(24),
235       children: [
236         const SizedBox(height: 80),
237         const Icon(Icons.show_chart, size: 48, color: AppColors.warmAmber),
238         const SizedBox(height: 16),
239         Text(
240           AppStrings.current.weatherError,
241           textAlign: TextAlign.center,
242           style: Theme.of(context).textTheme.titleMedium,
243         ),
244         const SizedBox(height: 8),
245         Text(
246           message,
247           textAlign: TextAlign.center,
248           style: Theme.of(context).textTheme.bodySmall,
249         ),
250       ],
251     );
252   }
253 }

```

app_flutter/lib/features/chuno/models/chuno_window.dart

```

1| class ChunoDay {
2|   ChunoDay({
3|     required this.date,
4|     required this.tier,
5|     required this.isGoodDay,
6|     this.temperatureMin,
7|     this.humidityMax,
8|     this.cloudCover,
9|     this.precipitation,
10|  });
11|
12|   final String date;
13|   final String tier;
14|   final bool isGoodDay;
15|   final double? temperatureMin;
16|   final double? humidityMax;
17|   final double? cloudCover;
18|   final double? precipitation;
19|
20|   factory ChunoDay.fromJson(Map<String, dynamic> json) => ChunoDay(
21|     date: json['date'] as String,
22|     tier: json['tier'] as String,
23|     isGoodDay: json['is_good_day'] as bool,
24|     temperatureMin: (json['temperature_min'] as num?)?.toDouble(),
25|     humidityMax: (json['humidity_max'] as num?)?.toDouble(),
26|     cloudCover: (json['cloud_cover_mean'] as num?)?.toDouble(),
27|     precipitation: (json['precipitation_sum'] as num?)?.toDouble(),
28|   );
29| }
30|
31| class ChunoWindow {
32|   ChunoWindow({
33|     required this.inSeason,
34|     required this.optimalWindow,
35|     required this.bestStreak,
36|     required this.message,
37|     required this.days,
38|   });
39|
40|   final bool inSeason;
41|   final bool optimalWindow;
42|   final int bestStreak;
43|   final String message;
44|   final List<ChunoDay> days;
45|
46|   factory ChunoWindow.fromJson(Map<String, dynamic> json) => ChunoWindow(
47|     inSeason: json['in_season'] as bool,
48|     optimalWindow: json['optimal_window'] as bool,
49|     bestStreak: (json['best_streak'] as num).toInt(),
50|     message: json['message'] as String,
51|     days: (json['days'] as List)
52|       .whereType<Map<String, dynamic>>()
53|       .map(ChunoDay.fromJson)
54|       .toList(),
55|   );
56| }

```

app_flutter/lib/features/chuno/screens/chuno_screen.dart

```

1| import 'package:flutter/material.dart';
2|
3| import '../../../core/api/frost_api_service.dart';
4| import '../../../core/i18n/app_strings.dart';

```

```

5 import '.././././core/theme/app_colors.dart';
6 import '.././././shared/widgets/glass_card.dart';
7 import '.././././shared/widgets/responsive_content.dart';
8 import '.././././shared/widgets/section_header.dart';
9 import '.././././shared/widgets/status_chip.dart';
10 import '../models/chuno_window.dart';
11
12 // Puno demo por defecto; en produccion la pantalla recibe GPS del usuario.
13 const _defaultLat = -15.8402;
14 const _defaultLon = -70.0219;
15
16 Color _dayColor(String tier) => switch (tier) {
17   'excelente' => AppColors.deepTeal,
18   'bueno' => AppColors.lowRisk,
19   'marginal' => AppColors.mediumRisk,
20   _ => AppColors.warmAmber,
21 };
22
23 class ChunoScreen extends StatefulWidget {
24   const ChunoScreen({required this.apiService, super.key});
25
26   final FrostApiService apiService;
27
28   @override
29   State<ChunoScreen> createState() => _ChunoScreenState();
30 }
31
32 class _ChunoScreenState extends State<ChunoScreen> {
33   late Future<ChunoWindow> _future = _load();
34
35   Future<ChunoWindow> _load() => widget.apiService.getChunoWindow(
36     latitude: _defaultLat,
37     longitude: _defaultLon,
38   );
39
40   @override
41   Widget build(BuildContext context) {
42     return SafeArea(
43       child: ResponsiveContent(
44         child: FutureBuilder<ChunoWindow>(
45           future: _future,
46           builder: (context, snapshot) {
47             if (snapshot.connectionState == ConnectionState.waiting) {
48               return const Center(child: CircularProgressIndicator());
49             }
50             if (snapshot.hasError) {
51               return _Error(message: snapshot.error.toString());
52             }
53             final data = snapshot.data!;
54             final s = AppStrings.current;
55             return RefreshIndicator(
56               onRefresh: () async {
57                 setState(() => _future = _load());
58                 await _future;
59               },
60               child: ListView(
61                 padding: const EdgeInsets.fromLTRB(20, 20, 20, 24),
62                 children: [
63                   SectionHeader(
64                     title: s.chunoTitle,
65                     subtitle: s.chunoSubtitle,
66                   ),
67                   const SizedBox(height: 20),
68                   _SeasonCard(data: data),
69                   const SizedBox(height: 24),
70                   Text(
71                     s.dailyForecast,
72                     style: Theme.of(context).textTheme.titleMedium,
73                   ),
74                   const SizedBox(height: 12),
75                   ...data.days.map((d) => _DayFile(day: d)),
76                 ],
77               ),
78             );
79           ),
80         ),
81       );
82     );
83   }
84 }
85
86 class _SeasonCard extends StatelessWidget {
87   const _SeasonCard({required this.data});
88
89   final ChunoWindow data;
90
91   @override
92   Widget build(BuildContext context) {
93     final onSurface = Theme.of(context).colorScheme.onSurface;
94     final s = AppStrings.current;
95     final color = data.optimalWindow
96       ? AppColors.deepTeal
97       : (data.inSeason ? AppColors.mediumRisk : AppColors.mutedTeal);
98     return GlassCard(
99       padding: const EdgeInsets.all(22),
100       child: Column(
101         crossAxisAlignment: CrossAxisAlignment.start,
102         children: [
103           Row(
104             children: [
105               Icon(Icons.ac_unit, color: color, size: 34),
106               const SizedBox(width: 12),
107               Expanded(
108                 child: Text(
109                   data.optimalWindow
110                     ? s.optimalWindow
111                     : data.inSeason
112                     ? s.inSeason
113                     : s.offSeason,
114                   style: Theme.of(context).textTheme.titleLarge,
115                 ),
116               ),
117               StatusChip(
118                 label: s.streak(data.bestStreak),
119                 color: color,
120               ),
121             ],
122           ),
123           const SizedBox(height: 14),
124           Text(
125             data.message,

```

```

126         style: Theme.of(context).textTheme.bodyMedium?.copyWith(
127           color: onSurface.withValues(alpha: 0.72),
128         ),
129       ),
130     ],
131   ),
132 );
133 }
134 }
135
136 class _DayTile extends StatelessWidget {
137   const _DayTile({required this.day});
138
139   final ChunoDay day;
140
141   @override
142   Widget build(BuildContext context) {
143     final color = _dayColor(day.tier);
144     return Padding(
145       padding: const EdgeInsets.only(bottom: 18),
146       child: GlassCard(
147         padding: const EdgeInsets.all(16),
148         child: Row(
149           children: [
150             Icon(
151               day.isGoodDay ? Icons.check_circle : Icons.remove_circle_outline,
152               color: color,
153             ),
154             const SizedBox(width: 14),
155             Expanded(
156               child: Column(
157                 crossAxisAlignment: CrossAxisAlignment.start,
158                 children: [
159                   Text(day.date, style: Theme.of(context).textTheme.titleSmall),
160                   const SizedBox(height: 2),
161                   Text(
162                     AppStrings.current.chunoDayMetrics(
163                       day.temperatureMin?.toStringAsFixed(1) ?? '-',
164                       day.cloudCover?.toStringAsFixed(0) ?? '-',
165                       day.humidityMax?.toStringAsFixed(0) ?? '-',
166                     ),
167                     style: Theme.of(context).textTheme.bodySmall,
168                   ),
169                 ],
170             ),
171             StatusChip(
172               label: AppStrings.current.tierWord(day.tier),
173               color: color,
174             ),
175           ],
176         ),
177       ),
178     );
179   }
180 }
181
182 class _Error extends StatelessWidget {
183   const _Error({required this.message});
184
185   final String message;
186
187   @override
188   Widget build(BuildContext context) {
189     return ListView(
190       padding: const EdgeInsets.all(24),
191       children: [
192         const SizedBox(height: 80),
193         const Icon(Icons.cloud_off, size: 48, color: AppColors.warmAmber),
194         const SizedBox(height: 16),
195         Text(
196           AppStrings.current.chunoError,
197           textAlign: TextAlign.center,
198           style: Theme.of(context).textTheme.titleMedium,
199         ),
200         const SizedBox(height: 8),
201         Text(
202           message,
203           textAlign: TextAlign.center,
204           style: Theme.of(context).textTheme.bodySmall,
205         ),
206       ],
207     );
208   }
209 }
210 }

```

app_flutter/lib/features/clusters/models/cluster_info.dart

```

1 class ClusterProfile {
2   ClusterProfile({
3     required this.clusterId,
4     required this.tier,
5     required this.size,
6     required this.temperature,
7     required this.dewPoint,
8   });
9
10  final int clusterId;
11  final String tier;
12  final int size;
13  final double temperature;
14  final double dewPoint;
15
16  factory ClusterProfile.fromJson(Map<String, dynamic> json) => ClusterProfile(
17    clusterId: (json['cluster_id'] as num).toInt(),
18    tier: json['tier'] as String,
19    size: (json['size'] as num).toInt(),
20    temperature: (json['temperature_2m'] as num).toDouble(),
21    dewPoint: (json['dew_point_2m'] as num).toDouble(),
22  );
23 }
24
25 class DistrictCluster {
26   DistrictCluster({
27     required this.district,
28     required this.tier,

```

```

29 required this.dominantCluster,
30 required this.coldShare,
31 required this.altitude,
32 required this.temperatureMean,
33 });
34
35 final String distrito;
36 final String tier;
37 final int dominantCluster;
38 final double coldShare;
39 final double altitude;
40 final double temperatureMean;
41
42 factory DistrictCluster.fromJson(Map<String, dynamic> json) =>
43   DistrictCluster(
44     distrito: json['distrito'] as String,
45     tier: json['tier'] as String,
46     dominantCluster: (json['dominant_cluster'] as num).toInt(),
47     coldShare: (json['cold_share'] as num).toDouble(),
48     altitude: (json['altitud_estimada'] as num).toDouble(),
49     temperatureMean: (json['temperature_2m_mean'] as num).toDouble(),
50   );
51 }
52
53 class ClustersResult {
54   ClustersResult({
55     required this.modelName,
56     required this.version,
57     required this.nClusters,
58     required this.silhouette,
59     required this.profiles,
60     required this.districts,
61   });
62
63   final String modelName;
64   final String version;
65   final int nClusters;
66   final double silhouette;
67   final List<ClusterProfile> profiles;
68   final List<DistrictCluster> districts;
69
70   factory ClustersResult.fromJson(Map<String, dynamic> json) => ClustersResult(
71     modelName: json['model_name'] as String,
72     version: json['version'] as String,
73     nClusters: (json['n_clusters'] as num).toInt(),
74     silhouette: (json['silhouette'] as num).toDouble(),
75     profiles: (json['profiles'] as List)
76       .whereType<Map<String, dynamic>>()
77       .map(ClusterProfile.fromJson)
78       .toList(),
79     districts: (json['districts'] as List)
80       .whereType<Map<String, dynamic>>()
81       .map(DistrictCluster.fromJson)
82       .toList(),
83   );
84 }

```

app_flutter/lib/features/clusters/screens/clusters_screen.dart

```

1 import 'package:flutter/material.dart';
2
3 import '../core/api/frost_api_service.dart';
4 import '../core/i18n/app_strings.dart';
5 import '../core/theme/app_colors.dart';
6 import '../shared/widgets/glass_card.dart';
7 import '../shared/widgets/responsive_content.dart';
8 import '../shared/widgets/section_header.dart';
9 import '../shared/widgets/status_chip.dart';
10 import '../models/cluster_info.dart';
11
12 Color tierColor(String tier) => switch (tier) {
13   'alto' => AppColors.warmAmber,
14   'medio' => AppColors.mediumRisk,
15   _ => AppColors.lowRisk,
16 };
17
18 class ClustersScreen extends StatefulWidget {
19   const ClustersScreen({required this.apiService, super.key});
20
21   final FrostApiService apiService;
22
23   @override
24   State<ClustersScreen> createState() => _ClustersScreenState();
25 }
26
27 class _ClustersScreenState extends State<ClustersScreen> {
28   late Future<ClustersResult> _future = widget.apiService.getClusters();
29
30   @override
31   Widget build(BuildContext context) {
32     return SafeArea(
33       child: ResponsiveContent(
34         child: RefreshIndicator(
35           onRefresh: () async {
36             setState(() => _future = widget.apiService.getClusters());
37             await _future;
38           },
39           child: FutureBuilder<ClustersResult>(
40             future: _future,
41             builder: (context, snapshot) {
42               if (snapshot.connectionState == ConnectionState.waiting) {
43                 return const Center(child: CircularProgressIndicator());
44               }
45               if (snapshot.hasError) {
46                 return _ErrorView(message: snapshot.error.toString());
47               }
48               final s = AppStrings.current;
49               final data = snapshot.data!;
50               return ListView(
51                 padding: const EdgeInsets.fromLTRB(20, 20, 20, 24),
52                 children: [
53                   SectionHeader(
54                     title: s.zonesTitle,
55                     subtitle: s.zonesSubtitle,
56                   ),
57                   const SizedBox(height: 20),

```

```

58         GlassCard(
59             child: Row(
60                 children: [
61                     Expanded(
62                         child: _Metric(
63                             label: s.clusters,
64                             value: '${data.nClusters}',
65                         ),
66                     ),
67                     Expanded(
68                         child: _Metric(
69                             label: s.silhouette,
70                             value: data.silhouette.toStringAsFixed(3),
71                         ),
72                     ),
73                     Expanded(
74                         child: _Metric(
75                             label: s.model,
76                             value: data.modelName,
77                         ),
78                     ),
79                 ],
80             ),
81         ),
82         const SizedBox(height: 24),
83         Text(
84             s.districtsByLevel,
85             style: Theme.of(context).textTheme.titleMedium,
86         ),
87         const SizedBox(height: 12),
88         ...data.districts.map((d) => _DistrictTile(district: d)),
89     ],
90 );
91 },
92 ),
93 ),
94 ),
95 );
96 }
97 }
98
99 class _DistrictTile extends StatelessWidget {
100     const _DistrictTile({required this.district});
101
102     final DistrictCluster district;
103
104     @override
105     Widget build(BuildContext context) {
106         final color = tierColor(district.tier);
107         return Padding(
108             padding: const EdgeInsets.only(bottom: 12),
109             child: GlassCard(
110                 padding: const EdgeInsets.all(18),
111                 child: Row(
112                     children: [
113                         Container(
114                             width: 18,
115                             height: 44,
116                             decoration: BoxDecoration(
117                                 color: color,
118                                 borderRadius: BorderRadius.circular(6),
119                             ),
120                         ),
121                         const SizedBox(width: 16),
122                         Expanded(
123                             child: Column(
124                                 crossAxisAlignment: CrossAxisAlignment.start,
125                                 children: [
126                                     Text(
127                                         district.districto,
128                                         style: Theme.of(context).textTheme.titleMedium,
129                                     ),
130                                     const SizedBox(height: 4),
131                                     Text(
132                                         '${district.altitude.toStringAsFixed(0)} m · ☼${district.temperatureMean.toStringAsFixed(1)} °C',
133                                         style: Theme.of(context).textTheme.bodyMedium?.copyWith(
134                                             color: Theme.of(
135                                                 context,
136                                             ).colorScheme.onSurface.withValues(alpha: 0.66),
137                                         ),
138                                     ),
139                                 ],
140                             ),
141                         ),
142                         Column(
143                             crossAxisAlignment: CrossAxisAlignment.end,
144                             children: [
145                                 StatusChip(
146                                     label: AppStrings.current.tierWord(district.tier),
147                                     color: color,
148                                 ),
149                                 const SizedBox(height: 6),
150                                 Text(
151                                     AppStrings.current.coldShare(
152                                         '${(district.coldShare * 100).toStringAsFixed(0)}%',
153                                     ),
154                                     style: Theme.of(context).textTheme.bodySmall,
155                                 ),
156                             ],
157                         ),
158                     ],
159                 ),
160             );
161     }
162 }
163
164 class _Metric extends StatelessWidget {
165     const _Metric({required this.label, required this.value});
166
167     final String label;
168     final String value;
169
170     @override
171     Widget build(BuildContext context) {
172         return Column(
173             children: [
174                 Text(value, style: Theme.of(context).textTheme.titleLarge),
175                 const SizedBox(height: 4),
176                 Text(label, style: Theme.of(context).textTheme.bodySmall),
177             ],
178         );

```

```

179     );
180   }
181 }
182
183 class _ErrorView extends StatelessWidget {
184   const _ErrorView({required this.message});
185
186   final String message;
187
188   @override
189   Widget build(BuildContext context) {
190     return ListView(
191       padding: const EdgeInsets.all(24),
192       children: [
193         const SizedBox(height: 80),
194         const Icon(Icons.cloud_off, size: 48, color: AppColors.warmAmber),
195         const SizedBox(height: 16),
196         Text(
197           AppStrings.current.zonesError,
198           textAlign: TextAlign.center,
199           style: Theme.of(context).textTheme.titleMedium,
200         ),
201         const SizedBox(height: 8),
202         Text(
203           message,
204           textAlign: TextAlign.center,
205           style: Theme.of(context).textTheme.bodySmall,
206         ),
207       ],
208     );
209   }
210 }

```

app_flutter/lib/features/data_sources/models/data_source_info.dart

```

1| import 'package:flutter/material.dart';
2|
3| class DataSourceInfo {
4|   const DataSourceInfo({
5|     required this.name,
6|     required this.tag,
7|     required this.description,
8|     required this.icon,
9|   });
10|
11|   final String name;
12|   final String tag;
13|   final String description;
14|   final IconData icon;
15| }

```

app_flutter/lib/features/data_sources/screens/data_sources_screen.dart

```

1| import 'package:flutter/material.dart';
2|
3| import '../../../core/i18n/app_strings.dart';
4| import '../../../core/theme/app_colors.dart';
5| import '../../../shared/widgets/glass_card.dart';
6| import '../../../shared/widgets/page_scaffold.dart';
7| import '../../../shared/widgets/responsive_content.dart';
8| import '../../../shared/widgets/section_header.dart';
9| import '../../../shared/widgets/status_chip.dart';
10| import '../../../models/data_source_info.dart';
11|
12| class DataSourcesScreen extends StatelessWidget {
13|   const DataSourcesScreen({super.key, this.showAppBar = true});
14|
15|   final bool showAppBar;
16|
17|   static List<DataSourceInfo> sourcesFor(AppStrings s) => [
18|     DataSourceInfo(
19|       name: 'Open-Meteo',
20|       tag: 'FALLBACK',
21|       description: s.sourceOpenMeteo,
22|       icon: Icons.cloud_outlined,
23|     ),
24|     DataSourceInfo(
25|       name: 'INEI',
26|       tag: 'TERRITORIO',
27|       description: s.sourceInei,
28|       icon: Icons.map_outlined,
29|     ),
30|     DataSourceInfo(
31|       name: 'SENAMHI',
32|       tag: 'OFICIAL',
33|       description: s.sourceSenamhi,
34|       icon: Icons.device_thermostat,
35|     ),
36|     DataSourceInfo(
37|       name: 'MIDAGRI/SIEA',
38|       tag: 'AGRO',
39|       description: s.sourceMidagri,
40|       icon: Icons.agriculture_outlined,
41|     ),
42|   ];
43|
44|   @override
45|   Widget build(BuildContext context) {
46|     final s = AppStrings.current;
47|     final sources = sourcesFor(s);
48|     final body = ListView(
49|       padding: const EdgeInsets.fromLTRB(24, 28, 24, 32),
50|       children: [
51|         SectionHeader(
52|           title: s.sourcesTitle,
53|           subtitle: s.sourcesSubtitle,
54|         ),
55|         const SizedBox(height: 28),
56|         for (final source in sources) ...[
57|           _SourceCard(source: source),
58|           const SizedBox(height: 18),

```



```

59     ],
60     GlassCard(
61       child: Row(
62         children: [
63           Icon(
64             Icons.info_outline,
65             color: Theme.of(
66               context,
67             ).colorScheme.onSurface.withValues(alpha: 0.66),
68           ),
69           const SizedBox(width: 14),
70           Expanded(
71             child: Text(
72               s.sourcesFooter,
73               style: Theme.of(context).textTheme.bodyLarge?.copyWith(
74                 color: Theme.of(
75                   context,
76                 ).colorScheme.onSurface.withValues(alpha: 0.68),
77               fontStyle: FontStyle.italic,
78             ),
79           ),
80         ],
81       ),
82     ),
83   ),
84   const SizedBox(height: 18),
85   const _ValidationPipelineCard(),
86 ],
87 );
88
89 if (!showAppBar) {
90   return SafeArea(child: ResponsiveContent(child: body));
91 }
92 return PageScaffold(child: body);
93 }
94 }
95
96 class _ValidationPipelineCard extends StatelessWidget {
97   const _ValidationPipelineCard();
98
99   @override
100   Widget build(BuildContext context) {
101     final textTheme = Theme.of(context).textTheme;
102     final onSurface = Theme.of(context).colorScheme.onSurface;
103     final s = AppStrings.current;
104     final steps = s.pipelineSteps;
105     return GlassCard(
106       child: Column(
107         crossAxisAlignment: CrossAxisAlignment.start,
108         children: [
109           Row(
110             children: [
111               const Icon(Icons.timeline, color: AppColors.warmAmber),
112               const SizedBox(width: 12),
113               Expanded(
114                 child: Text(
115                   s.improvementCycle,
116                   style: textTheme.headlineMedium,
117                 ),
118               ),
119             ],
120           ),
121           const SizedBox(height: 16),
122           for (final step in steps) ...[
123             _PipelineStep(
124               number: step.$1,
125               title: step.$2,
126               description: step.$3,
127             ),
128             if (step != steps.last) const SizedBox(height: 12),
129           ],
130           const SizedBox(height: 16),
131           Text(
132             s.pipelineFooter,
133             style: textTheme.bodyMedium?.copyWith(
134               color: onSurface.withValues(alpha: 0.66),
135             fontStyle: FontStyle.italic,
136           ),
137         ],
138       ),
139     );
140   }
141 }
142
143 class _PipelineStep extends StatelessWidget {
144   const _PipelineStep({
145     required this.number,
146     required this.title,
147     required this.description,
148   });
149
150   final String number;
151   final String title;
152   final String description;
153
154   @override
155   Widget build(BuildContext context) {
156     final textTheme = Theme.of(context).textTheme;
157     return Row(
158       crossAxisAlignment: CrossAxisAlignment.start,
159       children: [
160         CircleAvatar(
161           radius: 14,
162           backgroundColor: AppColors.mutedTeal,
163           child: Text(number, style: textTheme.labelSmall),
164         ),
165         const SizedBox(width: 12),
166         Expanded(
167           child: Column(
168             crossAxisAlignment: CrossAxisAlignment.start,
169             children: [
170               Text(title, style: textTheme.titleMedium),
171               Text(description, style: textTheme.bodyMedium),
172             ],
173           ),
174         ),
175       ],
176     );
177   }
178 }
179 }

```

```

180
181 class _SourceCard extends StatelessWidget {
182   const _SourceCard({required this.source});
183
184   final DataSourceInfo source;
185
186   @override
187   Widget build(BuildContext context) {
188     return GlassCard(
189       child: Column(
190         crossAxisAlignment: CrossAxisAlignment.start,
191         children: [
192           Row(
193             children: [
194               Icon(source.icon, color: AppColors.mutedTeal, size: 30),
195               const SizedBox(width: 12),
196               Text(
197                 source.name,
198                 style: Theme.of(context).textTheme.headlineMedium,
199               ),
200             ],
201           ),
202           const SizedBox(height: 14),
203           StatusChip(label: source.tag),
204           const SizedBox(height: 14),
205           Text(
206             source.description,
207             style: Theme.of(context).textTheme.bodyLarge,
208           ),
209         ],
210       ),
211     );
212   }
213 }

```

app_flutter/lib/features/history/models/prediction_history_item.dart

```

1| class PredictionHistoryItem {
2|   const PredictionHistoryItem({
3|     required this.id,
4|     required this.district,
5|     required this.province,
6|     required this.latitude,
7|     required this.longitude,
8|     required this.riskLevel,
9|     required this.confidence,
10|    required this.chunoConditions,
11|    required this.recommendation,
12|    required this.modelVersion,
13|    required this.dataSources,
14|    required this.createdAt,
15|    this.populatedCenter,
16|    this.altitude,
17|  });
18|
19|   final String id;
20|   final String district;
21|   final String province;
22|   final String? populatedCenter;
23|   final double latitude;
24|   final double longitude;
25|   final double? altitude;
26|   final String riskLevel;
27|   final double confidence;
28|   final String chunoConditions;
29|   final String recommendation;
30|   final String modelVersion;
31|   final List<String> dataSources;
32|   final String createdAt;
33|
34|   factory PredictionHistoryItem.fromJson(Map<String, dynamic> json) {
35|     return PredictionHistoryItem(
36|       id: json['id'] as String? ?? '-',
37|       district: json['district'] as String? ?? '-',
38|       province: json['province'] as String? ?? '-',
39|       populatedCenter: json['populated_center'] as String?,
40|       latitude: (json['latitude'] as num? ?? 0).toDouble(),
41|       longitude: (json['longitude'] as num? ?? 0).toDouble(),
42|       altitude: (json['altitude'] as num? ?? 0).toDouble(),
43|       riskLevel: json['risk_level'] as String? ?? 'bajo',
44|       confidence: (json['confidence'] as num? ?? 0).toDouble(),
45|       chunoConditions: json['chuno_conditions'] as String? ?? '-',
46|       recommendation: json['recommendation'] as String? ?? '',
47|       modelVersion: json['model_version'] as String? ?? '-',
48|       dataSources: (json['data_sources'] as List<dynamic>? ?? const [])
49|         .map((item) => item.toString())
50|         .toList(),
51|       createdAt: json['created_at'] as String? ?? '-',
52|     );
53|   }
54| }

```

app_flutter/lib/features/history/screens/history_screen.dart

```

1| import 'package:flutter/material.dart';
2|
3| import '../../../core/api/frost_api_service.dart';
4| import '../../../core/i18n/app_strings.dart';
5| import '../../../core/theme/app_colors.dart';
6| import '../../../shared/widgets/glass_card.dart';
7| import '../../../shared/widgets/page_scaffold.dart';
8| import '../../../shared/widgets/responsive_content.dart';
9| import '../../../shared/widgets/section_header.dart';
10| import '../../../shared/widgets/status_chip.dart';
11| import '../../../models/prediction_history_item.dart';
12|
13| class HistoryScreen extends StatefulWidget {
14|   const HistoryScreen({
15|     required this.apiService,
16|     super.key,
17|     this.showAppBar = true,

```

```

18  });
19
20  final FrostApiService apiService;
21  final bool showAppBar;
22
23  @override
24  State<HistoryScreen> createState() => _HistoryScreenState();
25 }
26
27 class _HistoryScreenState extends State<HistoryScreen> {
28   late Future<List<PredictionHistoryItem>> _future = widget.apiService
29     .getPredictionHistory();
30
31   @override
32   Widget build(BuildContext context) {
33     final s = AppStrings.current;
34     final body = RefreshIndicator(
35       onRefresh: () async {
36         setState(() => _future = widget.apiService.getPredictionHistory());
37         await _future;
38       },
39       child: ListView(
40         padding: const EdgeInsets.fromLTRB(24, 28, 24, 32),
41         children: [
42           SectionHeader(
43             title: s.historyTitle,
44             subtitle: s.historySubtitle,
45           ),
46           const SizedBox(height: 26),
47           FutureBuilder<List<PredictionHistoryItem>>(
48             future: _future,
49             builder: (context, snapshot) {
50               if (snapshot.connectionState == ConnectionState.waiting) {
51                 return const Center(
52                   child: Padding(
53                     padding: EdgeInsets.all(32),
54                     child: CircularProgressIndicator(),
55                   ),
56                 );
57               }
58               if (snapshot.hasError) {
59                 return _EmptyHistory(message: s.historyError);
60               }
61               final items = snapshot.data ?? const [];
62               if (items.isEmpty) {
63                 return _EmptyHistory(message: s.historyEmpty);
64               }
65               return Column(
66                 children: items
67                   .map((item) => _HistoryCard(item: item))
68                   .toList(),
69               );
70             },
71           ),
72         ],
73       ),
74     );
75
76     if (!widget.showAppBar) {
77       return SafeArea(child: ResponsiveContent(child: body));
78     }
79     return PageScaffold(child: body);
80   }
81 }
82
83 class _HistoryCard extends StatelessWidget {
84   const _HistoryCard({required this.item});
85
86   final PredictionHistoryItem item;
87
88   @override
89   Widget build(BuildContext context) {
90     final color = _riskColor(item.riskLevel);
91     final icon = _riskIcon(item.riskLevel);
92     final onSurface = Theme.of(context).colorScheme.onSurface;
93     return Padding(
94       padding: const EdgeInsets.only(bottom: 18),
95       child: GlassCard(
96         padding: const EdgeInsets.all(18),
97         child: Column(
98           crossAxisAlignment: CrossAxisAlignment.start,
99           children: [
100             Row(
101               children: [
102                 Expanded(
103                   child: Text(
104                     item.district,
105                     style: Theme.of(
106                       context,
107                     ).textTheme.headlineMedium?.copyWith(color: onSurface),
108                   ),
109                 ),
110                 StatusChip(
111                   icon: icon,
112                   label: AppStrings.current.riskChip(item.riskLevel),
113                   color: color,
114                   background: color.withValues(alpha: 0.14),
115                 ),
116               ],
117             ),
118             const SizedBox(height: 8),
119             Text(
120               _formatDate(item.createdAt),
121               style: TextStyle(
122                 fontFamily: 'monospace',
123                 color: onSurface.withValues(alpha: 0.66),
124             ),
125             ),
126             const SizedBox(height: 18),
127             const Divider(),
128             const SizedBox(height: 14),
129             Row(
130               mainAxisAlignment: MainAxisAlignment.spaceBetween,
131               children: [
132                 _SmallMetric(
133                   label: AppStrings.current.confidence,
134                   value: '${(item.confidence * 100).round()}%',
135                 ),
136                 _SmallMetric(
137                   label: AppStrings.current.date,
138                   value: _shortDate(item.createdAt),

```

```

139         ),
140       ],
141     ),
142   ],
143 ),
144 ),
145 );
146 }
147
148 static Color _riskColor(String value) {
149   return switch (value.toLowerCase()) {
150     'alto' => AppColors.warmAmber,
151     'medio' => AppColors.mediumRisk,
152     'moderado' => AppColors.mediumRisk,
153     _ => AppColors.lowRisk,
154   };
155 }
156
157 static IconData _riskIcon(String value) {
158   return switch (value.toLowerCase()) {
159     'alto' => Icons.warning_amber_rounded,
160     'medio' => Icons.report_problem_outlined,
161     'moderado' => Icons.report_problem_outlined,
162     _ => Icons.check_circle_outline,
163   };
164 }
165
166 static String _formatDate(String raw) {
167   final date = DateTime.tryParse(raw);
168   if (date == null) return raw;
169   return '${_two(date.day)}/${_two(date.month)}/${date.year} ${_two(date.hour)}:${_two(date.minute)}';
170 }
171
172 static String _shortDate(String raw) {
173   final date = DateTime.tryParse(raw);
174   if (date == null) return raw;
175   return '${_two(date.day)}/${_two(date.month)}';
176 }
177
178 static String _two(int value) => value.toString().padLeft(2, '0');
179 }
180
181 class _SmallMetric extends StatelessWidget {
182   const _SmallMetric({required this.label, required this.value});
183
184   final String label;
185   final String value;
186
187   @override
188   Widget build(BuildContext context) {
189     return Column(
190       crossAxisAlignment: CrossAxisAlignment.start,
191       children: [
192         Text(
193           label,
194           style: Theme.of(context).textTheme.labelSmall?.copyWith(
195             fontFamily: 'monospace',
196             letterSpacing: 0,
197           ),
198         ),
199         const SizedBox(height: 8),
200         Text(
201           value,
202           style: TextStyle(
203             fontFamily: 'monospace',
204             fontSize: 20,
205             fontWeight: FontWeight.w700,
206             color: Theme.of(context).colorScheme.onSurface,
207           ),
208         ),
209       ],
210     );
211   }
212 }
213
214 class _EmptyHistory extends StatelessWidget {
215   const _EmptyHistory({required this.message});
216
217   final String message;
218
219   @override
220   Widget build(BuildContext context) {
221     return GlassCard(
222       child: Column(
223         children: [
224           Icon(
225             Icons.history_toggle_off,
226             size: 42,
227             color: Theme.of(
228               context,
229             ).colorScheme.onSurface.withValues(alpha: 0.56),
230           ),
231           const SizedBox(height: 14),
232           Text(
233             message,
234             textAlign: TextAlign.center,
235             style: Theme.of(context).textTheme.bodyLarge,
236           ),
237         ],
238       ),
239     );
240   }
241 }

```

app_flutter/lib/features/home/screens/home_screen.dart

```

1| import 'package:flutter/material.dart';
2|
3| import '../../../core/api/frost_api_service.dart';
4| import '../../../core/i18n/app_strings.dart';
5| import '../../../core/models/health_status.dart';
6| import '../../../core/notifications/notification_service.dart';
7| import '../../../core/settings/user_settings.dart';
8| import '../../../core/theme/app_colors.dart';
9| import '../../../core/theme/theme_mode_scope.dart';
10| import '../../../alerts/models/daily_alert.dart';

```

```

11| import '../settings/screens/settings_screen.dart';
12| import '../shared/widgets/glass_card.dart';
13| import '../shared/widgets/primary_action_button.dart';
14| import '../shared/widgets/responsive_content.dart';
15| import '../shared/widgets/status_chip.dart';
16| import '../data_sources/screens/data_sources_screen.dart';
17| import '../history/screens/history_screen.dart';
18| import '../model_info/screens/model_info_screen.dart';
19| import '../prediction/screens/prediction_form_screen.dart';
20|
21| class HomeScreen extends StatefulWidget {
22|   const HomeScreen({required this.apiService, super.key});
23|
24|   final FrostApiService apiService;
25|
26|   @override
27|   State<HomeScreen> createState() => _HomeScreenState();
28| }
29|
30| class _HomeScreenState extends State<HomeScreen> {
31|   static const _defaultLat = -15.8402;
32|   static const _defaultLon = -70.0219;
33|
34|   late final Future<HealthStatus> _healthFuture = widget.apiService.getHealth();
35|   late final Future<DailyAlert> _alertFuture = widget.apiService.getDailyAlert(
36|     latitude: _defaultLat,
37|     longitude: _defaultLon,
38|   );
39|
40|   @override
41|   Widget build(BuildContext context) {
42|     final textTheme = Theme.of(context).textTheme;
43|     final onSurface = Theme.of(context).colorScheme.onSurface;
44|     final themeScope = ThemeModeScope.maybeOf(context);
45|     final s = AppStrings.current;
46|     return SafeArea(
47|       child: ResponsiveContent(
48|         child: ListView(
49|           padding: const EdgeInsets.fromLTRB(20, 28, 20, 24),
50|           children: [
51|             Row(
52|               children: [
53|                 Expanded(
54|                   child: FutureBuilder<HealthStatus>(
55|                     future: _healthFuture,
56|                     builder: (context, snapshot) {
57|                       final active = snapshot.data?.modelAvailable ?? false;
58|                       return StatusChip(
59|                         icon: Icons.circle,
60|                         label: active ? s.systemActive : s.demoMode,
61|                         color: active
62|                           ? AppColors.mutedTeal
63|                           : AppColors.warmAmber,
64|                       );
65|                     },
66|                   ),
67|                 IconButton(
68|                   tooltip: s.changeTheme,
69|                   icon: Icon(_themeIcon(themeScope?.themeMode)),
70|                   onPressed: themeScope == null
71|                     ? null
72|                     : () => themeScope.onThemeModeChanged(
73|                         _nextThemeMode(themeScope.themeMode),
74|                       ),
75|                 ),
76|                 IconButton(
77|                   tooltip: s.settings,
78|                   icon: const Icon(Icons.tune),
79|                   onPressed: () => Navigator.of(context).push(
80|                     MaterialPageRoute(builder: (_) => const SettingsScreen()),
81|                   ),
82|                 ),
83|               ],
84|             ),
85|             const SizedBox(height: 32),
86|             Text('FrostPuno', style: textTheme.displayLarge),
87|             const SizedBox(height: 16),
88|             Text(
89|               s.appTagline,
90|               style: textTheme.headlineMedium?.copyWith(
91|                 color: onSurface.withValues(alpha: 0.68),
92|                 fontSize: 22,
93|                 fontWeight: FontWeight.w400,
94|               ),
95|             ),
96|             const SizedBox(height: 24),
97|             const _AlertSection(future: _alertFuture),
98|             const SizedBox(height: 20),
99|             const _CurrentRiskCard(),
100|             const SizedBox(height: 32),
101|             PrimaryActionButton(
102|               label: s.checkRisk,
103|               icon: Icons.arrow_forward,
104|               onPressed: () => Navigator.of(context).push(
105|                 MaterialPageRoute(
106|                   builder: (_) =>
107|                     PredictionFormScreen(apiService: widget.apiService),
108|                 ),
109|             ),
110|             const SizedBox(height: 18),
111|             Row(
112|               children: [
113|                 Expanded(
114|                   child: _QuickTile(
115|                     icon: Icons.history,
116|                     label: s.viewHistory,
117|                     onTap: () => Navigator.of(context).push(
118|                       MaterialPageRoute(
119|                         builder: (_) =>
120|                           HistoryScreen(apiService: widget.apiService),
121|                       ),
122|                     ),
123|                 ),
124|                 const SizedBox(width: 16),
125|                 Expanded(
126|                   child: _QuickTile(
127|                     icon: Icons.storage_outlined,
128|                     label: s.dataSources,
129|                   ),
130|                 ),
131|               ],
132|             ),
133|

```

```

132         onTap: () => Navigator.of(context).push(
133             MaterialPageRoute(
134                 builder: (_) => const DataSourcesScreen(),
135             ),
136         ),
137     ),
138 ),
139 ],
140 ),
141 const SizedBox(height: 16),
142 _QuickTile(
143     icon: Icons.account_tree_outlined,
144     label: s.modelInfo,
145     onTap: () => Navigator.of(context).push(
146         MaterialPageRoute(
147             builder: (_) =>
148                 ModelInfoScreen(apiService: widget.apiService),
149         ),
150     ),
151 ),
152 const SizedBox(height: 18),
153 const _LearningCycleCard(),
154 ],
155 ),
156 ),
157 );
158 }
159
160 static ThemeMode _nextThemeMode(ThemeMode? mode) {
161     return switch (mode ?? ThemeMode.system) {
162         ThemeMode.system => ThemeMode.light,
163         ThemeMode.light => ThemeMode.dark,
164         ThemeMode.dark => ThemeMode.system,
165     };
166 }
167
168 static IconData _themeIcon(ThemeMode? mode) {
169     return switch (mode ?? ThemeMode.system) {
170         ThemeMode.system => Icons.brightness_auto_outlined,
171         ThemeMode.light => Icons.light_mode_outlined,
172         ThemeMode.dark => Icons.dark_mode_outlined,
173     };
174 }
175 }
176
177 Color _levelColor(String level) => switch (level) {
178     'alto' => AppColors.warmAmber,
179     'medio' => AppColors.mediumRisk,
180     _ => AppColors.lowRisk,
181 };
182
183 class _AlertSection extends StatefulWidget {
184     const _AlertSection({required this.future});
185
186     final Future<DailyAlert> future;
187
188     @override
189     State<_AlertSection> createState() => _AlertSectionState();
190 }
191
192 class _AlertSectionState extends State<_AlertSection> {
193     bool _notified = false;
194
195     bool _shouldAlert(DailyAlert alert, UserSettings settings) {
196         if (!settings.alertsEnabled) return false;
197         if (alert.frostAlert) return true;
198         final tmin = alert.temperatureMin;
199         return tmin != null && tmin <= settings.alertThreshold;
200     }
201
202     @override
203     Widget build(BuildContext context) {
204         final settings = UserSettings.instance;
205         return FutureBuilder<DailyAlert>({
206             future: widget.future,
207             builder: (context, snapshot) {
208                 if (!snapshot.hasData) return const SizedBox.shrink();
209                 final alert = snapshot.data!;
210                 return AnimatedBuilder(
211                     animation: settings,
212                     builder: (context, _) {
213                         final active = _shouldAlert(alert, settings);
214                         final s = AppStrings.current;
215                         if (active && !_notified) {
216                             _notified = true;
217                             WidgetsBinding.instance.addPostFrameCallback((_) {
218                                 NotificationService.showFrostAlert(
219                                     s.frostTitle(alert.severity),
220                                     s.frostMessage(alert.severity),
221                                 );
222                             });
223                         }
224
225                         final profile = settings.profile;
226                         final showLivestock =
227                             profile == UserProfile.general ||
228                             profile == UserProfile.ganadero;
229                         final cards = <Widget>[];
230
231                         if (settings.alertsEnabled) {
232                             cards.add(
233                                 _FrostBanner(alert: alert, active: active, settings: settings),
234                             );
235                         }
236                         if (showLivestock && alert.livestock.level != 'bajo') {
237                             cards.add(_LivestockCard(livestock: alert.livestock));
238                         }
239                         if (alert.anomaly.isUnusual) {
240                             cards.add(_AnomalyCard(anomaly: alert.anomaly));
241                         }
242                         if (cards.isEmpty) return const SizedBox.shrink();
243
244                         return Column(
245                             children: [
246                                 for (var i = 0; i < cards.length; i++) ...[
247                                     if (i > 0) const SizedBox(height: 12),
248                                     cards[i],
249                                 ],
250                             ],
251                         );
252                     },

```

```

253     );
254   },
255   );
256 }
257 }
258
259 class _FrostBanner extends StatelessWidget {
260   const _FrostBanner({
261     required this.alert,
262     required this.active,
263     required this.settings,
264   });
265
266   final DailyAlert alert;
267   final bool active;
268   final UserSettings settings;
269
270   @override
271   Widget build(BuildContext context) {
272     final color = active ? _levelColor(alert.riskLevel) : AppColors.lowRisk;
273     final icon = active ? Icons.warning_amber_rounded : Icons.check_circle_outline;
274     return GlassCard(
275       padding: const EdgeInsets.all(18),
276       color: color.withValues(alpha: 0.14),
277       child: Row(
278         crossAxisAlignment: CrossAxisAlignment.start,
279         children: [
280           Icon(icon, color: color, size: 30),
281           const SizedBox(width: 14),
282           Expanded(
283             child: Column(
284               crossAxisAlignment: CrossAxisAlignment.start,
285               children: [
286                 Text(
287                   AppStrings.current.frostTitle(alert.severity),
288                   style: Theme.of(context).textTheme.titleMedium?.copyWith(
289                     color: color,
290                     fontWeight: FontWeight.w700,
291                   ),
292                 ),
293                 const SizedBox(height: 6),
294                 Text(
295                   AppStrings.current.frostMessage(alert.severity),
296                   style: Theme.of(context).textTheme.bodyMedium,
297                 ),
298               ],
299             ),
300           ),
301         ],
302       ),
303     );
304   }
305 }
306
307 class _LivestockCard extends StatelessWidget {
308   const _LivestockCard({required this.livestock});
309
310   final LivestockRisk livestock;
311
312   @override
313   Widget build(BuildContext context) {
314     final color = _levelColor(livestock.level);
315     return GlassCard(
316       padding: const EdgeInsets.all(18),
317       color: color.withValues(alpha: 0.12),
318       child: Row(
319         crossAxisAlignment: CrossAxisAlignment.start,
320         children: [
321           Icon(Icons.pets, color: color, size: 28),
322           const SizedBox(width: 14),
323           Expanded(
324             child: Column(
325               crossAxisAlignment: CrossAxisAlignment.start,
326               children: [
327                 Text(
328                   AppStrings.current.livestockTitle(livestock.level),
329                   style: Theme.of(context).textTheme.titleMedium?.copyWith(
330                     color: color,
331                     fontWeight: FontWeight.w700,
332                   ),
333                 ),
334                 const SizedBox(height: 6),
335                 Text(
336                   AppStrings.current.livestockMessage(livestock.level),
337                   style: Theme.of(context).textTheme.bodyMedium,
338                 ),
339               ],
340             ),
341           ),
342         ],
343       ),
344     );
345   }
346 }
347
348 class _AnomalyCard extends StatelessWidget {
349   const _AnomalyCard({required this.anomaly});
350
351   final ClimateAnomaly anomaly;
352
353   @override
354   Widget build(BuildContext context) {
355     return GlassCard(
356       padding: const EdgeInsets.all(18),
357       color: AppColors.deepTeal.withValues(alpha: 0.12),
358       child: Row(
359         crossAxisAlignment: CrossAxisAlignment.start,
360         children: [
361           const Icon(Icons.insights, color: AppColors.deepTeal, size: 28),
362           const SizedBox(width: 14),
363           Expanded(
364             child: Column(
365               crossAxisAlignment: CrossAxisAlignment.start,
366               children: [
367                 Text(
368                   AppStrings.current.anomalyTitle,
369                   style: Theme.of(context).textTheme.titleMedium?.copyWith(
370                     color: AppColors.deepTeal,
371                     fontWeight: FontWeight.w700,
372                   ),
373               ],

```

```

374         const SizedBox(height: 6),
375         Text(
376           AppStrings.current.anomalyMessage(anomaly.delta),
377           style: Theme.of(context).textTheme.bodyMedium,
378         ),
379       ],
380     ),
381   ),
382 ],
383 ),
384 );
385 }
386 }
387
388 class _CurrentRiskCard extends StatelessWidget {
389   const _CurrentRiskCard();
390
391   @override
392   Widget build(BuildContext context) {
393     final textTheme = Theme.of(context).textTheme;
394     final onSurface = Theme.of(context).colorScheme.onSurface;
395     return GlassCard(
396       padding: const EdgeInsets.all(26),
397       child: Column(
398         crossAxisAlignment: CrossAxisAlignment.start,
399         children: [
400           Row(
401             children: [
402               Expanded(
403                 child: Text(
404                   AppStrings.current.currentFrostRisk,
405                   style: textTheme.headlineMedium,
406                 ),
407               const Icon(Icons.ac_unit, color: AppColors.mutedTeal, size: 42),
408             ],
409           ),
410           const SizedBox(height: 34),
411           Row(
412             crossAxisAlignment: CrossAxisAlignment.end,
413             children: [
414               Text(
415                 '-4.2',
416                 style: textTheme.displayLarge?.copyWith(fontSize: 54),
417               ),
418               const SizedBox(width: 10),
419               Padding(
420                 padding: const EdgeInsets.only(bottom: 8),
421                 child: Text(
422                   'C',
423                   style: textTheme.titleMedium?.copyWith(
424                     color: onSurface.withValues(alpha: 0.68),
425                   ),
426                 ),
427             ],
428           ),
429         ],
430       ),
431       const SizedBox(height: 26),
432       Wrap(
433         spacing: 14,
434         runSpacing: 10,
435         children: [
436           StatusChip(
437             icon: Icons.water_drop_outlined,
438             label: AppStrings.current.humShort('42%'),
439           ),
440           const StatusChip(icon: Icons.air, label: '12 km/h'),
441         ],
442       ),
443     ],
444   ),
445 );
446 }
447
448 class _LearningCycleCard extends StatelessWidget {
449   const _LearningCycleCard();
450
451   @override
452   Widget build(BuildContext context) {
453     final textTheme = Theme.of(context).textTheme;
454     final onSurface = Theme.of(context).colorScheme.onSurface;
455     return GlassCard(
456       padding: const EdgeInsets.all(22),
457       child: Column(
458         crossAxisAlignment: CrossAxisAlignment.start,
459         children: [
460           Row(
461             children: [
462               const Icon(Icons.model_training, color: AppColors.warmAmber),
463               const SizedBox(width: 12),
464               Expanded(
465                 child: Text(
466                   AppStrings.current.maintenanceTitle,
467                   style: textTheme.titleMedium,
468                 ),
469               const StatusChip(label: 'K-Means'),
470             ],
471           ),
472           const SizedBox(height: 14),
473           Text(
474             AppStrings.current.maintenanceBody,
475             style: textTheme.bodyMedium?.copyWith(
476               color: onSurface.withValues(alpha: 0.72),
477             ),
478           ),
479         ],
480       ),
481     ],
482   ),
483 );
484 }
485
486 class _QuickTile extends StatelessWidget {
487   const _QuickTile({
488     required this.icon,
489     required this.label,
490     required this.onTap,
491   });
492
493   final IconData icon;

```



```

495 final String label;
496 final VoidCallback onTap;
497
498 @override
499 Widget build(BuildContext context) {
500   return InkWell(
501     borderRadius: BorderRadius.circular(8),
502     onTap: onTap,
503     child: Container(
504       constraints: const BoxConstraints(minHeight: 128),
505       padding: const EdgeInsets.all(22),
506       decoration: BoxDecoration(
507         color: Theme.of(
508           context,
509         ).colorScheme.onSurface.withValues(alpha: 0.06),
510         borderRadius: BorderRadius.circular(8),
511         border: Border.all(
512           color: Theme.of(
513             context,
514           ).colorScheme.onSurface.withValues(alpha: 0.08),
515         ),
516       ),
517       child: Column(
518         crossAxisAlignment: CrossAxisAlignment.start,
519         mainAxisAlignment: MainAxisAlignment.spaceBetween,
520         children: [
521           Icon(
522             icon,
523             color: Theme.of(
524               context,
525             ).colorScheme.onSurface.withValues(alpha: 0.66),
526             size: 32,
527           ),
528           Text(
529             label,
530             maxLines: 2,
531             overflow: TextOverflow.ellipsis,
532             style: Theme.of(
533               context,
534             ).textTheme.titleMedium?.copyWith(fontSize: 17),
535           ),
536         ],
537       ),
538     ),
539   );
540 }
541 }

```

app_flutter/lib/features/location/models/location_state.dart

```

1 | class DetectedLocation {
2 |   const DetectedLocation({
3 |     required this.latitude,
4 |     required this.longitude,
5 |     required this.accuracy,
6 |     this.fromLastKnownPosition = false,
7 |   });
8 |
9 |   final double latitude;
10 |   final double longitude;
11 |   final double accuracy;
12 |   final bool fromLastKnownPosition;
13 | }
14 |
15 | enum LocationFailureType { serviceDisabled, denied, permanentlyDenied, unknown }
16 |
17 | class LocationFailure implements Exception {
18 |   const LocationFailure(this.type, this.message);
19 |
20 |   final LocationFailureType type;
21 |   final String message;
22 |
23 |   @override
24 |   String toString() => message;
25 | }

```

app_flutter/lib/features/location/services/location_service.dart

```

1 | import 'dart:async';
2 |
3 | import 'package:geolocator/geolocator.dart';
4 |
5 | import '../models/location_state.dart';
6 |
7 | class LocationService {
8 |   Future<DetectedLocation> getCurrentLocation() async {
9 |     final serviceEnabled = await Geolocator.isLocationServiceEnabled();
10 |     if (!serviceEnabled) {
11 |       throw const LocationFailure(
12 |         LocationFailureType.serviceDisabled,
13 |         'Activa el GPS del dispositivo para usar tu ubicacion actual.',
14 |       );
15 |     }
16 |
17 |     var permission = await Geolocator.checkPermission();
18 |     if (permission == LocationPermission.denied) {
19 |       permission = await Geolocator.requestPermission();
20 |     }
21 |     if (permission == LocationPermission.deniedForever) {
22 |       throw const LocationFailure(
23 |         LocationFailureType.permanentlyDenied,
24 |         'El permiso de ubicacion esta bloqueado. Activalo desde ajustes del sistema.',
25 |       );
26 |     }
27 |     if (permission == LocationPermission.denied ||
28 |         permission == LocationPermission.unableToDetermine) {
29 |       throw const LocationFailure(
30 |         LocationFailureType.denied,
31 |         'Permiso de ubicacion denegado.',
32 |       );
33 |     }

```

```

34
35     try {
36       final position = await Geolocator.getCurrentPosition(
37         locationSettings: const LocationSettings(
38           accuracy: LocationAccuracy.high,
39           timeLimit: Duration(seconds: 12),
40         ),
41       );
42
43       return DetectedLocation(
44         latitude: position.latitude,
45         longitude: position.longitude,
46         accuracy: position.accuracy,
47       );
48     } on TimeoutException {
49       final lastKnown = await Geolocator.getLastKnownPosition();
50       if (lastKnown == null) {
51         throw const LocationFailure(
52           LocationFailureType.unknown,
53           'El GPS no respondió a tiempo.',
54         );
55       }
56       return DetectedLocation(
57         latitude: lastKnown.latitude,
58         longitude: lastKnown.longitude,
59         accuracy: lastKnown.accuracy,
60         fromLastKnownPosition: true,
61       );
62     }
63   }
64 }

```

app_flutter/lib/features/model_info/models/model_info.dart

```

1| class ModelInfo {
2|   const ModelInfo({
3|     required this.modelName,
4|     required this.version,
5|     required this.createdAt,
6|     required this.features,
7|     required this.target,
8|     required this.metrics,
9|     required this.datasetSize,
10|    required this.dataSources,
11|    required this.limitations,
12|    this.nClusters,
13|  });
14|
15|   final String modelName;
16|   final String version;
17|   final String createdAt;
18|   final List<String> features;
19|   final String target;
20|   final Map<String, double> metrics;
21|   final int datasetSize;
22|   final List<String> dataSources;
23|   final List<String> limitations;
24|   final int? nClusters;
25|
26|   factory ModelInfo.fromJson(Map<String, dynamic> json) {
27|     final rawMetrics = json['metrics'] as Map<String, dynamic>? ?? const {};
28|     return ModelInfo(
29|       modelName: json['model_name'] as String? ?? '-',
30|       version: json['version'] as String? ?? '-',
31|       createdAt: json['created_at'] as String? ?? '-',
32|       features: (json['features'] as List<dynamic>? ?? const [])
33|         .map((item) => item.toString())
34|         .toList(),
35|       target: json['target'] as String? ?? '-',
36|       metrics: rawMetrics.map(
37|         (key, value) => MapEntry(key, (value as num).toDouble()),
38|       ),
39|       datasetSize: json['dataset_size'] as int? ?? 0,
40|       dataSources: (json['data_sources'] as List<dynamic>? ?? const [])
41|         .map((item) => item.toString())
42|         .toList(),
43|       limitations: (json['limitations'] as List<dynamic>? ?? const [])
44|         .map((item) => item.toString())
45|         .toList(),
46|       nClusters: (json['n_clusters'] as num?)?.toInt(),
47|     );
48|   }
49| }

```

app_flutter/lib/features/model_info/screens/model_info_screen.dart

```

1| import 'package:flutter/material.dart';
2|
3| import '../../../core/api/frost_api_service.dart';
4| import '../../../core/i18n/app_strings.dart';
5| import '../../../core/theme/app_colors.dart';
6| import '../../../shared/widgets/glass_card.dart';
7| import '../../../shared/widgets/page_scaffold.dart';
8| import '../../../shared/widgets/section_header.dart';
9| import '../../../shared/widgets/status_chip.dart';
10| import '../../../models/model_info.dart';
11|
12| class ModelInfoScreen extends StatelessWidget {
13|   const ModelInfoScreen({required this.apiService, super.key});
14|
15|   final FrostApiService apiService;
16|
17|   @override
18|   Widget build(BuildContext context) {
19|     return PageScaffold(
20|       child: FutureBuilder<ModelInfo>(
21|         future: apiService.getModelInfo(),
22|         builder: (context, snapshot) {
23|           if (snapshot.connectionState == ConnectionState.waiting) {
24|             return const Center(child: CircularProgressIndicator());
25|           }
26|         },
27|       ),
28|     );
29|   }
30| }

```

```

26      final s = AppStrings.current;
27      if (snapshot.hasError) {
28          return Center(child: Text(s.modelInfoError));
29      }
30      final info = snapshot.data!;
31      final silhouette = info.metrics['silhouette'] ?? 0;
32      return ListView(
33          padding: const EdgeInsets.fromLTRB(24, 28, 24, 32),
34          children: [
35              SectionHeader(
36                  title: s.modelInfoTitle,
37                  subtitle: s.modelInfoSubtitle,
38              ),
39              const SizedBox(height: 30),
40              GlassCard(
41                  color: AppColors.softWhite,
42                  child: Row(
43                      children: [
44                          Expanded(
45                              child: Column(
46                                  crossAxisAlignment: CrossAxisAlignment.start,
47                                  children: [
48                                      _MonoLabel(s.baseArchitecture),
49                                      const SizedBox(height: 12),
50                                      Text(
51                                          info.modelName,
52                                          style: Theme.of(context).textTheme.headlineMedium,
53                                      ),
54                                      const SizedBox(height: 20),
55                                      Wrap(
56                                          spacing: 10,
57                                          runSpacing: 10,
58                                          children: [
59                                              StatusChip(label: s.chipClustering),
60                                              StatusChip(label: s.chipUnsupervised),
61                                              const StatusChip(label: 'Scikit-Learn'),
62                                          ],
63                                      ),
64                                  ],
65                              ),
66                          ),
67                          const CircleAvatar(
68                              radius: 38,
69                              backgroundColor: AppColors.surfaceLow,
70                              child: Icon(
71                                  Icons.account_tree_outlined,
72                                  color: AppColors.deepNavy,
73                                  size: 34,
74                              ),
75                          ),
76                      ],
77                  ),
78              ),
79              const SizedBox(height: 18),
80              _MetricPanel(
81                  icon: Icons.analytics_outlined,
82                  label: s.mainMetric,
83                  value: silhouette.toStringAsFixed(3),
84                  suffix: 'silhouette',
85                  description: s.silhouetteDesc,
86              ),
87              const SizedBox(height: 18),
88              _MetricPanel(
89                  icon: Icons.hub_outlined,
90                  label: s.riskGroups,
91                  value: (info.nClusters ?? 0).toString(),
92                  suffix: s.clustersUnit,
93                  description: s.clustersDesc,
94              ),
95              const SizedBox(height: 18),
96              _MetricPanel(
97                  icon: Icons.dataset_outlined,
98                  label: s.datasetSize,
99                  value: info.datasetSize.toString(),
100                 suffix: s.samples,
101                 description: s.datasetDesc,
102             ),
103             const SizedBox(height: 18),
104             GlassCard(
105                 color: AppColors.softWhite,
106                 child: Column(
107                     crossAxisAlignment: CrossAxisAlignment.start,
108                     children: [
109                         _MonoLabel(s.versionStatus),
110                         const SizedBox(height: 14),
111                         Text(
112                             info.version,
113                             style: const TextStyle(
114                                 fontFamily: 'monospace',
115                                 fontSize: 28,
116                                 fontWeight: FontWeight.w700,
117                             ),
118                         ),
119                         const SizedBox(height: 18),
120                         Container(
121                             padding: const EdgeInsets.all(18),
122                             decoration: BoxDecoration(
123                                 color: AppColors.success.withValues(alpha: 0.10),
124                                 borderRadius: BorderRadius.circular(12),
125                                 border: Border.all(
126                                     color: AppColors.success.withValues(alpha: 0.24),
127                                 ),
128                             ),
129                         child: Row(
130                             children: [
131                                 const Icon(
132                                     Icons.check_circle_outline,
133                                     color: AppColors.success,
134                                 ),
135                                 const SizedBox(width: 12),
136                                 Expanded(
137                                     child: Text(
138                                         s.qualityGateApproved,
139                                         style: const TextStyle(
140                                             color: AppColors.success,
141                                             fontSize: 18,
142                                             fontWeight: FontWeight.w700,
143                                         ),
144                                     ),
145                                 ),
146                             ],
147                         ),
148                     ],
149                 ),
150             ),
151         ],
152     );

```

```

147         ),
148       ),
149     ],
150   ),
151   ),
152   ),
153   ),
154   ),
155   ),
156   );
157 }
158 }
159
160 class _MetricPanel extends StatelessWidget {
161   const _MetricPanel({
162     required this.icon,
163     required this.label,
164     required this.value,
165     required this.suffix,
166     required this.description,
167   });
168
169   final IconData icon;
170   final String label;
171   final String value;
172   final String suffix;
173   final String description;
174
175   @override
176   Widget build(BuildContext context) {
177     return GlassCard(
178       child: Column(
179         crossAxisAlignment: CrossAxisAlignment.start,
180         children: [
181           Row(
182             children: [
183               Icon(icon, color: AppColors.textSecondary),
184               const SizedBox(width: 10),
185               _MonoLabel(label),
186             ],
187           ),
188           const SizedBox(height: 16),
189           RichText(
190             text: TextSpan(
191               style: const TextStyle(
192                 color: AppColors.deepNavy,
193                 fontFamily: 'monospace',
194                 fontSize: 28,
195                 fontWeight: FontWeight.w700,
196               ),
197               children: [
198                 TextSpan(text: value),
199                 TextSpan(
200                   text: ' $suffix',
201                   style: const TextStyle(
202                     fontSize: 16,
203                     fontWeight: FontWeight.w500,
204                   ),
205                 ),
206               ],
207             ),
208           ),
209           const SizedBox(height: 12),
210           const Divider(),
211           Text(
212             description,
213             style: Theme.of(
214               context,
215             ).textTheme.bodyLarge?.copyWith(color: AppColors.textSecondary),
216           ),
217         ],
218       ),
219     );
220   }
221 }
222
223 class _MonoLabel extends StatelessWidget {
224   const _MonoLabel(this.text);
225
226   final String text;
227
228   @override
229   Widget build(BuildContext context) {
230     return Text(
231       text,
232       style: Theme.of(context).textTheme.labelSmall?.copyWith(
233         fontFamily: 'monospace',
234         letterSpacing: 1.4,
235         color: AppColors.textSecondary,
236         fontWeight: FontWeight.w700,
237       ),
238     );
239   }
240 }

```

app_flutter/lib/features/prediction/models/frost_risk_request.dart

```

1 class FrostRiskRequest {
2   const FrostRiskRequest({
3     required this.district,
4     required this.province,
5     required this.populatedCenter,
6     required this.latitude,
7     required this.longitude,
8     required this.altitude,
9     required this.ruralPopulation,
10    required this.agriculturalActivity,
11    required this.mainCrop,
12    required this.temperatureMin,
13    required this.temperatureMax,
14    required this.feelsLike,
15    required this.humidity,
16    required this.windSpeed,
17    required this.cloudCover,
18    required this.dewPoint,
19    required this.precipitation,

```

```

20    required this.month,
21    required this.hour,
22    required this.hoursBelowZero,
23    this.totalPopulation,
24    this.ruralPercentage,
25    });
26
27    final String district;
28    final String province;
29    final String populatedCenter;
30    final double latitude;
31    final double longitude;
32    final double altitude;
33    final int ruralPopulation;
34    final int? totalPopulation;
35    final double? ruralPercentage;
36    final bool agriculturalActivity;
37    final String mainCrop;
38    final double temperatureMin;
39    final double temperatureMax;
40    final double feelsLike;
41    final double humidity;
42    final double windSpeed;
43    final double cloudCover;
44    final double dewPoint;
45    final double precipitation;
46    final int month;
47    final int hour;
48    final int hoursBelowZero;
49
50    factory FrostRiskRequest.demo() {
51      return const FrostRiskRequest(
52        district: 'Puno',
53        province: 'Puno',
54        populatedCenter: 'Centro poblado demo',
55        latitude: -15.8402,
56        longitude: -70.0219,
57        altitude: 3827,
58        ruralPopulation: 1200,
59        totalPopulation: 5000,
60        ruralPercentage: 24,
61        agriculturalActivity: true,
62        mainCrop: 'papa',
63        temperatureMin: -2.5,
64        temperatureMax: 12.4,
65        feelsLike: -4.0,
66        humidity: 68,
67        windSpeed: 7,
68        cloudCover: 20,
69        dewPoint: -3.5,
70        precipitation: 0,
71        month: 6,
72        hour: 3,
73        hoursBelowZero: 4,
74      );
75    }
76
77    FrostRiskRequest copyWith({
78      String? district,
79      String? province,
80      String? populatedCenter,
81      double? latitude,
82      double? longitude,
83      double? altitude,
84      int? ruralPopulation,
85      int? totalPopulation,
86      double? ruralPercentage,
87      bool? agriculturalActivity,
88      String? mainCrop,
89      double? temperatureMin,
90      double? temperatureMax,
91      double? feelsLike,
92      double? humidity,
93      double? windSpeed,
94      double? cloudCover,
95      double? dewPoint,
96      double? precipitation,
97      int? month,
98      int? hour,
99      int? hoursBelowZero,
100    }) {
101      return FrostRiskRequest(
102        district: district ?? this.district,
103        province: province ?? this.province,
104        populatedCenter: populatedCenter ?? this.populatedCenter,
105        latitude: latitude ?? this.latitude,
106        longitude: longitude ?? this.longitude,
107        altitude: altitude ?? this.altitude,
108        ruralPopulation: ruralPopulation ?? this.ruralPopulation,
109        totalPopulation: totalPopulation ?? this.totalPopulation,
110        ruralPercentage: ruralPercentage ?? this.ruralPercentage,
111        agriculturalActivity: agriculturalActivity ?? this.agriculturalActivity,
112        mainCrop: mainCrop ?? this.mainCrop,
113        temperatureMin: temperatureMin ?? this.temperatureMin,
114        temperatureMax: temperatureMax ?? this.temperatureMax,
115        feelsLike: feelsLike ?? this.feelsLike,
116        humidity: humidity ?? this.humidity,
117        windSpeed: windSpeed ?? this.windSpeed,
118        cloudCover: cloudCover ?? this.cloudCover,
119        dewPoint: dewPoint ?? this.dewPoint,
120        precipitation: precipitation ?? this.precipitation,
121        month: month ?? this.month,
122        hour: hour ?? this.hour,
123        hoursBelowZero: hoursBelowZero ?? this.hoursBelowZero,
124      );
125    }
126
127    Map<String, dynamic> toJson() {
128      return {
129        'district': district,
130        'province': province,
131        'populated_center': populatedCenter,
132        'latitude': latitude,
133        'longitude': longitude,
134        'altitude': altitude,
135        'rural_population': ruralPopulation,
136        if (totalPopulation != null) 'total_population': totalPopulation,
137        if (ruralPercentage != null) 'rural_percentage': ruralPercentage,
138        'agricultural_activity': agriculturalActivity,
139        'main_crop': mainCrop,
140        'temperature_min': temperatureMin,

```

```

141|     'temperature_max': temperatureMax,
142|     'feels_like': feelslike,
143|     'humidity': humidity,
144|     'wind_speed': windSpeed,
145|     'cloud_cover': cloudCover,
146|     'dew_point': dewPoint,
147|     'precipitation': precipitation,
148|     'month': month,
149|     'hour': hour,
150|     'hours_below_zero': hoursBelowZero,
151|   });
152| }
153| }

```

app_flutter/lib/features/prediction/models/frost_risk_response.dart

```

1| class FrostRiskResponse {
2|   const FrostRiskResponse({
3|     required this.riskLevel,
4|     required this.confidence,
5|     required this.recommendation,
6|     required this.chunoConditions,
7|     required this.modelVersion,
8|     required this.dataSources,
9|   });
10|
11|   final String riskLevel;
12|   final double confidence;
13|   final String recommendation;
14|   final String chunoConditions;
15|   final String modelVersion;
16|   final List<String> dataSources;
17|
18|   factory FrostRiskResponse.fromJson(Map<String, dynamic> json) {
19|     return FrostRiskResponse(
20|       riskLevel: json['risk_level'] as String? ?? 'bajo',
21|       confidence: (json['confidence'] as num? ?? 0).toDouble(),
22|       recommendation: json['recommendation'] as String? ?? '',
23|       chunoConditions: json['chuno_conditions'] as String? ?? 'no_favorables',
24|       modelVersion: json['model_version'] as String? ?? '-',
25|       dataSources: (json['data_sources'] as List<dynamic>? ?? const [])
26|         .map((item) => item.toString())
27|         .toList(),
28|     );
29|   }
30| }

```

app_flutter/lib/features/prediction/screens/prediction_form_screen.dart

```

1| import 'package:flutter/material.dart';
2|
3| import '../core/api/frost_api_service.dart';
4| import '../core/i18n/app_strings.dart';
5| import '../core/models/current_weather.dart';
6| import '../core/theme/app_colors.dart';
7| import '../shared/widgets/glass_card.dart';
8| import '../shared/widgets/page_scaffold.dart';
9| import '../shared/widgets/primary_action_button.dart';
10| import '../shared/widgets/section_header.dart';
11| import '../shared/widgets/status_chip.dart';
12| import '../location/models/location_state.dart';
13| import '../location/services/location_service.dart';
14| import '../models/frost_risk_request.dart';
15| import 'result_screen.dart';
16|
17| class PredictionFormScreen extends StatefulWidget {
18|   const PredictionFormScreen({required this.apiService, super.key});
19|
20|   final FrostApiService apiService;
21|
22|   @override
23|   State<PredictionFormScreen> createState() => _PredictionFormScreenState();
24| }
25|
26| class _PredictionFormScreenState extends State<PredictionFormScreen> {
27|   final _locationService = LocationService();
28|   FrostRiskRequest _request = FrostRiskRequest.demo();
29|   DetectedLocation? _detectedLocation;
30|   CurrentWeather? _weather;
31|   bool _isLocating = false;
32|   bool _isLoadingWeather = false;
33|   bool _isPredicting = false;
34|   // null => se muestra la pista por defecto en el idioma activo.
35|   String? _locationMessage;
36|   String? _weatherMessage;
37|
38|   @override
39|   Widget build(BuildContext context) {
40|     final isBusy = _isLocating || _isLoadingWeather || _isPredicting;
41|     final s = AppStrings.current;
42|     return PageScaffold(
43|       child: ListView(
44|         padding: const EdgeInsets.fromLTRB(20, 24, 20, 28),
45|         children: [
46|           SectionHeader(
47|             title: s.frostRiskTitle,
48|             subtitle: s.frostRiskSubtitle,
49|           ),
50|           const SizedBox(height: 24),
51|           _LocationCard(
52|             request: _request,
53|             detectedLocation: _detectedLocation,
54|             message: _locationMessage ?? s.locationHint,
55|             locating: _isLocating,
56|             onUseLocation: isBusy ? null : _useCurrentLocation,
57|           ),
58|           const SizedBox(height: 18),
59|           _WeatherCard(
60|             request: _request,
61|             weather: _weather,
62|             message: _weatherMessage ?? s.weatherHint,

```

```

63         loading: _isLoadingWeather,
64         onRefreshWeather: isBusy ? null : _loadWeatherForCurrentRequest,
65     ),
66     const SizedBox(height: 22),
67     _ActionCard(
68       predicting: _isPredicting,
69       canPredict: !isBusy,
70       onPredict: _submit,
71     ),
72   ],
73 ),
74 );
75 }
76
77 Future<void> _submit() async {
78   setState(() => _isPredicting = true);
79   try {
80     final response = await widget.apiService.predict(_request);
81     if (!mounted) return;
82     await Navigator.of(context).push(
83       MaterialPageRoute(
84         builder: (_) => ResultScreen(request: _request, response: response),
85       ),
86     );
87   } catch (error) {
88     if (!mounted) return;
89     ScaffoldMessenger.of(context).showSnackBar(
90       SnackBar(content: Text(AppStrings.current.connectError(error))),
91     );
92   } finally {
93     if (mounted) {
94       setState(() => _isPredicting = false);
95     }
96   }
97 }
98
99 Future<void> _useCurrentLocation() async {
100   final s = AppStrings.current;
101   setState(() {
102     _isLoading = true;
103     _locationMessage = s.requestingGps;
104     _weatherMessage = s.waitingLocation;
105   });
106   try {
107     final location = await _locationService.getCurrentLocation();
108     if (!mounted) return;
109     setState(() {
110       _detectedLocation = location;
111       _request = _request.copyWith(
112         district: 'Ubicacion detectada',
113         province: 'Puno',
114         populatedCenter: 'GPS actual',
115         latitude: location.latitude,
116         longitude: location.longitude,
117       );
118       _locationMessage = s.gpsAccuracy(
119         location.fromLastKnownPosition,
120         location.accuracy.toStringAsFixed(0),
121       );
122     });
123     await _loadCurrentWeather();
124   } on LocationFailure catch (error) {
125     if (!mounted) return;
126     setState(() {
127       _locationMessage = s.locationFailed(error.message);
128       _weatherMessage = s.tapGetWeatherDemo;
129     });
130   } catch (error) {
131     if (!mounted) return;
132     setState(() {
133       _locationMessage = s.locationFailedGeneric;
134       _weatherMessage = s.detail(error);
135     });
136   } finally {
137     if (mounted) {
138       setState(() => _isLoading = false);
139     }
140   }
141 }
142
143 Future<void> _loadWeatherForCurrentRequest() async {
144   await _loadCurrentWeather();
145 }
146
147 Future<void> _loadCurrentWeather() async {
148   final s = AppStrings.current;
149   setState(() {
150     _isLoadingWeather = true;
151     _weatherMessage = s.queryingWeather;
152   });
153   try {
154     final weather = await widget.apiService.getCurrentWeather(
155       latitude: _request.latitude,
156       longitude: _request.longitude,
157     );
158     if (!mounted) return;
159     setState(() {
160       _weather = weather;
161       _request = _requestWithWeather(_request, weather);
162       _weatherMessage = s.weatherUpdated(
163         weather.provider,
164         weather.fallbackUsed,
165       );
166     });
167   } catch (error) {
168     if (!mounted) return;
169     setState(() {
170       _weatherMessage = s.weatherFailed(error);
171     });
172   } finally {
173     if (mounted) {
174       setState(() => _isLoadingWeather = false);
175     }
176   }
177 }
178
179 FrostRiskRequest _requestWithWeather(
180   FrostRiskRequest current,
181   CurrentWeather weather,
182 ) {
183   final now = DateTime.now();

```

```

184    final temperature = weather.temperature;
185    final feelslike = weather.apparentTemperature ?? temperature;
186    final hoursBelowZero = temperature == null
187      ? current.hoursBelowZero
188      : temperature <= 0
189      ? 1
190      : 0;
191
192    return current.copyWith(
193      temperatureMin: temperature ?? current.temperatureMin,
194      temperatureMax: temperature ?? current.temperatureMax,
195      feelslike: feelslike ?? current.feelslike,
196      humidity: weather.humidity ?? current.humidity,
197      windSpeed: weather.windSpeed ?? current.windSpeed,
198      cloudCover: weather.cloudCover ?? current.cloudCover,
199      dewPoint: weather.dewPoint ?? current.dewPoint,
200      precipitation: weather.precipitation ?? current.precipitation,
201      month: now.month,
202      hour: now.hour,
203      hoursBelowZero: hoursBelowZero,
204    );
205  }
206 }
207
208 class _LocationCard extends StatelessWidget {
209   const _LocationCard({
210     required this.request,
211     required this.message,
212     required this.locating,
213     required this.onUseLocation,
214     this.detectedLocation,
215   });
216
217   final FrostRiskRequest request;
218   final DetectedLocation? detectedLocation;
219   final String message;
220   final bool locating;
221   final VoidCallback? onUseLocation;
222
223   @override
224   Widget build(BuildContext context) {
225     final detected = detectedLocation != null;
226     return GlassCard(
227       padding: const EdgeInsets.all(22),
228       child: Column(
229         crossAxisAlignment: CrossAxisAlignment.start,
230         children: [
231           _CardTitle(icon: Icons.my_location, label: AppStrings.current.location),
232           const SizedBox(height: 16),
233           Text(
234             detected
235               ? AppStrings.current.detectedLocation
236               : AppStrings.current.punoDemo,
237             style: Theme.of(context).textTheme.headlineMedium,
238           ),
239           const SizedBox(height: 10),
240           Text(
241             '${request.latitude.toStringAsFixed(5)}, ${request.longitude.toStringAsFixed(5)}',
242             style: Theme.of(context).textTheme.bodyLarge?.copyWith(
243               color: Theme.of(
244                 context,
245               ).colorScheme.onSurface.withValues(alpha: 0.68),
246               fontFamily: 'monospace',
247             ),
248           ),
249           const SizedBox(height: 14),
250           _InfoBanner(icon: Icons.location_on_outlined, text: message),
251           const SizedBox(height: 16),
252           PrimaryActionButton(
253             label: locating
254               ? AppStrings.current.detectingLocation
255               : AppStrings.current.useMyLocation,
256             icon: Icons.gps_fixed,
257             onPressed: onUseLocation,
258           ),
259         ],
260       ),
261     );
262   }
263 }
264
265 class _WeatherCard extends StatelessWidget {
266   const _WeatherCard({
267     required this.request,
268     required this.message,
269     required this.loading,
270     required this.onRefreshWeather,
271     this.weather,
272   });
273
274   final FrostRiskRequest request;
275   final CurrentWeather? weather;
276   final String message;
277   final bool loading;
278   final VoidCallback? onRefreshWeather;
279
280   @override
281   Widget build(BuildContext context) {
282     final s = AppStrings.current;
283     return GlassCard(
284       padding: const EdgeInsets.all(22),
285       child: Column(
286         crossAxisAlignment: CrossAxisAlignment.start,
287         children: [
288           _CardTitle(icon: Icons.cloud_outlined, label: s.currentWeather),
289           const SizedBox(height: 18),
290           _WeatherHero(request: request, weather: weather),
291           const SizedBox(height: 16),
292           Wrap(
293             spacing: 10,
294             runSpacing: 10,
295             children: [
296               _WeatherMetric(
297                 icon: Icons.water_drop_outlined,
298                 label: s.humidity,
299                 value: '${request.humidity.toStringAsFixed(0)} %',
300               ),
301               _WeatherMetric(
302                 icon: Icons.air,
303                 label: s.wind,
304                 value: '${request.windSpeed.toStringAsFixed(1)} km/h',

```



```

305         ),
306         _WeatherMetric(
307             icon: Icons.cloud_queue,
308             label: s.clouds,
309             value: '${request.cloudCover.toStringAsFixed(0)}%',
310         ),
311     ),
312 ),
313 const SizedBox(height: 16),
314 _InfoBanner(icon: Icons.cloud_sync_outlined, text: message),
315 if (weather?.stationName != null) ...[
316     const SizedBox(height: 12),
317     StatusChip(
318         icon: Icons.place_outlined,
319         label: weather!.stationName!,
320     ),
321 ],
322 const SizedBox(height: 16),
323 PrimaryActionButton(
324     label: loading ? s.gettingWeather : s.getWeather,
325     icon: Icons.refresh,
326     onPressed: onRefreshWeather,
327 ),
328 ],
329 ),
330 );
331 }
332 }
333
334 class _ActionCard extends StatelessWidget {
335     const _ActionCard({
336         required this.predicting,
337         required this.canPredict,
338         required this.onPredict,
339     });
340
341     final bool predicting;
342     final bool canPredict;
343     final VoidCallback onPredict;
344
345     @override
346     Widget build(BuildContext context) {
347         final s = AppStrings.current;
348         return GlassCard(
349             padding: const EdgeInsets.all(22),
350             child: Column(
351                 crossAxisAlignment: CrossAxisAlignment.start,
352                 children: [
353                     Text(
354                         s.readyToPredict,
355                         style: Theme.of(context).textTheme.titleMedium,
356                     ),
357                     const SizedBox(height: 8),
358                     Text(
359                         s.readyToPredictBody,
360                         style: Theme.of(context).textTheme.bodyMedium?.copyWith(
361                             color: Theme.of(
362                                 context,
363                             ).colorScheme.onSurface.withValues(alpha: 0.68),
364                         ),
365                     ),
366                     const SizedBox(height: 18),
367                     PrimaryActionButton(
368                         label: predicting ? s.predicting : s.predict,
369                         icon: Icons.analytics_outlined,
370                         onPressed: canPredict ? onPredict : null,
371                     ),
372                 ],
373             ),
374         );
375     }
376 }
377
378 class _WeatherHero extends StatelessWidget {
379     const _WeatherHero({required this.request, this.weather});
380
381     final FrostRiskRequest request;
382     final CurrentWeather? weather;
383
384     @override
385     Widget build(BuildContext context) {
386         final temp = request.temperatureMin;
387         final color = temp <= 0 ? AppColors.warmAmber : AppColors.mutedTeal;
388         return Row(
389             crossAxisAlignment: CrossAxisAlignment.center,
390             children: [
391                 Container(
392                     width: 64,
393                     height: 64,
394                     decoration: BoxDecoration(
395                         color: color.withValues(alpha: 0.14),
396                         borderRadius: BorderRadius.circular(8),
397                     ),
398                     child: Icon(
399                         temp <= 0 ? Icons.ac_unit : Icons.thermostat,
400                         color: color,
401                         size: 34,
402                     ),
403                 ),
404                 const SizedBox(width: 16),
405                 Expanded(
406                     child: Column(
407                         crossAxisAlignment: CrossAxisAlignment.start,
408                         children: [
409                             Text(
410                                 '${temp.toStringAsFixed(1)} C',
411                                 style: Theme.of(
412                                     context,
413                                 ).textTheme.displayLarge?.copyWith(color: color, fontSize: 42),
414                             ),
415                             const SizedBox(height: 4),
416                             Text(
417                                 weather == null
418                                     ? AppStrings.current.internalDemoValue
419                                     : AppStrings.current.feelsLike(
420                                         request.feelsLike.toStringAsFixed(1),
421                                     ),
422                                 style: Theme.of(context).textTheme.bodyMedium,
423                             ),
424                         ],
425                 ),

```

```

426     ),
427   ],
428 );
429 }
430 }
431
432 class _WeatherMetric extends StatelessWidget {
433   const _WeatherMetric({
434     required this.icon,
435     required this.label,
436     required this.value,
437   });
438
439   final IconData icon;
440   final String label;
441   final String value;
442
443   @override
444   Widget build(BuildContext context) {
445     final onSurface = Theme.of(context).colorScheme.onSurface;
446     return Container(
447       constraints: const BoxConstraints(minWidth: 132, minHeight: 74),
448       padding: const EdgeInsets.all(12),
449       decoration: BoxDecoration(
450         color: onSurface.withValues(alpha: 0.06),
451         borderRadius: BorderRadius.circular(8),
452         border: Border.all(color: onSurface.withValues(alpha: 0.08)),
453       ),
454       child: Column(
455         crossAxisAlignment: CrossAxisAlignment.start,
456         mainAxisAlignment: MainAxisAlignment.spaceBetween,
457         children: [
458           Row(
459             children: [
460               Icon(icon, size: 18, color: AppColors.mutedTeal),
461               const SizedBox(width: 6),
462               Text(label, style: Theme.of(context).textTheme.labelSmall),
463             ],
464           ),
465           Text(value, style: Theme.of(context).textTheme.titleMedium),
466         ],
467       ),
468     );
469   }
470 }
471
472 class _CardTitle extends StatelessWidget {
473   const _CardTitle({required this.icon, required this.label});
474
475   final IconData icon;
476   final String label;
477
478   @override
479   Widget build(BuildContext context) {
480     return Row(
481       children: [
482         Icon(icon, color: AppColors.mutedTeal),
483         const SizedBox(width: 10),
484         Text(label, style: Theme.of(context).textTheme.titleMedium),
485       ],
486     );
487   }
488 }
489
490 class _InfoBanner extends StatelessWidget {
491   const _InfoBanner({required this.icon, required this.text});
492
493   final IconData icon;
494   final String text;
495
496   @override
497   Widget build(BuildContext context) {
498     final onSurface = Theme.of(context).colorScheme.onSurface;
499     return Container(
500       padding: const EdgeInsets.all(14),
501       decoration: BoxDecoration(
502         color: onSurface.withValues(alpha: 0.05),
503         borderRadius: BorderRadius.circular(8),
504         border: Border.all(color: onSurface.withValues(alpha: 0.08)),
505       ),
506       child: Row(
507         crossAxisAlignment: CrossAxisAlignment.start,
508         children: [
509           Icon(icon, size: 20, color: AppColors.mutedTeal),
510           const SizedBox(width: 10),
511           Expanded(
512             child: Text(text, style: Theme.of(context).textTheme.bodyMedium),
513           ),
514         ],
515       ),
516     );
517   }
518 }

```

app_flutter/lib/features/prediction/screens/result_screen.dart

```

1 | import 'package:flutter/material.dart';
2 |
3 | import '../../../core/i18n/app_strings.dart';
4 | import '../../../core/theme/app_colors.dart';
5 | import '../../../shared/widgets/glass_card.dart';
6 | import '../../../shared/widgets/page_scaffold.dart';
7 | import '../../../shared/widgets/primary_action_button.dart';
8 | import '../../../shared/widgets/section_header.dart';
9 | import '../../../shared/widgets/status_chip.dart';
10 | import '../../../models/frost_risk_request.dart';
11 | import '../../../models/frost_risk_response.dart';
12 |
13 | class ResultScreen extends StatelessWidget {
14 |   const ResultScreen({
15 |     required this.request,
16 |     required this.response,
17 |     super.key,
18 |   });
19 |
20 |   final FrostRiskRequest request;

```

```

21| final FrostRiskResponse response;
22|
23| @override
24| Widget build(BuildContext context) {
25|   final riskColor = _riskColor(response.riskLevel);
26|   final riskIcon = _riskIcon(response.riskLevel);
27|   final onSurface = Theme.of(context).colorScheme.onSurface;
28|   final s = AppStrings.current;
29|   return PageScaffold(
30|     child: ListView(
31|       padding: const EdgeInsets.fromLTRB(20, 24, 20, 28),
32|       children: [
33|         SectionHeader(
34|           title: s.resultTitle,
35|           subtitle: s.resultsSubtitle,
36|         ),
37|         const SizedBox(height: 24),
38|         GlassCard(
39|           padding: const EdgeInsets.all(24),
40|           child: Column(
41|             crossAxisAlignment: CrossAxisAlignment.start,
42|             children: [
43|               Row(
44|                 children: [
45|                   Expanded(
46|                     child: Column(
47|                       crossAxisAlignment: CrossAxisAlignment.start,
48|                       children: [
49|                         _label(s.riskLabel),
50|                         const SizedBox(height: 8),
51|                         Text(
52|                           s.riskWord(response.riskLevel).toUpperCase(),
53|                           style: Theme.of(context).textTheme.displayLarge
54|                             ?.copyWith(color: riskColor, fontSize: 44),
55|                         ),
56|                       ],
57|                     ),
58|                   Container(
59|                     width: 82,
60|                     height: 82,
61|                     decoration: BoxDecoration(
62|                       color: riskColor.withValues(alpha: 0.14),
63|                       borderRadius: BorderRadius.circular(8),
64|                     ),
65|                     child: Icon(riskIcon, color: riskColor, size: 42),
66|                   ),
67|                 ],
68|               ),
69|               const SizedBox(height: 20),
70|               Container(
71|                 padding: const EdgeInsets.all(16),
72|                 decoration: BoxDecoration(
73|                   color: riskColor.withValues(alpha: 0.12),
74|                   borderRadius: BorderRadius.circular(8),
75|                   border: Border.all(
76|                     color: riskColor.withValues(alpha: 0.24),
77|                   ),
78|                 ),
79|                 child: Row(
80|                   crossAxisAlignment: CrossAxisAlignment.start,
81|                   children: [
82|                     Icon(Icons.tips_and_updates_outlined, color: riskColor),
83|                     const SizedBox(width: 12),
84|                     Expanded(
85|                       child: Text(
86|                         s.recommendation(
87|                           response.riskLevel,
88|                           _shortRecommendation(response.recommendation),
89|                         ),
90|                         style: Theme.of(context).textTheme.bodyLarge,
91|                       ),
92|                     ),
93|                   ],
94|                 ),
95|               ),
96|               const SizedBox(height: 24),
97|               const Divider(),
98|               const SizedBox(height: 18),
99|               Row(
100|                 children: [
101|                   Expanded(
102|                     child: _MetricBlock(
103|                       label: s.confidence,
104|                       value: '${(response.confidence * 100).round()}%',
105|                     ),
106|                   Expanded(
107|                     child: Column(
108|                       crossAxisAlignment: CrossAxisAlignment.start,
109|                       children: [
110|                         _label(s.chunoConditionLabel),
111|                         const SizedBox(height: 8),
112|                         StatusChip(
113|                           icon: Icons.ac_unit,
114|                           label: s.chunoCondition(response.chunoConditions),
115|                           color: onSurface,
116|                           background: onSurface.withValues(alpha: 0.08),
117|                         ),
118|                       ],
119|                     ),
120|                   ),
121|                 ],
122|               ),
123|             ],
124|           ),
125|         ),
126|         const SizedBox(height: 20),
127|         GlassCard(
128|           child: Column(
129|             crossAxisAlignment: CrossAxisAlignment.start,
130|             children: [
131|               _label(s.summary),
132|               const SizedBox(height: 18),
133|               _infoRow(label: s.district, value: request.district),
134|               const Divider(),
135|               _infoRow(label: s.model, value: response.modelVersion),
136|             ],
137|           ),
138|         ),
139|       ],
140|     ),
141|     const SizedBox(height: 28),

```

```

142     PrimaryActionButton(
143       label: s.newQuery,
144       icon: Icons.add,
145       onPressed: () => Navigator.of(context).pop(),
146     ),
147   ],
148 );
149 }
150 }
151
152 static Color _riskColor(String value) {
153   return switch (value.toLowerCase()) {
154     'alto' => AppColors.warmAmber,
155     'medio' => AppColors.mediumRisk,
156     'moderado' => AppColors.mediumRisk,
157     _ => AppColors.lowRisk,
158   };
159 }
160
161 static IconData _riskIcon(String value) {
162   return switch (value.toLowerCase()) {
163     'alto' => Icons.warning_amber_rounded,
164     'medio' => Icons.report_problem_outlined,
165     'moderado' => Icons.report_problem_outlined,
166     _ => Icons.check_circle_outline,
167   };
168 }
169
170 static String _shortRecommendation(String value) {
171   final clean = value.trim();
172   if (clean.isEmpty) {
173     return 'Revisar condiciones locales y monitorear cambios de temperatura.';
174   }
175   final firstSentence = clean.split('.').first.trim();
176   return firstSentence.isEmpty ? clean : '$firstSentence.';
177 }
178
179 }
180
181 class _Label extends StatelessWidget {
182   const _Label(this.text);
183
184   final String text;
185
186   @override
187   Widget build(BuildContext context) {
188     return Text(
189       text,
190       style: Theme.of(context).textTheme.labelSmall?.copyWith(
191         color: Theme.of(context).colorScheme.onSurface.withValues(alpha: 0.64),
192         fontFamily: 'monospace',
193         letterSpacing: 0,
194         fontWeight: FontWeight.w700,
195       ),
196     );
197   }
198 }
199
200 class _MetricBlock extends StatelessWidget {
201   const _MetricBlock({required this.label, required this.value});
202
203   final String label;
204   final String value;
205
206   @override
207   Widget build(BuildContext context) {
208     return Column(
209       crossAxisAlignment: CrossAxisAlignment.start,
210       children: [
211         _Label(label),
212         const SizedBox(height: 8),
213         Text(value, style: Theme.of(context).textTheme.headlineMedium),
214       ],
215     );
216   }
217 }
218
219 class _InfoRow extends StatelessWidget {
220   const _InfoRow({required this.label, required this.value});
221
222   final String label;
223   final String value;
224
225   @override
226   Widget build(BuildContext context) {
227     return Padding(
228       padding: const EdgeInsets.symmetric(vertical: 10),
229       child: Row(
230         crossAxisAlignment: CrossAxisAlignment.start,
231         children: [
232           Expanded(
233             child: Text(label, style: Theme.of(context).textTheme.bodyLarge),
234           ),
235           Flexible(
236             child: Text(
237               value,
238               textAlign: TextAlign.end,
239               style: const TextStyle(
240                 fontFamily: 'monospace',
241                 fontWeight: FontWeight.w700,
242               ),
243             ),
244           ),
245         ],
246       ),
247     );
248   }
249 }

```

app_flutter/lib/features/settings/screens/settings_screen.dart

```

1 import 'package:flutter/material.dart';
2
3 import '../..//core/i18n/app_language.dart';
4 import '../..//core/i18n/app_strings.dart';
5 import '../..//core/settings/user_settings.dart';

```

```

6 import '../shared/widgets/glass_card.dart';
7 import '../shared/widgets/page_scaffold.dart';
8 import '../shared/widgets/section_header.dart';
9
10 class SettingsScreen extends StatelessWidget {
11   const SettingsScreen({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     final settings = UserSettings.instance;
16     return PageScaffold(
17       child: AnimatedBuilder(
18         animation: settings,
19         builder: (context, _) {
20           final s = AppStrings.current;
21           return ListView(
22             padding: const EdgeInsets.fromLTRB(24, 28, 24, 32),
23             children: [
24               SectionHeader(
25                 title: s.settings,
26                 subtitle: s.settingsSubtitle,
27               ),
28               const SizedBox(height: 24),
29               GlassCard(
30                 child: Column(
31                   crossAxisAlignment: CrossAxisAlignment.start,
32                   children: [
33                     SwitchListTile(
34                       contentPadding: EdgeInsets.zero,
35                       title: Text(s.enableAlerts),
36                       subtitle: Text(s.enableAlertsSub),
37                       value: settings.alertsEnabled,
38                       onChanged: settings.setAlertsEnabled,
39                     ),
40                     const Divider(height: 24),
41                     Text(
42                       s.alertThresholdLabel(
43                         settings.alertThreshold.toStringAsFixed(0),
44                       ),
45                       style: Theme.of(context).textTheme.titleMedium,
46                     ),
47                     Slider(
48                       min: -12,
49                       max: 2,
50                       divisions: 14,
51                       label: '${settings.alertThreshold.toStringAsFixed(0)} °C',
52                       value: settings.alertThreshold,
53                       onChanged: settings.alertsEnabled
54                         ? settings.setThreshold
55                         : null,
56                     ),
57                   ],
58                 ),
59               const SizedBox(height: 18),
60               GlassCard(
61                 child: Column(
62                   crossAxisAlignment: CrossAxisAlignment.start,
63                   children: [
64                     Text(s.profile, style: Theme.of(context).textTheme.titleMedium),
65                     const SizedBox(height: 6),
66                     Text(
67                       s.profileHint,
68                       style: Theme.of(context).textTheme.bodyMedium?.copyWith(
69                         color: Theme.of(
70                           context,
71                         ).colorScheme.onSurface.withValues(alpha: 0.66),
72                       ),
73                     ),
74                     const SizedBox(height: 14),
75                     Wrap(
76                       spacing: 10,
77                       runSpacing: 10,
78                       children: UserProfile.values.map((p) {
79                         final selected = settings.profile == p;
80                         return ChoiceChip(
81                           label: Text(s.profileName(p.name)),
82                           selected: selected,
83                           onSelected: (_) => settings.setProfile(p),
84                         );
85                       }).toList(),
86                     ),
87                   ],
88                 ),
89               const SizedBox(height: 18),
90               GlassCard(
91                 child: Column(
92                   crossAxisAlignment: CrossAxisAlignment.start,
93                   children: [
94                     Text(
95                       s.languageLabel,
96                       style: Theme.of(context).textTheme.titleMedium,
97                     ),
98                     const SizedBox(height: 14),
99                     Wrap(
100                      spacing: 10,
101                      runSpacing: 10,
102                      children: AppLanguage.values.map((l) {
103                        return ChoiceChip(
104                          label: Text(l.label),
105                          selected: settings.language == l,
106                          onSelected: (_) => settings.setLanguage(l),
107                        );
108                      }).toList(),
109                     ),
110                   ],
111                 ),
112               ),
113             ],
114           ),
115         );
116       );
117     );
118   );
119 }
120 }
121 }

```

app_flutter/lib/features/shell/app_shell.dart

```

1| import 'package:flutter/material.dart';
2|
3| import '../core/api/frost_api_service.dart';
4| import '../core/i18n/app_strings.dart';
5| import '../charts/screens/charts_screen.dart';
6| import '../chuno/screens/chuno_screen.dart';
7| import '../clusters/screens/clusters_screen.dart';
8| import '../history/screens/history_screen.dart';
9| import '../home/screens/home_screen.dart';
10|
11| class AppShell extends StatefulWidget {
12|   const AppShell({required this.apiService, super.key});
13|
14|   final FrostApiService apiService;
15|
16|   @override
17|   State<AppShell> createState() => _AppShellState();
18| }
19|
20| class _AppShellState extends State<AppShell> {
21|   int _selectedIndex = 0;
22|
23|   @override
24|   Widget build(BuildContext context) {
25|     final colorScheme = Theme.of(context).colorScheme;
26|     final s = AppStrings.current;
27|     final screens = [
28|       HomeScreen(apiService: widget.apiService),
29|       ClustersScreen(apiService: widget.apiService),
30|       ChunoScreen(apiService: widget.apiService),
31|       ChartsScreen(apiService: widget.apiService),
32|       HistoryScreen(apiService: widget.apiService, showAppBar: false),
33|     ];
34|
35|     return Scaffold(
36|       body: screens[_selectedIndex],
37|       bottomNavigationBar: NavigationBar(
38|         selectedIndex: _selectedIndex,
39|         onDestinationSelected: (index) =>
40|           setState(() => _selectedIndex = index),
41|         backgroundColor: colorScheme.surface.withValues(alpha: 0.96),
42|         indicatorColor: colorScheme.primary,
43|         labelTextStyle: WidgetStateProperty.resolveWith(
44|           (states) => TextStyle(
45|             fontFamily: 'monospace',
46|             fontSize: 12,
47|             fontWeight: FontWeight.w600,
48|             color: states.contains(WidgetState.selected)
49|               ? colorScheme.onPrimary
50|               : colorScheme.onSurface.withValues(alpha: 0.68),
51|           ),
52|         ),
53|         destinations: [
54|           NavigationDestination(
55|             icon: const Icon(Icons.home_outlined),
56|             selectedIcon: const Icon(Icons.home),
57|             label: s.tabHome,
58|           ),
59|           NavigationDestination(
60|             icon: const Icon(Icons.map_outlined),
61|             selectedIcon: const Icon(Icons.map),
62|             label: s.tabZones,
63|           ),
64|           NavigationDestination(icon: const Icon(Icons.ac_unit), label: s.tabChuno),
65|           NavigationDestination(icon: const Icon(Icons.show_chart), label: s.tabWeather),
66|           NavigationDestination(icon: const Icon(Icons.history), label: s.tabHistory),
67|         ],
68|       ),
69|     );
70|   }
71| }

```

app_flutter/lib/main.dart

```

1| import 'package:flutter/material.dart';
2|
3| import 'app.dart';
4| import 'core/notifications/notification_service.dart';
5| import 'core/settings/user_settings.dart';
6|
7| Future<void> main() async {
8|   WidgetsFlutterBinding.ensureInitialized();
9|   await UserSettings.instance.load();
10|  await NotificationService.init();
11|  runApp(const FrostPunoApp());
12| }

```

app_flutter/lib/shared/widgets/frost_app_bar.dart

```

1| import 'package:flutter/material.dart';
2|
3| import '../core/theme/theme_mode_scope.dart';
4|
5| class FrostAppBar extends StatelessWidget implements PreferredSizeWidget {
6|   const FrostAppBar({super.key, this.canPop = false});
7|
8|   final bool canPop;
9|
10|  @override
11|  Size get preferredSize => const Size.fromHeight(64);
12|
13|  @override
14|  Widget build(BuildContext context) {
15|    final themeScope = ThemeModeScope.maybeOf(context);
16|    final mode = themeScope?.themeMode ?? ThemeMode.system;
17|    return AppBar(
18|      leading: canPop

```

```

19      ? IconButton(
20        icon: const Icon(Icons.arrow_back),
21        onPressed: () => Navigator.of(context).maybePop(),
22      )
23      : null,
24      title: const Text('FrostPuno'),
25      actions: [
26        IconButton(
27          tooltip: _nextThemeLabel(mode),
28          icon: Icon(_themeIcon(mode)),
29          onPressed: themeScope == null
30            ? null
31            : () => themeScope.onThemeModeChanged(_nextThemeMode(mode)),
32        ),
33      ],
34    );
35  }
36
37  static ThemeMode _nextThemeMode(ThemeMode mode) {
38    return switch (mode) {
39      ThemeMode.system => ThemeMode.light,
40      ThemeMode.light => ThemeMode.dark,
41      ThemeMode.dark => ThemeMode.system,
42    };
43  }
44
45  static IconData _themeIcon(ThemeMode mode) {
46    return switch (mode) {
47      ThemeMode.system => Icons.brightness_auto_outlined,
48      ThemeMode.light => Icons.light_mode_outlined,
49      ThemeMode.dark => Icons.dark_mode_outlined,
50    };
51  }
52
53  static String _nextThemeLabel(ThemeMode mode) {
54    return switch (mode) {
55      ThemeMode.system => 'Usar modo claro',
56      ThemeMode.light => 'Usar modo oscuro',
57      ThemeMode.dark => 'Usar modo del sistema',
58    };
59  }
60 }

```

app_flutter/lib/shared/widgets/glass_card.dart

```

1  import 'package:flutter/material.dart';
2
3  import '../core/theme/app_colors.dart';
4
5  class GlassCard extends StatelessWidget {
6    const GlassCard({
7      required this.child,
8      super.key,
9      this.padding = const EdgeInsets.all(20),
10     this.color,
11   });
12
13   final Widget child;
14   final EdgeInsetsGeometry padding;
15   final Color? color;
16
17   @override
18   Widget build(BuildContext context) {
19     final colorScheme = Theme.of(context).colorScheme;
20     final isDark = colorScheme.brightness == Brightness.dark;
21     return Container(
22       width: double.infinity,
23       padding: padding,
24       decoration: BoxDecoration(
25         color:
26           color ??
27           (isDark
28             ? AppColors.darkSurfaceHigh.withValues(alpha: 0.84)
29             : AppColors.iceBlue.withValues(alpha: 0.45)),
30         borderRadius: BorderRadius.circular(8),
31         border: Border.all(
32           color: isDark ? AppColors.darkOutline : AppColors.outline,
33         ),
34         boxShadow: [
35           BoxShadow(
36             color: isDark
37               ? Colors.black.withValues(alpha: 0.14)
38               : AppColors.deepNavy.withValues(alpha: 0.04),
39             blurRadius: isDark ? 18 : 24,
40             offset: const Offset(0, 8),
41           ),
42         ],
43       ),
44       child: child,
45     );
46   }
47 }

```

app_flutter/lib/shared/widgets/page_scaffold.dart

```

1  import 'package:flutter/material.dart';
2
3  import 'frost_app_bar.dart';
4  import 'responsive_content.dart';
5
6  class PageScaffold extends StatelessWidget {
7    const PageScaffold({
8      required this.child,
9      super.key,
10     this.canPop = true,
11     this.bottomNavigationBar,
12   });
13
14   final Widget child;
15   final bool canPop;
16   final Widget? bottomNavigationBar;

```

```

17 |
18 | @override
19 | Widget build(BuildContext context) {
20 |   return Scaffold(
21 |     appBar: FrostAppBar(canPop: canPop),
22 |     bottomNavigationBar: bottomNavigationBar,
23 |     body: SafeArea(child: ResponsiveContent(child: child)),
24 |   );
25 | }
26 | }

```

app_flutter/lib/shared/widgets/primary_action_button.dart

```

1 | import 'package:flutter/material.dart';
2 |
3 | class PrimaryActionButton extends StatelessWidget {
4 |   const PrimaryActionButton({
5 |     required this.label,
6 |     required this.onPressed,
7 |     super.key,
8 |     this.icon,
9 |     this.expanded = true,
10 |  });
11 |
12 |   final String label;
13 |   final VoidCallback? onPressed;
14 |   final IconData? icon;
15 |   final bool expanded;
16 |
17 |   @override
18 |   Widget build(BuildContext context) {
19 |     final button = FilledButton.icon(
20 |       onPressed: onPressed,
21 |       icon: Icon(icon ?? Icons.arrow_forward),
22 |       label: Text(label),
23 |       style: FilledButton.styleFrom(
24 |         minimumSize: const Size(0, 60),
25 |         shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
26 |         textStyle: const TextStyle(fontSize: 18, fontWeight: FontWeight.w700),
27 |       ),
28 |     );
29 |     return expanded ? SizedBox(width: double.infinity, child: button) : button;
30 |   }
31 | }

```

app_flutter/lib/shared/widgets/responsive_content.dart

```

1 | import 'package:flutter/material.dart';
2 |
3 | class ResponsiveContent extends StatelessWidget {
4 |   const ResponsiveContent({
5 |     required this.child,
6 |     super.key,
7 |     this.maxWidth = 720,
8 |   });
9 |
10 |   final Widget child;
11 |   final double maxWidth;
12 |
13 |   @override
14 |   Widget build(BuildContext context) {
15 |     return LayoutBuilder(
16 |       builder: (context, constraints) {
17 |         final viewportWidth = MediaQuery.sizeOf(context).width;
18 |         final availableWidth = constraints.maxWidth.isFinite
19 |           ? constraints.maxWidth
20 |           : viewportWidth;
21 |         final width = availableWidth > maxWidth ? maxWidth : availableWidth;
22 |         return Align(
23 |           alignment: Alignment.topCenter,
24 |           child: SizedBox(width: width, child: child),
25 |         );
26 |       },
27 |     );
28 |   }
29 | }

```

app_flutter/lib/shared/widgets/section_header.dart

```

1 | import 'package:flutter/material.dart';
2 |
3 | class SectionHeader extends StatelessWidget {
4 |   const SectionHeader({required this.title, required this.subtitle, super.key});
5 |
6 |   final String title;
7 |   final String subtitle;
8 |
9 |   @override
10 |   Widget build(BuildContext context) {
11 |     final textTheme = Theme.of(context).textTheme;
12 |     final colorScheme = Theme.of(context).colorScheme;
13 |     return Column(
14 |       crossAxisAlignment: CrossAxisAlignment.start,
15 |       children: [
16 |         Text(title, style: textTheme.displayLarge),
17 |         const SizedBox(height: 12),
18 |         Text(
19 |           subtitle,
20 |           style: textTheme.bodyLarge?.copyWith(
21 |             color: colorScheme.onSurface.withValues(alpha: 0.68),
22 |           ),
23 |         ),
24 |       ],
25 |     );
26 |   }
27 | }

```


app_flutter/lib/shared/widgets/status_chip.dart

```

1| import 'package:flutter/material.dart';
2|
3| import '../core/theme/app_colors.dart';
4|
5| class StatusChip extends StatelessWidget {
6|   const StatusChip({
7|     required this.label,
8|     super.key,
9|     this.icon,
10|    this.color = AppColors.mutedTeal,
11|    this.background,
12|  });
13|
14|   final String label;
15|   final IconData? icon;
16|   final Color color;
17|   final Color? background;
18|
19|   @override
20|   Widget build(BuildContext context) {
21|     final onSurface = Theme.of(context).colorScheme.onSurface;
22|     return Container(
23|       padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 7),
24|       decoration: BoxDecoration(
25|         color: background ?? color.withValues(alpha: 0.10),
26|         borderRadius: BorderRadius.circular(999),
27|         border: Border.all(color: color.withValues(alpha: 0.18)),
28|       ),
29|       child: Row(
30|         mainAxisAlignment: MainAxisAlignment.min,
31|         children: [
32|           if (icon != null) ...[
33|             Icon(icon, size: 16, color: color),
34|             const SizedBox(width: 6),
35|           ],
36|           Text(
37|             label,
38|             style: TextStyle(
39|               color: color == AppColors.warmAmber ? onSurface : color,
40|               fontFamily: 'monospace',
41|               fontSize: 12,
42|               fontWeight: FontWeight.w600,
43|               letterSpacing: 0,
44|             ),
45|           ),
46|         ],
47|       ),
48|     );
49|   }
50| }

```

4. Integracion continua

.github/workflows/backend-tests.yml

```

1| name: Backend Tests
2|
3| on:
4|   push:
5|   pull_request:
6|
7| jobs:
8|   backend-tests:
9|     runs-on: ubuntu-latest
10|     env:
11|       ENABLE_SUPABASE: "false"
12|       PYTHONPATH: ${GITHUB_WORKSPACE}/backend_fastapi
13|
14|     steps:
15|       - name: Checkout repository
16|         uses: actions/checkout@v4
17|
18|       - name: Set up Python
19|         uses: actions/setup-python@v5
20|         with:
21|           python-version: "3.11"
22|           cache: "pip"
23|           cache-dependency-path: |
24|             backend_fastapi/requirements.txt
25|             backend_fastapi/requirements-dev.txt
26|             ml_pipeline/requirements.txt
27|
28|       - name: Install dependencies
29|         run: |
30|           python -m pip install --upgrade pip
31|           pip install -r backend_fastapi/requirements-dev.txt
32|           pip install -r ml_pipeline/requirements.txt
33|
34|       - name: Run backend tests
35|         run: pytest backend_fastapi/tests -q

```

.github/workflows/data-validation.yml

```

1| name: Data Validation
2|
3| on:
4|   push:
5|   pull_request:
6|   workflow_dispatch:
7|
8| jobs:
9|   data-validation:
10|     runs-on: ubuntu-latest
11|
12|     steps:

```

```

13 - name: Checkout repository
14   uses: actions/checkout@v4
15
16 - name: Set up Python
17   uses: actions/setup-python@v5
18   with:
19     python-version: "3.11"
20     cache: "pip"
21     cache-dependency-path: ml_pipeline/requirements.txt
22
23 - name: Install ML dependencies
24   run: |
25     python -m pip install --upgrade pip
26     pip install -r ml_pipeline/requirements.txt
27
28 - name: Validate and process locations
29   run: python -m ml_pipeline.data_ingestion.ingest_locations
30
31 - name: Build demo weather dataset
32   run: >
33     python -m ml_pipeline.data_ingestion.ingest_weather_open_meteo
34     --start-date 2024-06-01
35     --end-date 2024-06-03
36     --limit 3
37     --max-workers 3
38
39 - name: Build features
40   run: python -m ml_pipeline.features.build_features
41
42 - name: Validate training dataset
43   run: python -m ml_pipeline.preprocessing.validate_dataset
44
45 - name: Validate district clustering output
46   run: >
47     python -c "import pandas as pd;
48     df = pd.read_csv('data/processed/district_clusters.csv');
49     required = {'distrito', 'tier', 'dominant_cluster', 'cold_share'};
50     assert required.issubset(df.columns), f'missing columns: {required - set(df.columns)}';
51     assert set(df['tier']).issubset({'alto', 'medio', 'bajo'}), 'unexpected tier values';
52     print(f'district_clusters.csv OK: {len(df)} districts')"
```

.github/workflows/flutter-build.yml

```

1 | name: Flutter Build
2 |
3 | on:
4 |   push:
5 |   pull_request:
6 |   workflow_dispatch:
7 |
8 | jobs:
9 |   flutter-build:
10 |     runs-on: ubuntu-latest
11 |     defaults:
12 |       run:
13 |         working-directory: app_flutter
14 |
15 |     steps:
16 |       - name: Checkout repository
17 |         uses: actions/checkout@v4
18 |
19 |       - name: Set up Flutter
20 |         uses: subosito/flutter-action@v2
21 |         with:
22 |           channel: stable
23 |           cache: true
24 |
25 |       - name: Install dependencies
26 |         run: flutter pub get
27 |
28 |       - name: Analyze Flutter code
29 |         run: flutter analyze
30 |
31 |       - name: Run Flutter tests
32 |         run: flutter test
33 |
34 |       - name: Build Flutter Web
35 |         run: flutter build web --release --dart-define=API_BASE_URL=http://127.0.0.1:8000
```

.github/workflows/ml-training.yml

```

1 | name: ML Training
2 |
3 | on:
4 |   workflow_dispatch:
5 |     inputs:
6 |       start_date:
7 |         description: "Open-Meteo historical start date"
8 |         required: false
9 |         default: "2024-06-01"
10 |       end_date:
11 |         description: "Open-Meteo historical end date"
12 |         required: false
13 |         default: "2024-06-07"
14 |       location_limit:
15 |         description: "Number of locations for demo training. Use empty for all curated locations."
16 |         required: false
17 |         default: "5"
18 |       model_version:
19 |         description: "Model version label"
20 |         required: false
21 |         default: "ci-demo"
22 |     schedule:
23 |       - cron: "0 9 * * 1"
24 |
25 | jobs:
26 |   train-model:
27 |     runs-on: ubuntu-latest
28 |     env:
29 |       START_DATE: ${ github.event_name == 'workflow_dispatch' && github.event.inputs.start_date || '2024-06-01' }
30 |       END_DATE: ${ github.event_name == 'workflow_dispatch' && github.event.inputs.end_date || '2024-06-07' }
```

```

31| LOCATION_LIMIT: ${ github.event_name == 'workflow_dispatch' && github.event.inputs.location_limit || '5' }
32| MODEL_VERSION: ${ github.event_name == 'workflow_dispatch' && github.event.inputs.model_version || 'scheduled-demo' }
33|
34| steps:
35|   - name: Checkout repository
36|     uses: actions/checkout@v4
37|
38|   - name: Set up Python
39|     uses: actions/setup-python@v5
40|     with:
41|       python-version: "3.11"
42|       cache: "pip"
43|       cache-dependency-path: ml_pipeline/requirements.txt
44|
45|   - name: Install ML dependencies
46|     run: |
47|       python -m pip install --upgrade pip
48|       pip install -r ml_pipeline/requirements.txt
49|
50|   - name: Ingest locations
51|     run: python -m ml_pipeline.data_ingestion.ingest_locations
52|
53|   - name: Ingest Open-Meteo demo weather
54|     run: |
55|       if [ -n "$LOCATION_LIMIT" ]; then
56|         python -m ml_pipeline.data_ingestion.ingest_weather_open_meteo \
57|           --start-date "$START_DATE" \
58|           --end-date "$END_DATE" \
59|           --limit "$LOCATION_LIMIT" \
60|           --max-workers 4
61|       else
62|         python -m ml_pipeline.data_ingestion.ingest_weather_open_meteo \
63|           --start-date "$START_DATE" \
64|           --end-date "$END_DATE" \
65|           --max-workers 4
66|       fi
67|
68|   - name: Build features
69|     run: python -m ml_pipeline.features.build_features
70|
71|   - name: Validate dataset
72|     run: python -m ml_pipeline.preprocessing.validate_dataset
73|
74|   # Corrida manual: entrenamiento simple sobre la ventana indicada.
75|   - name: Train unsupervised K-Means model
76|     if: github.event_name != 'schedule'
77|     run: python -m ml_pipeline.clustering.train_clusters --version "$MODEL_VERSION"
78|
79|   # Mantenimiento programado: reentrenamiento ADAPTATIVO. Si la silhouette no
80|   # alcanza el objetivo, amplía la ventana de datos y reintentó; registra cada
81|   # corrida en training_history.json.
82|   - name: Adaptive retraining (scheduled maintenance)
83|     if: github.event_name == 'schedule'
84|     run: >
85|       python -m ml_pipeline.clustering.adaptive_retrain
86|       --target 0.45 --max-iters 3 --initial-days 14 --step-days 15 --max-workers 4
87|
88|   - name: Cluster quality gate (silhouette)
89|     run: >
90|       python -m ml_pipeline.registry.check_cluster_quality
91|       --metadata ml_pipeline/registry/cluster_metadata.json
92|
93|   - name: Upload model artifacts
94|     uses: actions/upload-artifact@v4
95|     with:
96|       name: frost-puno-model-${ github.run_id }
97|       retention-days: 14
98|       path: |
99|         ml_pipeline/registry/frost_cluster_model.joblib
100|         ml_pipeline/registry/cluster_metadata.json
101|         ml_pipeline/registry/training_history.json
102|         ml_pipeline/evaluation/cluster_profiles.csv
103|         data/processed/district_clusters.csv
104|         data/processed/frost_training_dataset.csv

```

.github/workflows/model-quality-gate.yml

```

1| name: Model Quality Gate
2|
3| on:
4|   push:
5|   pull_request:
6|   workflow_dispatch:
7|     inputs:
8|       min_silhouette:
9|         description: "Minimum accepted silhouette score"
10|        required: false
11|        default: "0.35"
12|
13| jobs:
14|   model-quality-gate:
15|     runs-on: ubuntu-latest
16|     env:
17|       MIN_SILHOUETTE: ${ github.event_name == 'workflow_dispatch' && github.event.inputs.min_silhouette || '0.35' }
18|
19|     steps:
20|       - name: Checkout repository
21|         uses: actions/checkout@v4
22|
23|       - name: Set up Python
24|         uses: actions/setup-python@v5
25|         with:
26|           python-version: "3.11"
27|
28|       - name: Install ML dependencies
29|         run: |
30|           python -m pip install --upgrade pip
31|           pip install -r ml_pipeline/requirements.txt pytest
32|
33|       - name: Evaluate cluster metadata quality (silhouette)
34|         run: >
35|           python -m ml_pipeline.registry.check_cluster_quality
36|           --metadata ml_pipeline/registry/cluster_metadata.json
37|
38|       # Pruebas de funcionamiento del mantenimiento e IC: verifican artefactos del
39|       # reentrenamiento, hiperparámetros optimizados y que el gate BLOQUEA un modelo

```

```

40 | # degradado.
41 | - name: Maintenance & CI functional tests
42 |   run: python -m pytest ml_pipeline/tests/test_maintenance_ci.py -v
43 |
44 | - name: Future active-model comparison
45 |   run: |
46 |     echo "Future step: download the currently active production cluster metadata,"
47 |     echo "compare silhouette / Davies-Bouldin drift, then block deployment on regression."

```

5. Configuración de despliegue

render.yaml

```

1 | services:
2 |   - type: web
3 |     name: frost-puno-api
4 |     runtime: python
5 |     plan: free
6 |     region: oregon
7 |     rootDir: .
8 |     buildCommand: pip install --upgrade pip && pip install -r backend_fastapi/requirements.txt
9 |     startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT --app-dir backend_fastapi
10 |    healthCheckPath: /health
11 |    autoDeploy: true
12 |    envVars:
13 |      - key: PYTHON_VERSION
14 |        value: 3.11.9
15 |      - key: ENABLE_SUPABASE
16 |        value: true
17 |      - key: CORS_ORIGINS
18 |        value: https://frost-puno.vercel.app,http://localhost:3000,http://127.0.0.1:5174,http://localhost:5174
19 |      - key: MODEL_PATH
20 |        value: ml_pipeline/registry/frost_cluster_model.joblib
21 |      - key: MODEL_METADATA_PATH
22 |        value: ml_pipeline/registry/cluster_metadata.json
23 |      - key: DISTRICT_CLUSTERS_PATH
24 |        value: data/processed/district_clusters.csv
25 |      - key: SUPABASE_URL
26 |        value: https://quwfojxuebxfhimmvbaa.supabase.co
27 |      - key: SUPABASE_SERVICE_ROLE_KEY
28 |    sync: false

```

supabase/migrations/001_create_core_tables.sql

```

1 | create extension if not exists pgcrypto;
2 |
3 | create or replace function public.set_updated_at()
4 | returns trigger
5 | language plpgsql
6 | as $$
7 | begin
8 |   new.updated_at = now();
9 |   return new;
10 | end;
11 | $$;
12 |
13 | create table if not exists public.users_profile (
14 |   id uuid primary key default gen_random_uuid(),
15 |   user_id uuid unique references auth.users(id) on delete cascade,
16 |   full_name text,
17 |   role text not null default 'producer' check (role in ('producer', 'student', 'admin', 'researcher')),
18 |   department text not null default 'Puno',
19 |   province text,
20 |   district text,
21 |   community text,
22 |   source text not null default 'app',
23 |   metadata jsonb not null default '{} '::jsonb,
24 |   created_at timestampz not null default now(),
25 |   updated_at timestampz not null default now()
26 | );
27 |
28 | create table if not exists public.inei_locations (
29 |   id uuid primary key default gen_random_uuid(),
30 |   ubigeo text not null unique check (ubigeo ~ '^[0-9]{6}$'),
31 |   department text not null,
32 |   province text not null,
33 |   district text not null,
34 |   latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
35 |   longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
36 |   estimated_altitude numeric(8,2) check (estimated_altitude between 0 and 7000),
37 |   total_population integer check (total_population >= 0),
38 |   rural_population integer check (rural_population >= 0),
39 |   rural_percentage numeric(5,2) check (rural_percentage between 0 and 100),
40 |   source text not null default 'INEI',
41 |   metadata jsonb not null default '{} '::jsonb,
42 |   created_at timestampz not null default now(),
43 |   updated_at timestampz not null default now(),
44 |   constraint rural_population_lte_total check (
45 |     total_population is null
46 |     or rural_population is null
47 |     or rural_population <= total_population
48 |   )
49 | );
50 |
51 | create table if not exists public.populated_centers (
52 |   id uuid primary key default gen_random_uuid(),
53 |   location_id uuid references public.inei_locations(id) on delete set null,
54 |   ubigeo text references public.inei_locations(ubigeo) on update cascade on delete set null,
55 |   name text not null,
56 |   category text,
57 |   latitude numeric(9,6) check (latitude between -18.5 and -13.0),
58 |   longitude numeric(9,6) check (longitude between -71.5 and -68.0),
59 |   altitude numeric(8,2) check (altitude between 0 and 7000),
60 |   population integer check (population >= 0),
61 |   source text not null default 'INEI Sistema de Centros Poblados',
62 |   metadata jsonb not null default '{} '::jsonb,
63 |   created_at timestampz not null default now(),
64 |   updated_at timestampz not null default now()

```

```

65 );
66
67 create table if not exists public.agricultural_context (
68   id uuid primary key default gen_random_uuid(),
69   location_id uuid references public.inei_locations(id) on delete cascade,
70   ubigeo text references public.inei_locations(ubigeo) on update cascade on delete cascade,
71   crop text not null,
72   agricultural_activity boolean not null default true,
73   cultivated_area_ha numeric(12,2) check (cultivated_area_ha is null or cultivated_area_ha >= 0),
74   production_tons numeric(12,2) check (production_tons is null or production_tons >= 0),
75   yield_ton_ha numeric(12,4) check (yield_ton_ha is null or yield_ton_ha >= 0),
76   reference_year integer check (reference_year between 1900 and 2100),
77   source text not null default 'INEI/MIDAGRI-SIEA',
78   metadata jsonb not null default '{}':jsonb,
79   created_at timestampz not null default now(),
80   updated_at timestampz not null default now()
81 );
82
83 create table if not exists public.weather_records (
84   id uuid primary key default gen_random_uuid(),
85   location_id uuid references public.inei_locations(id) on delete set null,
86   ubigeo text references public.inei_locations(ubigeo) on update cascade on delete set null,
87   observed_at timestampz not null,
88   latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
89   longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
90   temperature_2m numeric(6,2),
91   relative_humidity_2m numeric(5,2) check (relative_humidity_2m between 0 and 100),
92   apparent_temperature numeric(6,2),
93   dew_point_2m numeric(6,2),
94   precipitation numeric(8,2) check (precipitation is null or precipitation >= 0),
95   cloud_cover numeric(5,2) check (cloud_cover between 0 and 100),
96   wind_speed_10m numeric(7,2) check (wind_speed_10m is null or wind_speed_10m >= 0),
97   source text not null default 'Open-Meteo',
98   metadata jsonb not null default '{}':jsonb,
99   created_at timestampz not null default now()
100 );
101
102 create table if not exists public.model_versions (
103   id uuid primary key default gen_random_uuid(),
104   version text not null unique,
105   model_name text not null,
106   algorithm text not null,
107   artifact_path text not null,
108   accuracy numeric(8,6) check (accuracy between 0 and 1),
109   precision_macro numeric(8,6) check (precision_macro between 0 and 1),
110   recall_macro numeric(8,6) check (recall_macro between 0 and 1),
111   f1_macro numeric(8,6) check (f1_macro between 0 and 1),
112   dataset_size integer not null check (dataset_size >= 0),
113   status text not null default 'candidate' check (status in ('candidate', 'active', 'rejected', 'archived')),
114   source text not null default 'ml_pipeline',
115   metadata jsonb not null default '{}':jsonb,
116   created_at timestampz not null default now(),
117   deployed_at timestampz
118 );
119
120 create table if not exists public.training_runs (
121   id uuid primary key default gen_random_uuid(),
122   model_version_id uuid references public.model_versions(id) on delete set null,
123   run_name text,
124   dataset_path text,
125   dataset_hash text,
126   git_commit text,
127   accuracy numeric(8,6) check (accuracy between 0 and 1),
128   precision_macro numeric(8,6) check (precision_macro between 0 and 1),
129   recall_macro numeric(8,6) check (recall_macro between 0 and 1),
130   f1_macro numeric(8,6) check (f1_macro between 0 and 1),
131   status text not null default 'completed' check (status in ('started', 'completed', 'failed', 'promoted', 'rejected')),
132   source text not null default 'github_actions_or_local',
133   metadata jsonb not null default '{}':jsonb,
134   created_at timestampz not null default now(),
135   updated_at timestampz not null default now()
136 );
137
138 create table if not exists public.frost_predictions (
139   id uuid primary key default gen_random_uuid(),
140   user_id uuid references auth.users(id) on delete set null,
141   location_id uuid references public.inei_locations(id) on delete set null,
142   model_version_id uuid references public.model_versions(id) on delete set null,
143   district text not null,
144   province text not null,
145   populated_center text,
146   latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
147   longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
148   altitude numeric(8,2) check (altitude between 0 and 7000),
149   input_payload jsonb not null,
150   risk_level text not null check (risk_level in ('bajo', 'medio', 'alto')),
151   confidence numeric(6,5) not null check (confidence between 0 and 1),
152   chuno_conditions text not null check (chuno_conditions in ('favorables', 'posibles', 'no_favorables')),
153   recommendation text not null,
154   model_version text not null,
155   data_sources jsonb not null default '[]':jsonb,
156   source text not null default 'backend_fastapi',
157   metadata jsonb not null default '{}':jsonb,
158   created_at timestampz not null default now()
159 );
160
161 create table if not exists public.alerts (
162   id uuid primary key default gen_random_uuid(),
163   location_id uuid references public.inei_locations(id) on delete set null,
164   prediction_id uuid references public.frost_predictions(id) on delete set null,
165   title text not null,
166   message text not null,
167   risk_level text not null check (risk_level in ('bajo', 'medio', 'alto')),
168   channel text not null default 'app' check (channel in ('app', 'email', 'push', 'sms', 'whatsapp')),
169   status text not null default 'draft' check (status in ('draft', 'sent', 'cancelled', 'failed')),
170   starts_at timestampz,
171   ends_at timestampz,
172   source text not null default 'backend_fastapi',
173   metadata jsonb not null default '{}':jsonb,
174   created_at timestampz not null default now(),
175   updated_at timestampz not null default now()
176 );
177
178 create table if not exists public.data_sources_log (
179   id uuid primary key default gen_random_uuid(),
180   source_name text not null,
181   source_type text not null check (source_type in ('INEI', 'SENAMHI', 'Open-Meteo', 'MIDAGRI-SIEA', 'manual', 'other')),
182   resource_url text,
183   file_path text,
184   rows_processed integer check (rows_processed is null or rows_processed >= 0),
185   status text not null default 'completed' check (status in ('started', 'completed', 'failed')),

```

```

186 checksum text,
187 error_message text,
188 source text not null default 'pipeline',
189 metadata jsonb not null default '{}':jsonb,
190 created_at timestampz not null default now()
191 );
192
193 create index if not exists idx_inei_locations_ubigeo on public.inei_locations(ubigeo);
194 create index if not exists idx_populated_centers_ubigeo on public.populated_centers(ubigeo);
195 create index if not exists idx_agricultural_context_ubigeo_crop on public.agricultural_context(ubigeo, crop);
196 create index if not exists idx_weather_records_ubigeo_observed_at on public.weather_records(ubigeo, observed_at desc);
197 create index if not exists idx_frost_predictions_created_at on public.frost_predictions(created_at desc);
198 create index if not exists idx_frost_predictions_user_id on public.frost_predictions(user_id);
199 create index if not exists idx_model_versions_status on public.model_versions(status);
200
201 drop trigger if exists set_users_profile_updated_at on public.users_profile;
202 create trigger set_users_profile_updated_at
203 before update on public.users_profile
204 for each row execute function public.set_updated_at();
205
206 drop trigger if exists set_inei_locations_updated_at on public.inei_locations;
207 create trigger set_inei_locations_updated_at
208 before update on public.inei_locations
209 for each row execute function public.set_updated_at();
210
211 drop trigger if exists set_populated_centers_updated_at on public.populated_centers;
212 create trigger set_populated_centers_updated_at
213 before update on public.populated_centers
214 for each row execute function public.set_updated_at();
215
216 drop trigger if exists set_agricultural_context_updated_at on public.agricultural_context;
217 create trigger set_agricultural_context_updated_at
218 before update on public.agricultural_context
219 for each row execute function public.set_updated_at();
220
221 drop trigger if exists set_training_runs_updated_at on public.training_runs;
222 create trigger set_training_runs_updated_at
223 before update on public.training_runs
224 for each row execute function public.set_updated_at();
225
226 drop trigger if exists set_alerts_updated_at on public.alerts;
227 create trigger set_alerts_updated_at
228 before update on public.alerts
229 for each row execute function public.set_updated_at();

```

supabase/migrations/002_add_feedback_and_observations.sql

```

1| create table if not exists public.prediction_feedback (
2|   id uuid primary key default gen_random_uuid(),
3|   prediction_id uuid references public.frost_predictions(id) on delete set null,
4|   district text not null,
5|   latitude numeric(9,6) check (latitude between -18.5 and -13.0),
6|   longitude numeric(9,6) check (longitude between -71.5 and -68.0),
7|   observed_risk_level text check (observed_risk_level in ('bajo', 'medio', 'alto')),
8|   user_observed_frost boolean,
9|   crop_damage_observed boolean,
10|  notes text,
11|  source text not null default 'app_feedback',
12|  metadata jsonb not null default '{}':jsonb,
13|  created_at timestampz not null default now()
14| );
15|
16| create table if not exists public.official_frost_observations (
17|   id uuid primary key default gen_random_uuid(),
18|   observed_at timestampz not null,
19|   district text not null,
20|   latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
21|   longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
22|   altitude numeric(8,2) check (altitude between 0 and 7000),
23|   temperature_2m numeric(6,2),
24|   relative_humidity_2m numeric(5,2) check (relative_humidity_2m between 0 and 100),
25|   apparent_temperature numeric(6,2),
26|   dew_point_2m numeric(6,2),
27|   precipitation numeric(8,2) check (precipitation is null or precipitation >= 0),
28|   cloud_cover numeric(5,2) check (cloud_cover between 0 and 100),
29|   wind_speed_10m numeric(7,2) check (wind_speed_10m is null or wind_speed_10m >= 0),
30|   observed_risk_level text not null check (observed_risk_level in ('bajo', 'medio', 'alto')),
31|   source_name text not null default 'SENAMHI',
32|   source_url text,
33|   metadata jsonb not null default '{}':jsonb,
34|   created_at timestampz not null default now()
35| );
36|
37| create index if not exists idx_prediction_feedback_created_at on public.prediction_feedback(created_at desc);
38| create index if not exists idx_official_frost_observations_district_time
39|   on public.official_frost_observations(district, observed_at desc);

```

supabase/migrations/20260529201250_apply_rls_policies.sql

```

1| alter table public.users_profile enable row level security;
2| alter table public.inei_locations enable row level security;
3| alter table public.populated_centers enable row level security;
4| alter table public.agricultural_context enable row level security;
5| alter table public.weather_records enable row level security;
6| alter table public.frost_predictions enable row level security;
7| alter table public.alerts enable row level security;
8| alter table public.model_versions enable row level security;
9| alter table public.training_runs enable row level security;
10| alter table public.data_sources_log enable row level security;
11| alter table public.prediction_feedback enable row level security;
12| alter table public.official_frost_observations enable row level security;
13|
14| grant usage on schema public to anon, authenticated, service_role;
15|
16| grant select on public.inei_locations to anon, authenticated;
17| grant select on public.populated_centers to anon, authenticated;
18| grant select on public.agricultural_context to anon, authenticated;
19| grant select on public.model_versions to anon, authenticated;
20| grant select on public.frost_predictions to authenticated;
21|
22| grant select, insert, update, delete on public.users_profile to authenticated, service_role;

```

```

23 grant select, insert, update, delete on public.inei_locations to service_role;
24 grant select, insert, update, delete on public.populated_centers to service_role;
25 grant select, insert, update, delete on public.agricultural_context to service_role;
26 grant select, insert, update, delete on public.weather_records to service_role;
27 grant select, insert, update, delete on public.frost_predictions to service_role;
28 grant select, insert, update, delete on public.alerts to service_role;
29 grant select, insert, update, delete on public.model_versions to service_role;
30 grant select, insert, update, delete on public.training_runs to service_role;
31 grant select, insert, update, delete on public.data_sources_log to service_role;
32 grant select, insert, update, delete on public.prediction_feedback to service_role;
33 grant select, insert, update, delete on public.official_frost_observations to service_role;
34
35 drop policy if exists "Users can read own profile" on public.users_profile;
36 create policy "Users can read own profile"
37 on public.users_profile
38 for select
39 to authenticated
40 using ((select auth.uid()) = user_id);
41
42 drop policy if exists "Users can update own profile" on public.users_profile;
43 create policy "Users can update own profile"
44 on public.users_profile
45 for update
46 to authenticated
47 using ((select auth.uid()) = user_id)
48 with check ((select auth.uid()) = user_id);
49
50 drop policy if exists "Users can insert own profile" on public.users_profile;
51 create policy "Users can insert own profile"
52 on public.users_profile
53 for insert
54 to authenticated
55 with check ((select auth.uid()) = user_id);
56
57 drop policy if exists "Public can read INEI locations" on public.inei_locations;
58 create policy "Public can read INEI locations"
59 on public.inei_locations
60 for select
61 to anon, authenticated
62 using (true);
63
64 drop policy if exists "Public can read populated centers" on public.populated_centers;
65 create policy "Public can read populated centers"
66 on public.populated_centers
67 for select
68 to anon, authenticated
69 using (true);
70
71 drop policy if exists "Public can read agricultural context" on public.agricultural_context;
72 create policy "Public can read agricultural context"
73 on public.agricultural_context
74 for select
75 to anon, authenticated
76 using (true);
77
78 drop policy if exists "Users can read own frost predictions" on public.frost_predictions;
79 create policy "Users can read own frost predictions"
80 on public.frost_predictions
81 for select
82 to authenticated
83 using ((select auth.uid()) = user_id);
84
85 drop policy if exists "Public can read active model versions" on public.model_versions;
86 create policy "Public can read active model versions"
87 on public.model_versions
88 for select
89 to anon, authenticated
90 using (status = 'active');
91
92 -- No anon/authenticated insert policy is defined for frost_predictions,
93 -- prediction_feedback or official_frost_observations in this MVP.
94 -- FastAPI/ML operators write with the server-side service role key, which bypasses RLS.
95 -- Never expose the service role key in Flutter, Flutter Web, Vercel client code, or Logs.

```

supabase/migrations/20260529202739_harden_private_rls_and_functions.sql

```

1 create or replace function public.set_updated_at()
2 returns trigger
3 language plpgsql
4 set search_path = public
5 as $function$
6 begin
7   new.updated_at = now();
8   return new;
9 end;
10 $function$;
11
12 drop policy if exists "Service role manages weather records" on public.weather_records;
13 create policy "Service role manages weather records"
14 on public.weather_records
15 for all
16 to service_role
17 using (true)
18 with check (true);
19
20 drop policy if exists "Service role manages alerts" on public.alerts;
21 create policy "Service role manages alerts"
22 on public.alerts
23 for all
24 to service_role
25 using (true)
26 with check (true);
27
28 drop policy if exists "Service role manages training runs" on public.training_runs;
29 create policy "Service role manages training runs"
30 on public.training_runs
31 for all
32 to service_role
33 using (true)
34 with check (true);
35
36 drop policy if exists "Service role manages data source logs" on public.data_sources_log;
37 create policy "Service role manages data source logs"
38 on public.data_sources_log
39 for all
40 to service_role

```

```

41| using (true)
42| with check (true);
43|
44| drop policy if exists "Service role manages prediction feedback" on public.prediction_feedback;
45| create policy "Service role manages prediction feedback"
46| on public.prediction_feedback
47| for all
48| to service_role
49| using (true)
50| with check (true);
51|
52| drop policy if exists "Service role manages official frost observations" on public.official_frost_observations;
53| create policy "Service role manages official frost observations"
54| on public.official_frost_observations
55| for all
56| to service_role
57| using (true)
58| with check (true);

```

supabase/policies/rls_policies.sql

```

1| alter table public.users_profile enable row level security;
2| alter table public.inei_locations enable row level security;
3| alter table public.populated_centers enable row level security;
4| alter table public.agricultural_context enable row level security;
5| alter table public.weather_records enable row level security;
6| alter table public.frost_predictions enable row level security;
7| alter table public.alerts enable row level security;
8| alter table public.model_versions enable row level security;
9| alter table public.training_runs enable row level security;
10| alter table public.data_sources_log enable row level security;
11| alter table public.prediction_feedback enable row level security;
12| alter table public.official_frost_observations enable row level security;
13|
14| grant usage on schema public to anon, authenticated, service_role;
15|
16| grant select on public.inei_locations to anon, authenticated;
17| grant select on public.populated_centers to anon, authenticated;
18| grant select on public.agricultural_context to anon, authenticated;
19| grant select on public.model_versions to anon, authenticated;
20| grant select on public.frost_predictions to authenticated;
21|
22| grant select, insert, update, delete on public.users_profile to authenticated, service_role;
23| grant select, insert, update, delete on public.inei_locations to service_role;
24| grant select, insert, update, delete on public.populated_centers to service_role;
25| grant select, insert, update, delete on public.agricultural_context to service_role;
26| grant select, insert, update, delete on public.weather_records to service_role;
27| grant select, insert, update, delete on public.frost_predictions to service_role;
28| grant select, insert, update, delete on public.alerts to service_role;
29| grant select, insert, update, delete on public.model_versions to service_role;
30| grant select, insert, update, delete on public.training_runs to service_role;
31| grant select, insert, update, delete on public.data_sources_log to service_role;
32| grant select, insert, update, delete on public.prediction_feedback to service_role;
33| grant select, insert, update, delete on public.official_frost_observations to service_role;
34|
35| drop policy if exists "Users can read own profile" on public.users_profile;
36| create policy "Users can read own profile"
37| on public.users_profile
38| for select
39| to authenticated
40| using ((select auth.uid()) = user_id);
41|
42| drop policy if exists "Users can update own profile" on public.users_profile;
43| create policy "Users can update own profile"
44| on public.users_profile
45| for update
46| to authenticated
47| using ((select auth.uid()) = user_id)
48| with check ((select auth.uid()) = user_id);
49|
50| drop policy if exists "Users can insert own profile" on public.users_profile;
51| create policy "Users can insert own profile"
52| on public.users_profile
53| for insert
54| to authenticated
55| with check ((select auth.uid()) = user_id);
56|
57| drop policy if exists "Public can read INEI locations" on public.inei_locations;
58| create policy "Public can read INEI locations"
59| on public.inei_locations
60| for select
61| to anon, authenticated
62| using (true);
63|
64| drop policy if exists "Public can read populated centers" on public.populated_centers;
65| create policy "Public can read populated centers"
66| on public.populated_centers
67| for select
68| to anon, authenticated
69| using (true);
70|
71| drop policy if exists "Public can read agricultural context" on public.agricultural_context;
72| create policy "Public can read agricultural context"
73| on public.agricultural_context
74| for select
75| to anon, authenticated
76| using (true);
77|
78| drop policy if exists "Users can read own frost predictions" on public.frost_predictions;
79| create policy "Users can read own frost predictions"
80| on public.frost_predictions
81| for select
82| to authenticated
83| using ((select auth.uid()) = user_id);
84|
85| drop policy if exists "Public can read active model versions" on public.model_versions;
86| create policy "Public can read active model versions"
87| on public.model_versions
88| for select
89| to anon, authenticated
90| using (status = 'active');
91|
92| -- No anon/authenticated insert policy is defined for frost_predictions in this MVP.
93| -- No anon/authenticated insert policy is defined for frost_predictions,
94| -- prediction_feedback or official_frost_observations in this MVP.
95| -- FastAPI/ML operators write with the server-side service role key, which bypasses RLS.

```



```
96 -- Never expose the service role key in Flutter, Flutter Web, Vercel client code, or Logs.
```

supabase/seed/seed_locations.sql

```
1 | insert into public.inei_locations (
2 | ubigeo,
3 | department,
4 | province,
5 | district,
6 | latitude,
7 | longitude,
8 | estimated_altitude,
9 | total_population,
10 | rural_population,
11 | rural_percentage,
12 | source,
13 | metadata
14 | ) values
15 | ('210101', 'Puno', 'Puno', 'Puno', -15.840200, -70.021900, 3827, 128637, 21868, 16.99, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before
16 | production."}':jsonb),
17 | ('210201', 'Puno', 'Azangaro', 'Azangaro', -14.908400, -70.196200, 3859, 28137, 15194, 54.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before
18 | production."}':jsonb),
19 | ('210301', 'Puno', 'Carabaya', 'Macusani', -14.069700, -70.431100, 4315, 12240, 4651, 38.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before
20 | production."}':jsonb),
21 | ('210401', 'Puno', 'Chucuito', 'Juli', -16.212200, -69.459600, 3869, 22310, 12717, 57.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before
22 | production."}':jsonb),
23 | ('210501', 'Puno', 'El Collao', 'Ilave', -16.083300, -69.638900, 3850, 50239, 34163, 68.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before
24 | production."}':jsonb),
25 | ('211001', 'Puno', 'San Roman', 'Juliaca', -15.499700, -70.133300, 3825, 276110, 22889, 8.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before
26 | production."}':jsonb)
27 |
28 | on conflict (ubigeo) do update set
29 | department = excluded.department,
30 | province = excluded.province,
31 | district = excluded.district,
32 | latitude = excluded.latitude,
33 | longitude = excluded.longitude,
34 | estimated_altitude = excluded.estimated_altitude,
35 | total_population = excluded.total_population,
36 | rural_population = excluded.rural_population,
37 | rural_percentage = excluded.rural_percentage,
38 | source = excluded.source,
39 | metadata = excluded.metadata;
40 |
41 | insert into public.model_versions (
42 | version,
43 | model_name,
44 | algorithm,
45 | artifact_path,
46 | accuracy,
47 | precision_macro,
48 | recall_macro,
49 | f1_macro,
50 | dataset_size,
51 | status,
52 | deployed_at,
53 | metadata
54 | ) values (
55 | 'v0.1.0',
56 | 'RandomForestClassifier',
57 | 'RandomForestClassifier',
58 | 'ml_pipeline/registry/frost_risk_model.joblib',
59 | 1.000000,
60 | 1.000000,
61 | 1.000000,
62 | 1.000000,
63 | 4368,
64 | 'active',
65 | now(),
66 | '{"limitations":["Initial labels are rule-based.", "Metrics are optimistic because label-derived features are included."], "sources":["Open-Meteo", "INEI-compatible MVP seed"]}'::jsonb
67 | )
68 |
69 | on conflict (version) do update set
70 | model_name = excluded.model_name,
71 | algorithm = excluded.algorithm,
72 | artifact_path = excluded.artifact_path,
73 | accuracy = excluded.accuracy,
74 | precision_macro = excluded.precision_macro,
75 | recall_macro = excluded.recall_macro,
76 | f1_macro = excluded.f1_macro,
77 | dataset_size = excluded.dataset_size,
78 | status = excluded.status,
79 | deployed_at = excluded.deployed_at,
80 | metadata = excluded.metadata;
```